Scrollutil

Programmierhandbuch (deutsch)

2026-10-25

Inhalt

index **3**
pagesman **3**
plainnotebook **10**
scrollableframe **23**
scrollarea **38**
scrollednotebook **46**
scrollsync **59**
scrollutil **65**
wheelEvent **117**
Lizenz **126**

index

Das Scrollutil-Paket 2.11

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

Scrollutil-Programmierhandbuch
Die Befehle scrollutil::scrollarea und scrollutil::getscrollarea
Die Befehle scrollutil::scrollsync und scrollutil::getscrollsync
Die Befehle scrollutil::scrollableframe und scrollutil::adaptBtn2EventHandling
Der Befehl scrollutil::scrollednotebook und das Stilelement "closetab"
Der Befehl scrollutil::plainnotebook
Der Befehl scrollutil::pagesman
Befehle im Zusammenhang mit dem Mausrad und der Ereignisbehandlung für <TouchpadScroll>

Stichwörter: index

pagesman

Der Befehl `scrollutil::pagesman`

Für Scrollutil Version 2.11

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

- Kurzanleitung
- Ausführliche Referenz

Kurzreferenz

NAME

`scrollutil::pagesman` – Seiten-Widgets erstellen und bearbeiten

ÜBERSICHT

```
scrollutil::pagesman pathName ?options?
```

STANDARDOPTIONEN

```
-background   -highlightbackground  -relief
-borderwidth  -highlightcolor
-cursor       -highlightthickness
```

WIDGET-SPEZIFISCHE OPTIONEN

```
  -forgetcommand command

  -height screenDistance

  -leavecommand command

  -takefocus 0|1|""|command

  -width screenDistance
```

SEITE KONFIGURATION OPTIONS

```
  -padding padding

  -sticky stickyness
```

Widget COMMAND

```
  pathName add window ?option value option value ...?

  pathName adjustsize

  pathName attrib ?name ?value name value ...??

  pathName cget option

  pathName configure ?option ?value option value ...??

  pathName forget pageIndex

  pathName hasattrib name

  pathName haspageattrib pageIndex name

  pathName index window

  pathName insert pageIndex window ?option value option value ...?

  pathName pageattrib pageIndex ?name ?value name value ...??

  pathName pagecget pageIndex option

  pathName pageconfigure pageIndex ?option ?value option value ...??

  pathName pages
```

```
pathName select ?pageIndex?

pathName size

pathName unsetattrib name

pathName unsetpageattrib pageIndex name

pathName window pageIndex
```

SCHLÜSSELWÖRTER

pagesman, widget

Ausführliche Referenz

NAME

`scrollutil::pagesman` – Seiten-Widgets erstellen und bearbeiten

ÜBERSICHT

```
scrollutil::pagesman pathName ?options?
```

BESCHREIBUNG

Der Befehl `scrollutil::pagesman` erstellt ein neues Fenster mit dem Namen `pathName` und der Klasse `Pagesman` und wandelt es in ein **pagesman**-Widget um. Zusätzliche Optionen, die im Folgenden beschrieben werden, können über die Befehlszeile oder in der Optionsdatenbank festgelegt werden, um Aspekte des pagesman-Widgets wie Breite, Höhe, Rahmenbreite und Relief zu konfigurieren. Der Befehl `scrollutil::pagesman` gibt sein Argument `pathName` zurück. Zum Zeitpunkt des Aufrufs dieses Befehls darf kein Fenster mit dem Namen `pathName` existieren, aber das übergeordnete Fenster von `pathName` muss vorhanden sein.

Ein "Pagesman" ist ein Megawidget, das die Grundfunktionalität eines Seitenmanagers bereitstellt, d. h., es verwaltet eine Liste von Fenstern, die als **Seiten** bezeichnet werden, von denen jeweils nur eine sichtbar ist. Durch die Verwendung von plainnotebook-Widgets als Seiten ist es recht einfach, Anwendungen zu schreiben, in denen der Benutzer mit einem einzigen Mausklick von einem "plainnotebook" zu einem anderen wechseln und auf dieselbe Weise wieder zum ursprünglichen zurückkehren kann. Alles, was für diese Navigation benötigt wird, wird durch entsprechende Optionen und Unterbefehle des "plainnotebook"-Widgets bereitgestellt.

STANDARDOPTIONEN

```
-background   -highlightbackground  -relief
-borderwidth  -highlightcolor
-cursor       -highlightthickness
```

Weitere Informationen zu den Standardoptionen für Tk-Widgets finden Sie im Handbucheintrag zu den **options**. Die Optionen `-background`, `-highlightbackground`,

`-highlightcolor` und `-highlightthickness` werden nur vom Scrollutil-Paket unterstützt, nicht jedoch von Scrollutil_tile. Sie haben dieselben Standardwerte wie die gleichnamigen Optionen für Tk-Frame-Widgets. Die Standardwerte der übrigen Standardoptionen lauten:

```
-borderwidth 0 -cursor "" -relief flat
```

WIDGET-SPEZIFISCHE OPTIONEN

Befehlszeilenname:	-forgetcommand
Name der Datenbank:	forgetCommand
Datenbankklasse:	ForgetCommand

Legt einen Befehl fest, der aufgerufen werden soll, wenn versucht wird, ein Fenster mithilfe des Unterbefehls `forget` aus der Verwaltung zu entfernen. Wenn der Wert der Option eine nicht leere Zeichenkette ist, wird diese mit dem Pfadnamen des "pagesman"-Widgets und dem des Fensters, dessen Verwaltung aufgehoben werden soll, verkettet, und das resultierende Skript, das einen booleschen Wert geben muss, wird im globalen Geltungsbereich ausgewertet. Wenn der Wert dieses Skripts "false" ist, wird der Unterbefehl `forget` abgebrochen, d. h., das Fenster wird nicht aus der Verwaltung entfernt. Der Standardwert ist eine leere Zeichenkette.

Befehlszeilenname:	-height
Name der Datenbank:	height
Datenbankklasse:	Height

Falls vorhanden und größer als Null, legt dieser Wert die gewünschte Gesamthöhe des Widgets fest, einschließlich des Rahmens und des Hervorhebungsrechtecks (falls vorhanden), die um das Widget herum gezeichnet werden, und zwar in einem der von `Tk_GetPixels` akzeptierten Formate. Andernfalls wird die maximal angeforderte Höhe aller Seiten verwendet. Der Standardwert ist `0`.

Befehlszeilenname:	-leavecommand
Name der Datenbank:	leaveCommand
Datenbankklasse:	LeaveCommand

Legt einen Befehl fest, der aufgerufen werden soll, wenn versucht wird, das aktuell ausgewählte Fenster zu verlassen, indem mithilfe des Unterbefehls `select` ein anderes Fenster ausgewählt wird. Wenn der Wert der Option eine nicht leere Zeichenkette ist, wird diese mit dem Pfadnamen des "pagesman"-Widgets und dem des aktuell ausgewählten Fensters verkettet, und das resultierende Skript, das einen booleschen Wert zurückgeben muss, wird im globalen Geltungsbereich ausgewertet. Wenn der Wert dieses Skripts "false" ist, wird der Vorgang der Auswahl eines anderen Fensters abgebrochen. Der Standardwert ist eine leere Zeichenkette.

Befehlszeilenname:	-takefocus
Name der Datenbank:	takeFocus
Datenbankklasse:	TakeFocus

Diese Option legt fest, ob das Pagesman-Widget während der Tastaturnavigation den Fokus annimmt. Sie ist nahezu identisch mit der gleichnamigen Standardoption (Einzelheiten finden Sie im Handbucheintrag **options**). Der einzige Unterschied besteht darin, dass nicht das "pagesman"-Widget selbst, sondern das aktuell ausgewählte Fenster (sofern vorhanden) den Fokus erhält, wenn die Standardtasten (`Tab` und `Shift-Tab`) zur Tastaturnavigation verwendet werden. Der Standardwert ist eine leere Zeichenkette.

Befehlszeilenname:	-width
Name der Datenbank:	width
Datenbankklasse:	Width

Wenn vorhanden und größer als Null, legt dieser Wert die gewünschte Gesamtbreite des Widgets fest, einschließlich des Rahmens und des Hervorhebungsrechtecks (falls vorhanden), die um das Widget herum gezeichnet werden, und zwar in einer der für `Tk_GetPixels` zulässigen Formen. Andernfalls wird die maximal angeforderte Breite aller Seiten verwendet. Der Standardwert ist `0`.

SEITENKONFIGURATIONSOPTIONEN

Die folgenden Optionen werden derzeit von den Befehlen `add`, `insert`, `pagecget` und `pageconfigure` unterstützt:

`-padding padding`

Legt fest, wie viel zusätzlicher Abstand zwischen dem Widget und dieser Seite eingefügt werden soll. `padding` muss eine Liste mit bis zu vier nicht-negativen Bildschirmabständen `{left top right bottom}` sein. Wenn weniger als vier Elemente festgelegt werden, wird `bottom` standardmäßig auf `top` gesetzt, `right` auf `left` und `top` auf `left`. Ein als leere Liste festgelegter Innenabstand entspricht `0`, was dem Standardwert entspricht.

`-sticky stickyness`

Legt fest, wie das Seitenfenster innerhalb des Widgets positioniert wird. `stickyness` muss eine Zeichenkette sein, die null oder mehr der Zeichen `n`, `s`, `e` oder `w` enthalten. Jeder Buchstabe bezieht sich auf eine Seite (Norden, Süden, Osten oder Westen), an der das Seitenfenster gemäß dem Geometriemanager `grid` "haften" wird. Die Standard-Haftung ist `nsew`.

WIDGET-BEFEHL

Der Befehl `scrollutil::pagesman` erstellt einen neuen Tcl-Befehl mit dem Namen `pathName`. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget auszuführen. Er hat die folgende allgemeine Form:

```
pathName option ?arg arg ...?
```

`option` und die `args` bestimmen das genaue Verhalten des Befehls. In den folgenden Beschreibungen der Unterbefehle bezeichnet `pageIndex` stets einen numerischen Index der Seiten, der – mit Ausnahme des Unterbefehls `insert` – größer oder gleich Null und kleiner als die Seitenanzahl sein muss. Für Pagesman-Widgets sind die folgenden Unterbefehle möglich:

`pathName add window ?option value option value ...?`
Fügt dem "pagesman"-Widget eine neue Seite hinzu, die aus dem angegebenen Fenster besteht. Eine Liste der verfügbaren Optionen finden Sie im Abschnitt SEITENKONFIGURATIONSOPTIONEN für eine Liste der verfügbaren Optionen. Wenn `window` bereits vom "pagesman"-Widget verwaltet wird, verwendet der Unterbefehl seine optionalen Argumente, um die Optionen der entsprechenden Seite zu aktualisieren.

`pathName adjustsize`
Setzt die Option `-height` des Pagesman-Widgets auf die maximale angeforderte Höhe aller Seiten und die Option `-width` auf die maximale angeforderte Breite aller Seiten.

HINWEIS: Wenn alle Seiten des "pagesman"-Widgets "plainnotebooks" sind, deren Optionen `-height` und `-width` den Standardwert `0` haben, ist es in vielen Fällen nicht erforderlich, einem Aufruf dieser Unterbefehl für das "pagesman"-Widget durch Aufrufe des gleichen Namens für dessen Seiten voranzustellen, da jedes "plainnotebook" seine angeforderte Größe automatisch an die maximal angeforderte Größe seiner Teilflächen anpasst. Beispielsweise könnte eine Änderung der Schriftart in einigen Teilflächen bestimmter Seiten es erforderlich machen, diesen Unterbefehl für das "pagesman"-Widget aufzurufen, und zum Zeitpunkt des Aufrufs werden die angeforderten Größen seiner Seiten bereits automatisch an die neue Schriftart angepasst sein.

`pathName attrib ?name ?value name value ...??`
Fragt die Attribute des Widgets ab oder ändert sie. Wenn kein `name` angegeben ist, gibt der Befehl eine Liste von Paaren zurück, von denen jedes den Namen und den Wert eines Attributs für `pathName` enthält. Wenn `name` ohne `value` festgelegt wird, gibt der Befehl den Wert des genannten Attributs zurück oder eine leere Zeichenkette, falls kein entsprechender Wert vorhanden ist (mit dem Unterbefehl `hasattrib` können Sie diesen Fall von dem unterscheiden, bei dem der Wert eines *vorhandenen* Attributs eine leere Zeichenkette ist). Wenn ein oder mehrere Paare aus `name` und `value` angegeben werden, legt der Befehl das/die angegebene(n) Widget-Attribut(e) auf den/die angegebenen Wert(e) fest; in diesem Fall ist der Rückgabewert eine leere Zeichenkette. Jedes `name` kann eine beliebige Zeichenkette sein.

`pathName cget option`
Gibt den aktuellen Wert der durch `option` angegebenen Konfigurationsoption zurück, die einen der vom Befehl `scrollutil::pagesman` akzeptierten Werte annehmen kann.

`pathName configure ?option ?value option value ...??`
Ruft die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn kein `option` angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für `pathName` beschreibt (Informationen zum Format dieser Liste finden Sie unter `Tk_ConfigureInfo`). Wenn `option` ohne `value` festgelegt wird, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Unterliste des Werts, der zurückgegeben wird, wenn kein `option` angegeben wurde). Wenn ein oder

mehrere Paare `option–value` festgelegt werden, ändert der Befehl die festgelegten Widget-Optionen so, dass sie die festgelegten Werte annehmen; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. `option` kann jeden der vom Befehl `scrollutil::pagesman` akzeptierten Werte annehmen.

`pathName forget pageIndex`
Hebt die Zuordnung und Verwaltung der durch `pageIndex` identifizierten Seite auf. Der Unterbefehl gibt `0` zurück, wenn er durch den Befehl abgebrochen wurde, der als Wert der Option `-forgetcommand` festgelegt wurde, andernfalls `1`.

`pathName hasattrib name`
Gibt `1` zurück, wenn das Attribut `name` vorhanden ist, andernfalls `0`.

`pathName haspageattrib pageIndex name`
Gibt `1` zurück, wenn das Attribut `name` für die durch `pageIndex` angegebene Seite vorhanden ist, andernfalls `0`.

`pathName index window`
Gibt den numerischen Index der Seite zurück, die durch das angegebene Fenster identifiziert wird. Dieses Fenster muss zuvor mithilfe des Unterbefehls `add` oder `insert` zum Widget hinzugefügt worden sein.

`pathName insert pageIndex window ?option value option value ...?`
Fügt eine neue Seite ein, die aus dem angegebenen Fenster besteht, und zwar an der durch `pageIndex` angegebenen Position. Eine Liste der verfügbaren Optionen finden Sie im Abschnitt SEITENKONFIGURATIONSOPTIONEN. Wenn `window` bereits vom "pagesman"-Widget verwaltet wird, verschiebt der Unterbefehl es an die angegebene Position und verwendet seine optionalen Argumente, um die Optionen der entsprechenden Seite zu aktualisieren. Wenn `pageIndex` kleiner oder gleich Null ist, wird das Fenster an den Anfang der Liste der Seiten eingefügt; wenn es größer oder gleich der Anzahl der Seiten ist, wird das Fenster an das Ende der Liste der Seiten angehängt.

`pathName pageattrib pageIndex ?name ?value name value ...??`
Fragt die Attribute der durch `pageIndex` angegebenen Seite ab oder ändert sie. Wenn kein `name` gelegt ist, gibt der Befehl eine Liste von Paaren zurück, von denen jedes den Namen und den Wert eines Attributs der Seite enthält. Wenn `name` ohne `value` festgelegt wird, gibt der Befehl den Wert des einen benannten Seitenattributs zurück oder eine leere Zeichenkette, falls kein entsprechender Wert vorhanden ist (Sie können den Unterbefehl `haspageattrib` verwenden, um diesen Fall von dem zu unterscheiden, bei dem der Wert eines *vorhandenen* Seitenattributs eine leere Zeichenkette ist). Wenn ein oder mehrere Paare aus `name` und `value` angegeben werden, legt der Befehl das/die angegebene(n) Seitenattribut(e) auf den/die angegebenen Wert(e) fest; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. Jedes `name` kann eine beliebige Zeichenkette sein.

`pathName pagecget pageIndex option`
Gibt den aktuellen Wert der durch `option` angegebenen Seitenkonfigurationsoption für die durch `pageIndex` angegebene Seite zurück. `option` kann einen beliebigen Wert annehmen, der vom Befehl `pageconfigure` akzeptiert wird.

`pathName pageconfigure pageIndex ?option ?value option value ...??`
Ruft die Konfigurationsoptionen der durch `pageIndex` angegebenen Seite ab oder ändert diese. Wenn kein `option` angegeben ist, gibt der Befehl eine Liste zurück, die alle für die Seite verfügbaren Optionen beschreibt (Informationen zum Format dieser Liste finden Sie

unter `Tk_ConfigureInfo`). Wenn `option` ohne `value` festgelegt wird, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste entspricht der entsprechenden Unterliste des Werts, der zurückgegeben wird, wenn kein `option` festgelegt wurde). Wenn ein oder mehrere Paare aus `option` und `value` festgelegt werden, ändert der Befehl die festgelegten Seitenoptionen so, dass sie die festgelegten Werte annehmen; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. `option` kann einen beliebigen der im Abschnitt SEITENKONFIGURATIONSOPTIONEN beschriebenen Werte annehmen.

`pathName pages`
Gibt die Liste aller vom "pagesman"-Widget verwalteten Fenster in der Reihenfolge ihrer numerischen Seitenindizes zurück.

`pathName select ?pageIndex?`
Wenn `pageIndex` weggelassen wird, gibt der Unterbefehl den Pfadnamen des aktuell ausgewählten Fensters zurück. Andernfalls wählt er die angegebene Seite aus, indem er das entsprechende Fenster zuordnet und die Zuordnung des zuvor ausgewählten Fensters (sofern es sich um ein anderes handelt) aufhebt. In diesem Fall gibt der Befehl, der als Wert der Option `-leavecommand` angegeben wurde, den Wert `0` zurück, wenn der Vorgang der Auswahl eines anderen Fensters abgebrochen wurde, und andernfalls den Wert `1`.

`pathName size`
Gibt die Anzahl der Seiten zurück, d. h. die Anzahl der Fenster, die vom "pagesman"-Widget verwaltet werden.

`pathName unsetattrib name`
Hebt das Attribut `name` auf. Gibt eine leere Zeichenfolge zurück.

`pathName unsetpageattrib pageIndex name`
Hebt das Attribut `name` für die durch `pageIndex` angegebene Seite auf. Gibt eine leere Zeichenkette zurück.

`pathName window pageIndex`
Gibt den Pfadnamen des Seitenfensters zurück, das durch `pageIndex` identifiziert wird.

VIRTUELLE EREIGNISSE
Nachdem eine neue Seite ausgewählt wurde, generiert das Widget ein virtuelles Ereignis mit der Bezeichnung `<<PagesmanPageChanged>>`.

VORGABEBINDUNGEN
Wenn ein neues "pagesman"-Widget erstellt wird, verfügt es über keine Standard-Ereignis-Bindungen: "pagesman"-Widgets sind nicht dafür vorgesehen, interaktiv zu sein.

SCHLÜSSELWÖRTER
pagesman, pages manager, widget

Stichwörter: pagesman

plainnotebook

Der Befehl `scrollutil::plainnotebook`

Für Scrollutil Version 2.11

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

- Kurzanleitung
- Ausführliche Referenz

Kurzreferenz

NAME

`scrollutil::plainnotebook` – Erstellen und Bearbeiten von PlainNotebook-Widgets

ÜBERSICHT

```
package require scrollutil_tile
scrollutil::plainnotebook pathName ?options?
```

STANDARDOPTIONEN

```
-class  -cursor  -style
```

WIDGET-SPEZIFISCHE OPTIONEN

```
-caller pageIndex

-closabletabs boolean

-forgetcommand command

-height screenDistance

-leavecommand command

-movabletabs boolean

-side left|right

-takefocus 0|1|""|command

-title text

-width screenDistance
```

Widget COMMAND

```
pathName add window ?options?

pathName addbutton pageIndex text ?image?

pathName addlabel text ?image?
```

```
pathName addseparator

pathName adjustsize

pathName attrib ?name ?value name value ...??

pathName cget option

pathName closablestate tabId ?boolean?

pathName configure ?option ?value option value ...??

pathName forget tabId

pathName hasattrib name

pathName hastabattrib tabId name

pathName hide tabId

pathName index tabId

pathName insert pos window ?options?

pathName insertbutton pos pageIndex text ?image?

pathName insertlabel pos text ?image?

pathName insertseparator pos

pathName instate stateSpec ?script?

pathName see tabId

pathName select ?tabId?

pathName state ?stateSpec?

pathName style

pathName tab tabId ?option ?value option value ...??

pathName tabattrib tabId ?name ?value name value ...??

pathName tabpath tabId
```

```
pathName tabs

pathName titlepath

pathName unsetattrib name

pathName unsettabattrib tabId name
```

SCHLÜSSELWÖRTER

plainnotebook, widget, notebook, closebtn

Ausführliche Referenz

NAME

`scrollutil::plainnotebook` – Erstellen und Bearbeiten von PlainNotebook-Widgets

ÜBERSICHT

```
package require scrollutil_tile
scrollutil::plainnotebook pathName ?options?
```

HINWEIS: Der Befehl `scrollutil::plainnotebook` wird vom Paket "Scrollutil_tile" bereitgestellt, nicht jedoch von Scrollutil.

BESCHREIBUNG

Der Befehl `scrollutil::plainnotebook` erstellt ein neues Fenster mit dem Namen `pathName` und wandelt es in ein **plainnotebook**-Widget um. Zusätzliche Optionen, die im Folgenden beschrieben werden, können über die Befehlszeile oder in der Optionsdatenbank festgelegt werden, um Aspekte des "plainnotebook"-Widgets wie Breite, Höhe und Seite zu konfigurieren. Der Befehl `scrollutil::plainnotebook` gibt sein Argument `pathName` zurück. Zum Zeitpunkt des Aufrufs dieses Befehls darf kein Fenster mit dem Namen `pathName` existieren, aber das übergeordnete Fenster von `pathName` muss vorhanden sein.

Ein "plainnotebook" ist ein Megawidget, das ein ttk::notebook mit einer beliebigen Anzahl von Seiten und unsichtbaren Registerkarten um ein ttk::frame links oder rechts davon erweitert, das unter anderem ein scrollableframe, dessen Inhaltsrahmen das übergeordnete Element einer Reihe von Widgets ist, die die Rolle vertikal angeordneter Notizbuchregisterkarten übernehmen. Derzeit nicht sichtbare "Registerkarten" können durch Navigation mit dem Mausrad, dem Touchpad oder der Tastatur sowie durch Scrollen mit der vertikalen Bildlaufleiste des scrollarea, der den scrollableframe enthält. Ein einzelner "Reiter" kann programmgesteuert mithilfe des Unterbefehls `see` sichtbar gemacht werden. Im Gegensatz zu einem ttk::notebook-Widget mit vertikal ausgerichteten Registerkarten, das in den meisten Themen ein suboptimales Erscheinungsbild aufweist und dessen `-height`-Option häufig durch die Gesamthöhe der Registerkarten überschrieben wird, beachtet das "plainnotebook"-Widget den Wert seiner Option `-height`, unabhängig vom Platzbedarf der "Registerkarten".

Im weiteren Verlauf dieses Dokuments werden die Widgets, die als vertikal angeordnete Notizbuchregisterkarten dienen, selbst als **Registerkarten** bezeichnet.

Der obere Teil des Rahmens links oder rechts neben der Komponente ttk::notebook enthält eine Schaltfläche und ein ttk::label-Widget, die als **Aufwärts-Schaltfläche** bzw. **Titelbeschriftung** bezeichnet werden. Ihre Sichtbarkeit wird durch die Widget-Optionen `-caller` und `-title` gesteuert. Die Aufwärts-Schaltfläche zeigt ein Bild an, dessen Form der eines <-Zeichens ähnelt.

Die API des "plainnotebook"-Widgets ähnelt stark der von ttk::notebook. Abgesehen von einigen zusätzlichen, plainnotebook-spezifischen Unterbefehlen und dem fehlenden Unterbefehl `identify` bietet der mit einem "plainnotebook"-Widget verknüpfte Tcl-Befehl dieselben Unterbefehle wie der mit einem ttk::notebook verknüpfte. Beachten Sie, dass die API keinen Befehl zum Abrufen des Pfadnamens des in einem "plainnotebook" enthaltenen "ttk::notebook"-Widgets bereitstellt, da kein direkter Zugriff auf dieses Widget erforderlich ist. Darüber hinaus führt die Verwendung seiner Optionen oder Unterbefehle anstelle derjenigen des "plainnotebook"-Widgets in den meisten Fällen zu unvorhersehbarem Verhalten.

Beachten Sie außerdem, dass nur die mit den Unterbefehlen `add` und `insert` erstellten Registerkarten auswählbar sind. Die mit den plainnotebook-spezifischen Unterbefehlen `addbutton` und `insertbutton` erstellten Registerkarten dienen dazu, zu einem anderen plainnotebook-Widget zu wechseln, während die mit `addlabel`, `addseparator`, `insertlabel` und `insertseparator` erstellt wurden, können als Titel und Trennzeichen für Registerkartengruppen verwendet werden.

STANDARDOPTIONEN

```
-class  -cursor  -style
```

Einzelheiten zu den Standardoptionen finden Sie im Handbucheintrag zu **ttk_widget**. Der Wert der Option `-class` lautet `Plainnotebook`, der der Option `-cursor` ist eine leere Zeichenkette. Die Option `-style`, deren Wert `Plain.TNotebook` ist, bezieht sich auf den Stil des im "plainnotebook" enthaltenen Widgets ttk::notebook. Das "schlichte" Erscheinungsbild dieses ttk::notebook-Widgets wird erreicht, indem das Layout des Stils `Plain.TNotebook.Tab` auf `null` und die Optionen `-mintabwidth`, `-tabmargins` und `-padding` des Stils `Plain.TNotebook` auf `0` gesetzt werden.

WIDGET-SPEZIFISCHE OPTIONEN

Befehlszeilenname:	-caller
Name der Datenbank:	caller
Datenbankklasse:	Caller

Der Wert dieser Option muss eine ganze Zahl sein. Ist er nicht negativ, muss das übergeordnete Widget des Widgets ein pagesman-Widget sein. In diesem Fall wird der Wert als numerischer Index einer bereits vorhandenen Seite des übergeordneten pagesman-Widgets interpretiert, und die Schaltfläche "Nach oben" des "plainnotebook" wählt diese Seite aus, die als "Aufrufer" des aktuellen "plainnotebook"-Widgets angesehen wird – das heißt, das aktuelle "plainnotebook" wurde zuvor von dieser Seite aus ausgewählt. Wenn der Wert der Option beispielsweise `0`

lautet, führt das Drücken und anschließende Loslassen der linken Maustaste über der "Auf"-Schaltfläche dazu, dass Sie zur pagesman-Seite mit dem Index `0` zurückwechselt, von der aus der Benutzer vermutlich zuvor zur aktuellen Seite hinabgestiegen ist. Der Standardwert ist `-1`.

Befehlszeilenname:	-closabletabs
Name der Datenbank:	closableTabs
Datenbankklasse:	ClosableTabs

Legt einen booleschen Wert fest, der bestimmt, ob die (ausschließlich) mit den Unterbefehlen `add` und `insert` erstellten Registerkarten das Element `closebtn` enthalten. Dieses Stilelement wird automatisch vom Paket "Scrollutil_tile" erstellt und ähnelt stark dem Element `closetab`. Siehe den Abschnitt WEITERE BINDUNGEN weiter unten für Details zum Schließen der Registerkarten mithilfe des Elements `closebtn`. Der Standardwert ist `0`. Diese Option kann für einzelne Registerkarten mithilfe des Unterbefehls `closablestate` überschrieben werden.

Befehlszeilenname:	-forgetcommand
Name der Datenbank:	forgetCommand
Datenbankklasse:	ForgetCommand

Legt einen Befehl fest, der aufgerufen werden soll, wenn versucht wird, eine Registerkarte zu entfernen und das zugehörige Fenster mithilfe des Unterbefehls `forget` aus der Verwaltung zu nehmen. Wenn der Wert der Option eine nicht leere Zeichenkette ist, wird diese mit dem Pfadnamen des "plainnotebook"-Widgets und dem des Fensters, dessen Verwaltung aufgehoben werden soll, verkettet, und das resultierende Skript, das einen booleschen Wert geben muss, wird im globalen Geltungsbereich ausgewertet. Wenn der Wert des Skripts "false" ist, wird der Unterbefehl `forget` abgebrochen, d. h., die Registerkarte wird nicht entfernt und das zugehörige Fenster wird nicht aus der Verwaltung entlassen. Der Standardwert ist eine leere Zeichenkette.

Befehlszeilenname:	-height
Name der Datenbank:	height
Datenbankklasse:	Height

Falls vorhanden und größer als Null, legt dieser Wert die gewünschte Höhe der Teilfläche des Fensters in einem der von `Tk_GetPixels` akzeptierten Formate fest. Andernfalls wird die maximal angeforderte Höhe aller Teilflächen verwendet. Der Standardwert ist `0`.

Befehlszeilenname:	-leavecommand

Name der Datenbank:	leaveCommand
Datenbankklasse:	LeaveCommand

Legt einen Befehl fest, der aufgerufen werden soll, wenn versucht wird, das aktuell ausgewählte Fenster zu verlassen, indem eine andere Auswahl getroffen wird. Wenn der Wert der Option eine nicht leere Zeichenkette ist, wird diese mit dem Pfadnamen des "plainnotebook"-Widgets und dem des aktuell ausgewählten Fensters verkettet, und das resultierende Skript, das einen booleschen Wert zurückgeben muss, wird im globalen Geltungsbereich ausgewertet. Wenn der Wert dieses Skripts "false" ist, wird der Vorgang der Auswahl eines anderen Fensters abgebrochen. Der Standardwert ist eine leere Zeichenkette.

Befehlszeilenname:	-movabletabs
Name der Datenbank:	movableTabs
Datenbankklasse:	MovableTabs

Legt einen booleschen Wert fest, der bestimmt, ob die Registerkarten mit der Maus verschoben werden können. Weitere Informationen finden Sie im Abschnitt WEITERE BINDUNGEN weiter unten. Der Standardwert ist `1`.

Befehlszeilenname:	-side
Name der Datenbank:	side
Datenbankklasse:	Side

Legt fest, auf welcher Seite (`left` oder `right`) des Widgets das Element ttk::frame angezeigt werden soll, das die Registerkarten, die Schaltfläche "Aufsteigend" und die Titelbeschriftung enthält. Die Standardeinstellung ist `left`.

Befehlszeilenname:	-takefocus
Name der Datenbank:	takeFocus
Datenbankklasse:	TakeFocus

Diese Option legt fest, ob das "plainnotebook"-Widget während der Tastaturnavigation den Fokus annimmt. Sie ist nahezu identisch mit der gleichnamigen Standardoption (Einzelheiten finden Sie im Handbucheintrag **options**). Der einzige Unterschied besteht darin, dass nicht das "plainnotebook"-Widget selbst, sondern der Inhaltsrahmen des darin enthaltenen "scrollableframe" den Fokus erhält, wenn die Standardtasten (`Tab` und `Shift-Tab`) bei der Tastaturnavigation betätigt werden. Die Standardeinstellung ist `"ttk::takefocus"` (genau wie bei ttk::notebook und einigen anderen Widgets im Tk-Thema).

Befehlszeilenname:	-title
Name der Datenbank:	title
Datenbankklasse:	Title

Legt den Text fest, der in der Titelbeschriftung des "plainnotebook" angezeigt werden soll, bei dem es sich um ein ttk::label-Widget im Stil `PnbTitle.TLabel` handelt. Die für diesen Stil verwendete Schriftart leitet sich von `TkDefaultFont` ab, wobei deren Größe um den Faktor 1,2 vergrößert wurde. Um eine andere Schriftart festzulegen, verwenden Sie den Befehl `ttk::style` oder ermitteln Sie den Pfadnamen dieses Widgets mit dem Unterbefehl `titlepath` und rufen Sie anschließend `... configure -font ...` auf.

Befehlszeilenname:	-width
Name der Datenbank:	width
Datenbankklasse:	Width

Falls vorhanden und größer als Null, legt dieser Wert die gewünschte Breite der Teilflächen im Fensterbereich in einem der von `Tk_GetPixels` akzeptierten Formate fest. Andernfalls wird die maximal angeforderte Breite aller Teilflächen verwendet. Der Standardwert ist `0`.

REGISTEROPTIONEN

Siehe den Handbucheintrag zu **ttk_notebook**.

REGISTERKENNUNGEN

Siehe den Handbucheintrag zu **ttk_notebook**. Beachten Sie, dass bei Positionsangaben der Form `"@x,y` die Koordinaten relativ zum "plainnotebook"-Fenster angegeben werden müssen.

WIDGET-BEFEHL

Der Befehl `scrollutil::plainnotebook` erstellt einen neuen Tcl-Befehl mit dem Namen `pathName`. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget auszuführen. Er hat die folgende allgemeine Form:

```
pathName option ?arg arg ...?
```

`option` und `arg` legen das genaue Verhalten des Befehls fest. Für Plainnotebook-Widgets sind folgende Befehle möglich:

`pathName add window ?options?`

Siehe den Handbucheintrag zu **ttk_notebook**. Wie dort erwähnt, stellt der Unterbefehl das Fenster `window` an seiner vorherigen Position wieder her, sofern es derzeit vom Notebook verwaltet wird, aber ausgeblendet ist. Dies gilt gleichermaßen für die nicht auswählbaren (Dummy-)Fenster, die mit den Registerkarten verknüpft sind, die mit den Befehlen `addbutton`, `addlabel`, `addseparator`, `insertbutton`, `insertlabel` und `insertseparator` wurden als Unterbefehle erstellt. Wenn `window` vom Notizbuch noch nicht verwaltet wird, ist die entsprechende Registerkarte ein ttk::radiobutton-Widget im Stil einer Werkzeugschaltfläche (`Page.Toolbutton` oder `ClosablePage.Toolbutton`,

abhängig vom Wert der Option `-closabletabs`). Der Wert, der zurückgegeben wird, ist der Pfadname des Widgets, das für die Registerkarte verwendet wird, die `window` entspricht (ein ttk::radiobutton oder ttk::button, beide im Toolbutton-Stil, oder ein ttk::label oder ein ttk::separator).

`pathName addbutton pageIndex text ?image?`

Fügt eine neue Registerkarte hinzu, die als ttk::button-Widget im Toolbutton-Stil implementiert ist (`Desc.Toolbutton`), das neben dem angegebenen `text` und dem optionalen `image` das Element `descend` anzeigt, dessen Form der eines >-Zeichens ähnelt. Dieses Stilelement wird automatisch vom Paket "Scrollutil_tile" erstellt. Seine Größe hängt vom Skalierungsfaktor von Tk ab, während seine Vordergrundfarbe themenspezifisch ist und bei jedem Wechsel des Themas aktualisiert wird. Der Rückgabewert ist der Pfadname des oben genannten ttk::button-Widgets.

Damit dieser Unterbefehl funktioniert, muss das übergeordnete Element des Widgets ein pagesman-Widget sein. `pageIndex`, das eine nicht-negative Ganzzahl sein muss, legt den numerischen Index einer Seite innerhalb des pagesman-Widgets fest, die beim Aufrufen der Werkzeugschaltfläche ausgewählt wird. Es wird nicht erwartet, dass die durch `pageIndex` identifizierte Seite zum Zeitpunkt des Aufrufs dieses Unterbefehls bereits existiert; sie kann später erstellt und dem "pagesman"-Widget hinzugefügt werden.

Dieser Unterbefehl macht es zusammen mit der Option `-caller` einfach, Anwendungen zu schreiben, in denen der Benutzer mit einem einzigen Mausklick von einem plainnotebook zu einem anderen innerhalb eines gemeinsamen pagesman-Elternteils absteigt und auf dieselbe Weise zum ursprünglichen zurückwechselt. Haben die beiden plainnotebook-Widgets beispielsweise die Seitenindizes `0` und `1`, genügt es, den Seitenindex `1` an den Unterbefehl `addbutton` des ersten plainnotebook zu übergeben und die Option `-caller` des zweiten auf den Seitenindex `0` zu setzen.

`pathName addlabel text ?image?`

Fügt eine neue Registerkarte hinzu, die als ttk::label-Widget im Stil `PnbLabel.TLabel` implementiert ist und das angegebene `text` sowie optional `image` anzeigt, und wird typischerweise als Titel einer Gruppe von Registerkarten verwendet. Die für diesen Stil verwendete Schriftart leitet sich von `TkDefaultFont` ab, wobei deren Größe um den Faktor 1,2 vergrößert wurde, und kann mit dem Befehl `ttk::style` geändert werden. Der Wert, der zurückgegeben wird, ist der Pfadname des oben genannten ttk::label-Widgets; Sie können ihn beispielsweise verwenden, um die Schriftart des Stils für diese bestimmte Beschriftung über`... configure -font ...` zu überschreiben.

`pathName addseparator`

Fügt eine neue Registerkarte hinzu, die als horizontales ttk::separator-Widget implementiert ist und in der Regel als Trennzeichen für eine Gruppe von Registerkarten dient. Der Wert ist der Pfadname des oben genannten ttk::separator-Widgets.

`pathName adjustsize`

Setzt die Option `-height` des PlainNotebooks auf die maximal angeforderte Höhe aller Teilflächen und die Option `-width` auf die maximal angeforderte Breite aller Teilflächen.

HINWEIS: Wenn die Optionen `-height` und `-width` von plainnotebook den Standardwert `0` haben, ist in vielen Fällen kein Aufruf dieses Unterbefehls erforderlich, da das Widget

automatisch versucht, seine Größe an die maximal angeforderte Größe seiner Teilflächen anzupassen.

`pathName attrib ?name ?value name value ...??`
Fragt die Attribute des Widgets ab oder ändert sie. Wenn kein `name` angegeben ist, gibt der Befehl eine Liste von Paaren zurück, von denen jedes den Namen und den Wert eines Attributs für `pathName` enthält. Wenn `name` ohne `value` festgelegt wird, gibt der Befehl den Wert des genannten Attributs zurück oder eine leere Zeichenkette, falls kein entsprechender Wert vorhanden ist (mit dem Unterbefehl `hasattrib` können Sie diesen Fall von dem unterscheiden, bei dem der Wert eines *vorhandenen* Attributs eine leere Zeichenkette ist). Wenn ein oder mehrere Paare aus `name` und `value` angegeben werden, legt der Befehl das/die angegebene(n) Widget-Attribut(e) auf den/die angegebenen Wert(e) fest; in diesem Fall ist der Rückgabewert eine leere Zeichenkette. Jedes `name` kann eine beliebige Zeichenkette sein.

`pathName cget option`
Gibt den aktuellen Wert der durch `option` angegebenen Konfigurationsoption zurück, die einen beliebigen der vom Befehl `scrollutil::plainnotebook` akzeptierten Werte annehmen kann.

`pathName closablestate tabId ?boolean?`
Legt das Vorhandensein des Elements `closebtn` in der durch `tabId` angegebenen Registerkarte fest oder fragt es ab. Das optionale Argument wird nur für Registerkarten unterstützt, die über `add` oder `insert` erstellt wurden. Dieser Unterbefehl überschreibt die Option `-closabletabs` für die angegebene Registerkarte.

`pathName configure ?option ?value option value ...??`
Ruft die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn kein `option` festgelegt ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für `pathName` beschreibt (Informationen zum Format dieser Liste finden Sie unter `Tk_ConfigureInfo`). Wenn `option` ohne `value` festgelegt wird, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste entspricht der entsprechenden Unterliste des gegebenen Werts, wenn kein `option` angegeben wurde). Wenn ein oder mehrere Paare aus `option` und `value` festgelegt werden, ändert der Befehl die festgelegten Widget-Optionen so, dass sie die festgelegten Werte annehmen; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. `option` kann jeden der vom Befehl `scrollutil::plainnotebook` akzeptierten Werte annehmen.

`pathName forget tabId`
Siehe den Handbucheintrag zu **ttk_notebook**.

`pathName hasattrib name`
Gibt `1` zurück, wenn das Attribut `name` vorhanden ist, andernfalls `0`.

`pathName hastabattrib tabId name`
Gibt `1` zurück, wenn das Attribut `name` für die durch `tabId` angegebene Registerkarte vorhanden ist, andernfalls `0`.

`pathName hide tabId`
Siehe den Handbucheintrag zu **ttk_notebook**.

`pathName index tabId`
Siehe den Handbucheintrag zu **ttk_notebook**.

`pathName insert pos window ?options?`

Siehe den Handbucheintrag zu **ttk_notebook**. Wie dort erwähnt, verschiebt der Unterbefehl `window` an die angegebene Position, sofern es bereits vom Notebook verwaltet wird. Dies gilt gleichermaßen für die nicht auswählbaren (Dummy-)Fenster, die mit den Registerkarten verknüpft sind, die mit den Befehlen `addbutton`, `addlabel`, `addseparator`, `insertbutton`, `insertlabel` und `insertseparator` erstellten Unterbefehle, die nicht auswählbare (Dummy-)Fenster erstellen. Wenn `window` vom Notebook noch nicht verwaltet wird, ist die entsprechende Registerkarte ein ttk::radiobutton-Widget im Stil einer Werkzeugschaltfläche (`Page.Toolbutton` oder `ClosablePage.Toolbutton`, je nach Wert der Option `-closabletabs`). Der Wert, der zurückgegeben wird, ist der Pfadname des Widgets, das für die Registerkarte verwendet wird, die `window` entspricht (entweder ein ttk::radiobutton oder ttk::button, beide im Toolbutton-Stil, oder ein ttk::label oder ein ttk::separator).

`pathName insertbutton pos pageIndex text ?image?`
Fügt eine neue Registerkarte ein, die als ttk::button-Widget im Stil einer Werkzeugschaltfläche implementiert ist (`Desc.Toolbutton`), das neben dem angegebenen `text` und dem optionalen `image` das Element `descend` an der angegebenen Position anzeigt. Die Details entsprechen weitgehend denen des Unterbefehls `addbutton`.

`pathName insertlabel pos text ?image?`
Fügt eine neue Registerkarte ein, die als ttk::label-Widget im Stil `PnbLabel.TLabel` implementiert ist und das angegebene `text` sowie optional `image` an an der angegebenen Position anzeigt. Die Details ähneln weitgehend denen des Unterbefehls `addlabel`.

`pathName insertseparator pos`
Fügt an der festgelegten Position eine neue Registerkarte ein, die als horizontales ttk::separator-Widget implementiert ist. Die Details entsprechen weitgehend denen des Unterbefehls `addseparator`.

`pathName instate stateSpec ?script?`
Siehe den Handbucheintrag zu **ttk_widget**.

`pathName see tabId`
Dieser Unterbefehl passt die Ansicht im Fenster von "plainnotebook" so an, dass die durch `tabId` angegebene Registerkarte darin sichtbar wird. HINWEIS: Dieser Befehl wird automatisch aufgerufen, sobald eine Registerkarte ausgewählt wird, um diese Registerkarte sichtbar zu machen.

`pathName select ?tabId?`
Siehe den Handbucheintrag zu **ttk_notebook**.

`pathName state ?stateSpec?`
Siehe den Handbucheintrag zu **ttk_widget**.

`pathName style`
Gibt den Wert der Konfigurationsoption `-style` zurück (d. h. den Stil, der vom Widget ttk::notebook verwendet wird, das im "plainnotebook" enthalten ist). Für Widgets mit Tk-Thema wurde dieser Unterbefehl in Tk 8.7a4 eingeführt, das "plainnotebook"-Widget stellt ihn jedoch für alle unterstützten Tk-Versionen bereit.

`pathName tab tabId ?option ?value option value ...??`
Siehe den Handbucheintrag zu **ttk_notebook**.

`pathName tabattrib tabId ?name ?value name value ...??`
Fragt die Attribute der durch `tabId` angegebenen Registerkarte ab oder ändert sie. Wenn

kein `name` festgelegt ist, gibt der Befehl eine Liste von Paaren zurück, von denen jedes den Namen und den Wert eines Attributs der Registerkarte enthält. Wenn `name` ohne `value` festgelegt wird, gibt der Befehl den Wert des einen benannten Registerkartenattributs zurück oder eine leere Zeichenkette, falls kein entsprechender Wert vorhanden ist (Sie können den Unterbefehl `hastabattrib` verwenden, um diesen Fall von dem zu unterscheiden, bei dem der Wert eines *vorhandenen* Registerkartenattributs eine leere Zeichenkette ist). Wenn ein oder mehrere Paare aus `name` und `value` gelegt werden, setzt der Befehl das/die angegebene(n) Registerkartenattribut(e) auf den/die angegebenen Wert(e); in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. Jedes `name` kann eine beliebige Zeichenkette sein.

`pathName tabpath tabId`
Gibt den Pfadnamen des Widgets zurück, das für die angegebene Registerkarte verwendet wird. Bei diesem Widget kann es sich um ein "ttk::radiobutton" oder "ttk::button" handeln, die beide den Stil "Toolbutton" verwenden, oder um ein "ttk::label" oder ein "ttk::separator".

`pathName tabs`
Siehe den Handbucheintrag zu **ttk_notebook**.

`pathName titlepath`
Gibt den Pfadnamen der Titelbeschriftung des "plainnotebook" zurück, bei der es sich um ein Widget vom Typ "ttk::label" im Stil `PnbTitle.TLabel` handelt.

`pathName unsetattrib name`
Hebt das Attribut `name` auf. Gibt eine leere Zeichenfolge zurück.

`pathName unsettabattrib tabId name`
Hebt das Attribut `name` für die durch `tabId` angegebene Registerkarte auf. Gibt eine leere Zeichenkette zurück.

TASTATURNAVIGATION
Siehe den Handbucheintrag zu **ttk_notebook**.

VIRTUELLE EREIGNISSE
Wie im Handbucheintrag zu **ttk_notebook** beschrieben, generiert das Widget nach der Auswahl einer neuen Registerkarte eines ttk::notebook (programmgesteuert oder interaktiv) ein virtuelles Ereignis vom Typ `<<NotebookTabChanged>>`. Dies gilt auch für "plainnotebook"-Widgets.

Das Paket "Scrollutil_tile" definiert das virtuelle Ereignis `<<Button3>>` für alle Fenstersysteme als `<Button-3>` und zusätzlich für Mac OS X/11+ als `<Control-Button-1>`. Tritt dieses Ereignis über einer Registerkarte eines "plainnotebook" auf, dessen Widget-Zustandsflag `disabled` nicht gesetzt ist, und handelt es sich um die Tk-Version 8.5a2 oder höher, dann erzeugt das Widget ein virtuelles Ereignis `<<MenuItemsRequested>>`, indem es `event generate` aufruft, wobei die Option `-data` auf eine Liste gesetzt ist, die aus dem Pfadnamen eines leeren Popup-Menüs und dem numerischen Index der betreffenden Registerkarte besteht. (Die Unterstützung für diese Option wurde in Tk 8.5a2 eingeführt.) Wenn die Anwendung eine Bindung für dieses virtuelle Ereignis erstellt, kann das Skript der Bindung auf die Benutzerdaten als Wert des Feldes `%d` zugreifen, um das Menü mit anwendungsspezifischen Einträgen zu füllen. HINWEIS: Das Popup-Menü wird mit einer Verzögerung von 100 ms angezeigt, um sicherzustellen, dass es bereits von der Anwendung ausgefüllt erscheint.

Das folgende Beispiel zeigt einen Eintrag im Popup-Menü zum Schließen der Registerkarten eines "plainnotebook"-Widgets mit der ID `$nb`:

```
bind $nb <<MenuItemsRequested>> { populateMenu %W %d }

proc populateMenu {nb data} {
    foreach {menu tabIdx} $data {}
    $menu add command -label "Close Tab" -command [list $nb forget
    $tabIdx]
}
```

Eine BESCHREIBUNG der virtuellen EREIGNISSE `<<NotebookTabMoved>>` und `<<NotebookTabClosed>>` finden Sie im nächsten Abschnitt.

WEITERE BINDUNGEN

Das Paket "Scrollutil_tile" erstellt die folgenden zusätzlichen Bindungen für die "plainnotebook"-Widgets:

1. **Navigation zwischen den auswählbaren Registerkarten eines PlainNotebook-Widgets über das Mausrad oder das Touchpad:** Zur Erinnerung: Bei den auswählbaren Registerkarten handelt es sich um diejenigen, die mit den Unterbefehlen `add` und `insert` erstellt wurden. Ein Mausrad-Klick oder eine Zwei-Finger-Geste auf dem Touchpad wählt zyklisch die Registerkarte mit dem Status `normal` aus, die auf die aktuell ausgewählte folgt bzw. ihr vorausgeht. (HINWEIS: Der gleiche Effekt kann auch mit den Tasten `Down`, `Up`, `Control-Tab` und `Control-Shift-Tab` erzielt werden.) Im Fenstersystem `aqua` wird, wenn die Taste `Option` gedrückt ist, während das Mausrad gedreht wird, durch einen Radsprung zyklisch die 10. auswählbare Registerkarte des Zustands `normal` ausgewählt, die auf die aktuell ausgewählte folgt bzw. ihr vorausgeht (modulo der Anzahl der auswählbaren Registerkarten). Das Gleiche gilt für die Fenstersysteme `x11` und `win32`, wenn die Tk-Version 8.7a4 oder höher ist und das Mausrad bei gedrückter `Alt`-Taste gedreht wird.
2. **Verschieben der Registerkarten eines "plainnotebook"-Widgets mit der Maus**: Wenn die linke Maustaste über einer Registerkarte, jedoch außerhalb des Elements `closebtn` (sofern vorhanden) gedrückt und die Registerkarte anschließend außerhalb dieses Elements gezogen wird, nimmt der Mauszeiger – vorausgesetzt, der Wert der Option `-movabletabs` des Widgets ist "true" – nimmt der Mauszeiger die Form eines Aufwärts-/Abwärtspfeils an, was anzeigt, dass die betreffende Registerkarte an eine andere Position verschoben wird. Dieser Vorgang endet, wenn Maustaste 1 losgelassen wird, und kann durch Drücken der Taste `Escape` abgebrochen werden. In beiden Fällen wird der Mauszeiger auf seinen ursprünglichen Wert zurückgesetzt, der durch die Konfigurationsoption `-cursor` festgelegt ist. Nach dem Loslassen der Maustaste 1 wird die Ausgangsregisterkarte an die Position verschoben, auf die der Pfeil zuvor zeigte (vorausgesetzt, die Aktion wurde nicht über `Escape` abgebrochen). Wenn die Tk-Version 8.5a2 oder höher ist, generiert das Widget nach dem Verschieben der Registerkarte an ihre neue Position ein virtuelles Ereignis vom Typ `<<NotebookTabMoved>>`, indem es `event generate` aufruft, wobei die Option `-data` auf eine Liste gesetzt ist, die aus dem Pfadnamen des im Quellbereich enthaltenen Widgets und dem numerischen Index der Zielposition der Registerkarte besteht. (Die Unterstützung für diese Option wurde in Tk 8.5a2 eingeführt.)
3. **Schließen eines Reiters eines "plainnotebook"-Widgets mithilfe des**

"closebtn"-Elements: Wenn die Maustaste 1 über dem Element `closebtn` (sofern vorhanden) eines Reiters gedrückt und später über demselben Element wieder losgelassen wird, vorausgesetzt, der Wert der Option `-state` der Registerkarte ist `normal`, wird die Registerkarte mittels des Unterbefehls `forget` des Widgets geschlossen. Beachten Sie, dass dieser Vorgang durch den als Wert der Option `-forgetcommand` angegebenen Befehl abgebrochen werden kann; sollte dies nicht der Fall sein und die Tk-Version 8.5a2 oder höher sein, dann erzeugt das Widget nach dem Schließen der Registerkarte ein virtuelles Ereignis vom Typ `<<NotebookTabClosed>>`, indem es `event generate` aufruft, wobei die Option `-data` auf den Pfadnamen des Widgets gesetzt ist, das sich auf der Teilfläche dieser Registerkarte befindet.

Wenn das Statusflag `disabled` des Widgets gesetzt ist, findet keine der oben genannten Aktionen statt: Die Registerkarten sind vollständig deaktiviert.

SCHLÜSSELWÖRTER

plainnotebook, widget, notebook, closebtn

Stichwörter: plainnotebook

scrollableframe

Das `scrollutil::scrollableframe` und `scrollutil::adaptBtn2EventHandling` BEFEHLE

Für Scrollutil Version 2.11

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

- Der Befehl scrollutil::scrollableframe – Kurzreferenz
- Der Befehl scrollutil::scrollableframe – Ausführliche Referenz
- Der Befehl scrollutil::adaptBtn2EventHandling
- Vergleich mit BWidget ScrollableFrame und iwidgets::scrolledframe

---

Der Befehl `scrollutil::scrollableframe` – Kurzreferenz

NAME

`scrollutil::scrollableframe` – Erstellen und Bearbeiten von "ScrollableFrame"-Widgets

ÜBERSICHT

```
scrollutil::scrollableframe pathName ?options?
```

STANDARDOPTIONEN

```
-background   -highlightbackground  -relief
-borderwidth  -highlightcolor       -xscrollcommand
-cursor       -highlightthickness   -yscrollcommand
```

WIDGET-SPEZIFISCHE OPTIONEN

```
-contentheight screenDistance

-contentwidth screenDistance

-fitcontentheight boolean

-fitcontentwidth boolean

-height screenDistance

-takefocus 0|1|""|command

-width screenDistance

-xscrollincrement screenDistance

-yscrollincrement screenDistance
```

Widget COMMAND

```
pathName attrib ?name ?value name value ...??

pathName autofillx ?boolean?

pathName autofilly ?boolean?

pathName autosize ?w|h|wh?

pathName cget option

pathName configure ?option ?value option value ...??

pathName contentframe

pathName hasattrib name

pathName preparescan

pathName preparescroll

pathName scan option args

pathName scan mark x y
```

```
pathName scan dragto x y ?gain?
pathName see widget ?nw|ne|sw|se?
pathName seerect x1 y1 x2 y2 ?nw|ne|sw|se?
pathName unsetattrib name
pathName xview ?args?
pathName xview
pathName xview units
pathName xview moveto fraction
pathName xview scroll number units|pages
pathName yview ?args?
pathName yview
pathName yview units
pathName yview moveto fraction
pathName yview scroll number units|pages
```

SCHLÜSSELWÖRTER

scrollableframe, widget, frame, scrollable

Der Befehl `scrollutil::scrollableframe` – Ausführliche Referenz

NAME

`scrollutil::scrollableframe` – Erstellen und Bearbeiten von "scrollableframe"-Widgets

ÜBERSICHT

```
scrollutil::scrollableframe pathName ?options?
```

BESCHREIBUNG

Der Befehl `scrollutil::scrollableframe` erstellt ein neues Fenster mit dem Namen `pathName` und der Klasse `Scrollableframe` und wandelt es in ein **scrollableframe** -Widget um. Zusätzliche Optionen, die im Folgenden beschrieben werden, können über die Befehlszeile oder in der Optionsdatenbank festgelegt werden, um Aspekte des "scrollableframe"-Widgets wie dessen Breite, Höhe und Scroll-Schritte zu konfigurieren. Der Befehl `scrollutil::scrollableframe` gibt sein Argument `pathName` zurück. Zum Zeitpunkt des Aufrufs dieses Befehls darf kein Fenster mit dem Namen `pathName` vorhanden sein, aber das übergeordnete Fenster von `pathName` muss vorhanden sein.

Ein "scrollableframe" ist ein Megawidget, das einen **Inhaltsrahmen** enthält, dessen Abmessungen größer sein können als die des Widgets selbst. Beliebige Bereiche dieses Rahmens können durch Scrollen in den sichtbaren Bereich gebracht werden, da das "scrollableframe"-Widget die Konfigurationsoptionen `-xscrollcommand` und `-yscrollcommand` unterstützt und der zugehörige Tcl-Befehl über die Unterbefehle `xview` und `yview` verfügt. Der Inhaltsrahmen kann eine beliebige Anzahl beliebiger Widgets enthalten. Mit anderen Worten: Der Befehl `scrollutil::scrollableframe` implementiert einen **scrollbaren Widget-Container**.

Im Scrollutil-Paket sind sowohl das "scrollableframe"-Widget als auch dessen Inhaltsrahmen als Tk-Core-Frames implementiert, während es sich bei Scrollutil_tile um ttk::frame-Widgets handelt. Auf den Inhaltsrahmen kann über den Unterbefehl `contentframe` zugegriffen werden. Die darin enthaltenen einzelnen Widgets können mithilfe des Unterbefehls `see` im "scrollableframe"-Fenster sichtbar gemacht werden. Neben dem Scrollen unterstützen "scrollableframe"-Widgets auch das Scannen mithilfe entsprechender Mausereignis-Bindungen, die den Unterbefehl `scan` aufrufen.

Ein "ScrollableFrame"-Widget wird in der Regel innerhalb eines scrollarea erstellt, wie im folgenden Beispiel gezeigt:

```
set sa [scrollutil::scrollarea ...]
set sf [scrollutil::scrollableframe $sa.sf ...]
$sa setwidget $sf

set cf [$sf contentframe]
<populate the content frame>

$sf autosize w
update idletasks
$sf configure -height ... -yscrollincrement ...

pack $sa -expand yes -fill both
```

HINWEIS: Nach der Erstellung des "scrollableframe"-Widgets registriert der Befehl `scrollutil::scrollableframe` dieses automatisch für das Scrollen mit dem Mausrad oder dem Touchpad, indem er dessen Pfadnamen an den Befehl `scrollutil::enableScrollingByWheel` übergibt.

STANDARDOPTIONEN

```
-background   -highlightbackground  -relief
-borderwidth  -highlightcolor       -xscrollcommand
-cursor       -highlightthickness   -yscrollcommand
```

Weitere Informationen zu den Standardoptionen für Tk-Widgets finden Sie im Handbucheintrag zu den **options**. Die Optionen `-background`, `-highlightbackground`, `-highlightcolor` und `-highlightthickness` werden nur vom Scrollutil-Paket unterstützt, nicht jedoch von Scrollutil_tile. Sie haben dieselben Standardwerte wie die

gleichnamigen Optionen für Tk-Frame-Widgets. Die Standardwerte der übrigen Standardoptionen lauten:

```
-borderwidth 0 -cursor "" -relief flat -xscrollcommand ""
    -yscrollcommand ""
```

WIDGET-SPEZIFISCHE OPTIONEN

Befehlszeilenname:	-contentheight
Name der Datenbank:	contentHeight
Datenbankklasse:	ContentHeight

Legt die gewünschte Höhe für den Inhaltsrahmen in einem der von `Tk_GetPixels` akzeptierten Formate fest. Wenn der Wert der Option kleiner oder gleich Null ist, wird die Höhe des Inhaltsrahmens gerade so groß gewählt, dass alle untergeordneten Elemente darin Platz finden. Diese Option ist nur relevant, wenn der Wert der Option `-fitcontentheight` auf "false" gesetzt ist; ansonsten hat der Inhaltsrahmen dieselbe Höhe wie das "scrollableframe"-Fenster (ohne dessen Rahmen und gegebenenfalls vorhandenes Hervorhebungsrechteck), unabhängig vom Wert der Option `-contentheight`. Der Standardwert ist `0`, was für die überwiegende Mehrheit der Anwendungen geeignet ist.

Befehlszeilenname:	-contentwidth
Name der Datenbank:	contentWidth
Datenbankklasse:	ContentWidth

Legt die gewünschte Breite für den Inhaltsrahmen in einem der von `Tk_GetPixels` akzeptierten Formate fest. Wenn der Wert der Option kleiner oder gleich Null ist, wird die Breite des Inhaltsrahmens gerade so groß gewählt, dass alle seine untergeordneten Elemente Platz finden. Diese Option ist nur relevant, wenn der Wert der Option `-fitcontentwidth` auf "false" gesetzt ist; ansonsten hat der Inhaltsrahmen dieselbe Breite wie das "scrollableframe"-Fenster (ohne dessen Rahmen und gegebenenfalls vorhandenes Hervorhebungsrechteck), unabhängig vom Wert der Option `-contentwidth`. Der Standardwert ist `0`, was für die überwiegende Mehrheit der Anwendungen geeignet ist.

Befehlszeilenname:	-fitcontentheight
Name der Datenbank:	fitContentHeight
Datenbankklasse:	FitContentHeight

Legt einen booleschen Wert fest, der angibt, ob der Inhaltsrahmen dieselbe Höhe wie das "scrollableframe"-Fenster haben soll (ohne dessen Rahmen und gegebenenfalls vorhandenes Hervorhebungsrechteck). Wenn der Wert "true" ist, wird die Höhe des Inhaltsrahmens mit der des Widgets synchronisiert, und der Wert der Option `-contentheight` wird ignoriert. Der

Standardwert ist `0`.

Befehlszeilenname:	-fitcontentwidth
Name der Datenbank:	fitContentWidth
Datenbankklasse:	FitContentWidth

Legt einen booleschen Wert fest, der angibt, ob der Inhaltsrahmen dieselbe Breite wie das "scrollableframe"-Fenster haben soll (ohne dessen Rahmen und gegebenenfalls vorhandenes Hervorhebungsrechteck). Wenn der Wert "true" ist, wird die Breite des Inhaltsrahmens an die des Widgets angepasst, und der Wert der Option `-contentwidth` wird ignoriert. Der Standardwert ist `0`.

Befehlszeilenname:	-height
Name der Datenbank:	height
Datenbankklasse:	Height

Legt die gewünschte Höhe für das "scrollableframe"-Widget in einem der von `Tk_GetPixels` akzeptierten Formate fest. Beachten Sie, dass hiermit die *innere* Höhe festgelegt wird, ohne den Rahmen und das Hervorhebungsrechteck (falls vorhanden), die um das Widget herum gezeichnet werden. Der Standardwert ist `7c`, der durch einen geeigneten anwendungsspezifischen Wert überschrieben werden sollte.

Befehlszeilenname:	-takefocus
Name der Datenbank:	takeFocus
Datenbankklasse:	TakeFocus

Diese Option legt fest, ob das "scrollableframe"-Widget während der Tastaturnavigation den Fokus annimmt. Sie ist nahezu identisch mit der gleichnamigen Standardoption (Einzelheiten finden Sie im Handbucheintrag **options**). Der einzige Unterschied besteht darin, dass nicht das "scrollableframe"-Widget selbst, sondern dessen Inhaltsrahmen den Fokus erhält, wenn die Standardtasten (`Tab` und `Shift-Tab`) bei der Tastaturnavigation gedrückt werden. Die Standardeinstellung ist `0`, da ein "scrollableframe"-Widget im Wesentlichen ein Rahmen ist, der als Ansicht für seine Inhaltsrahmen-Komponente dient.

Befehlszeilenname:	-width
Name der Datenbank:	width
Datenbankklasse:	Width

Legt die gewünschte Breite für das "scrollableframe"-Widget in einem der von `Tk_GetPixels` akzeptierten Formate fest. HINWEIS: Hiermit wird die *innere* Breite festgelegt, ohne den

Rahmen und das Hervorhebungsrechteck (falls vorhanden), die um das Widget herum gezeichnet werden. Der Standardwert ist `10c`, der durch einen geeigneten anwendungsspezifischen Wert überschrieben werden sollte.

Befehlszeilenname:	-xscrollincrement
Name der Datenbank:	xScrollIncrement
Datenbankklasse:	ScrollIncrement

Legt einen Schrittwert für das horizontale Scrollen fest, in einer der von `Tk_GetPixels` akzeptierten Formen. Wenn der Wert dieser Option größer als Null ist, wird die horizontale Ansicht im Fenster so begrenzt, dass die x-Koordinate innerhalb des Inhaltsrahmens am linken Rand des Fensters stets ein gerades Vielfaches von `xScrollIncrement` ist; außerdem entsprechen die Einheiten für den horizontalen Bildlauf (siehe den Unterbefehl `xview`) ebenfalls `xScrollIncrement`. Wenn der Wert kleiner oder gleich Null ist, ist der horizontale Bildlauf uneingeschränkt. Der Standardwert ist `0`.

Befehlszeilenname:	-yscrollincrement
Name der Datenbank:	yScrollIncrement
Datenbankklasse:	ScrollIncrement

Legt einen Schrittwert für das vertikale Scrollen fest, in einer der von `Tk_GetPixels` akzeptierten Formen. Wenn der Wert dieser Option größer als Null ist, wird die vertikale Ansicht im Fenster so begrenzt, dass die y-Koordinate innerhalb des Inhaltsrahmens am oberen Rand des Fensters stets ein gerades Vielfaches von `yScrollIncrement` ist; außerdem entsprechen die Einheiten für das vertikale Scrollen (siehe den Unterbefehl `yview`) ebenfalls `yScrollIncrement`. Wenn der Wert kleiner oder gleich Null ist, ist das vertikale Scrollen uneingeschränkt. Der Standardwert ist `0`.

WIDGET-BEFEHL

Der Befehl `scrollutil::scrollableframe` erstellt einen neuen Tcl-Befehl mit dem Namen `pathName`. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget auszuführen. Er hat die folgende allgemeine Form:

```
pathName option ?arg arg ...?
```

`option` und `arg` legen das genaue Verhalten des Befehls fest. Für "scrollableframe"-Widgets sind folgende Befehle möglich:

`pathName attrib ?name ?value name value ...??`

Fragt die Attribute des Widgets ab oder ändert sie. Wenn kein `name` angegeben ist, gibt der Befehl eine Liste von Paaren zurück, von denen jedes den Namen und den Wert eines Attributs für `pathName` enthält. Wenn `name` ohne `value` festgelegt wird, gibt der Befehl den Wert des genannten Attributs zurück oder eine leere Zeichenkette, falls kein entsprechender Wert vorhanden ist (mit dem Unterbefehl `hasattrib` können Sie diesen Fall von dem unterscheiden, bei dem der Wert eines *vorhandenen* Attributs eine leere

Zeichenkette ist). Wenn ein oder mehrere Paare aus `name` und `value` angegeben werden, legt der Befehl das/die angegebene(n) Widget-Attribut(e) auf den/die angegebenen Wert(e) fest; in diesem Fall ist der Rückgabewert eine leere Zeichenkette. Jedes `name` kann eine beliebige Zeichenkette sein.

`pathName autofillx ?boolean?`
Prüft oder fragt ab, ob der Inhaltsrahmen die gesamte innere Breite des Widgets ausfüllt. Wenn das optionale boolesche Argument "true" ist, setzt der Befehl die Option `-fitcontentwidth` auf "true", wenn die angeforderte Breite des Inhaltsrahmens kleiner ist als die innere Breite des scrollbaren Rahmens, andernfalls auf "false". Außerdem stellt der Befehl sicher, dass diese Aktion wiederholt wird, sobald sich die angeforderte Breite des Inhaltsrahmens oder die Innenbreite des scrollbaren Rahmens ändert. Wenn das optionale Argument "false" ist, löscht der Befehl das Flag, das diese dynamische Aktion steuert. In beiden Fällen ist der Rückgabewert eine leere Zeichenkette. Ohne das optionale Argument gibt der Befehl den aktuellen Wert des oben genannten Flags zurück.

`pathName autofilly ?boolean?`
Prüft oder fragt ab, ob der Inhaltsrahmen die gesamte innere Höhe des Widgets ausfüllt. Wenn das optionale boolesche Argument "true" ist, setzt der Befehl die Option `-fitcontentheight` auf "true", wenn die angeforderte Höhe des Inhaltsrahmens kleiner ist als die innere Höhe des scrollbaren Rahmens, andernfalls auf "false". Außerdem stellt der Befehl sicher, dass diese Aktion immer dann wiederholt wird, wenn sich die angeforderte Höhe des Inhaltsrahmens oder die innere Höhe des scrollbaren Rahmens ändert. Wenn das optionale Argument "false" ist, löscht der Befehl das Flag, das diese dynamische Aktion steuert. In beiden Fällen ist der Rückgabewert eine leere Zeichenkette. Ohne das optionale Argument gibt der Befehl den aktuellen Wert des oben genannten Flags zurück.

`pathName autosize ?dimensions?`
Dieser Befehl setzt die Option `-width` und/oder `-height` des scrollableframe auf die gewünschte Breite und/oder Höhe seines Inhaltsrahmens, entsprechend dem optionalen Argument `dimensions`, dessen mögliche Werte `w` (Breite), `h` (Höhe) und `wh` (Breite / Höhe). Der Standardwert ist eine leere Zeichenkette, was bedeutet, dass weder die Breite noch die Höhe *explizit* festgelegt werden soll. Beachten Sie, dass der Befehl, wenn der Wert der Option `-fitcontentwidth` "true" ist, die innere Breite des Widgets auf die angeforderte Breite seines Inhaltsrahmens setzt, unabhängig davon, ob diese *explizit* angegeben wurde oder nicht. Ebenso setzt der Befehl, wenn der Wert der Option `-fitcontentheight` "true" ist, die innere Höhe des Widgets auf die angeforderte Höhe seines Inhaltsrahmens, auch wenn diese nicht *explizit* angegeben wurde.

HINWEIS: Das Festlegen der inneren Breite und/oder Höhe des Widgets im Rahmen dieses Unterbefehls wird als `"after 100"` -Callback ausgeführt, um sicherzustellen, dass die Berechnungen zur Geometrieverwaltung im Inhaltsrahmen des Widgets abgeschlossen sind, bevor dessen Breite und/oder Höhe festgelegt werden.

`pathName cget option`
Gibt den aktuellen Wert der durch `option` angegebenen Konfigurationsoption zurück, die einen beliebigen der vom Befehl `scrollutil::scrollableframe` akzeptierten Werte annehmen kann.

`pathName configure ?option ?value option value ...??`
Ruft die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn kein `option`

festgelegt wird, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für `pathName` beschreibt (Informationen zum Format dieser Liste finden Sie unter `Tk_ConfigureInfo`). Wenn `option` ohne `value` festgelegt wird, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste entspricht der entsprechenden Unterliste des Werts, der zurückgegeben wird, wenn kein `option` festgelegt wurde). Wenn ein oder mehrere Paare aus `option` und `value` festgelegt werden, ändert der Befehl die festgelegten Widget-Optionen so, dass sie die festgelegten Werte annehmen; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. `option` kann jeden der vom Befehl `scrollutil::scrollableframe` akzeptierten Werte annehmen.

`pathName contentframe`
Gibt den Pfadnamen des Inhaltsrahmens des Widgets zurück.

`pathName hasattrib name`
Gibt `1` zurück, wenn das Attribut `name` vorhanden ist, andernfalls `0`.

`pathName preparescan`
Bereitet den ScrollableFrame für das Scannen vor, indem er seine untergeordneten Elemente, die sich im gleichen Toplevel-Fenster befinden und über Bindungen für Ereignisse der Maustaste 2 verfügen, an den Befehl `scrollutil::adaptBtn2EventHandling` übergibt.

HINWEIS: Wenn der "scrollableframe" viele untergeordnete Elemente enthält, ist es wesentlich sicherer, diesen Unterbefehl aufzurufen, als manuell eine große Anzahl von Widgets an `scrollutil::adaptBtn2EventHandling` zu übergeben, da die Entscheidung, welche untergeordneten Elemente verarbeitet werden sollen, mühsam und fehleranfällig sein kann.

`pathName preparescroll`
Bereitet den "scrollableframe" für das Scrollen mit dem Mausrad und dem Touchpad vor, indem er seine untergeordneten Elemente, die sich auf dem gleichen Toplevel-Fenster befinden und über Bindungen für Mausrad- oder `<TouchpadScroll>`-Ereignisse verfügen, an den Befehl `scrollutil::adaptWheelEventHandling` übergibt.

HINWEIS: Der Aufruf dieses Unterbefehls ist lediglich eine Abkürzung, um den Wert `pathName` an den Befehl `scrollutil::prepareScrollingByWheel` zu übergeben.

`pathName scan option args`
Dieser Befehl dient dazu, das Scannen bei "scrollableframe"-Widgets zu implementieren. Je nach `option` gibt es zwei Varianten:

`pathName scan mark x y`
Speichert die Datensätze `x` und `y` sowie die aktuelle Ansicht im ScrollableFrame-Fenster; wird in Verbindung mit den nachfolgenden Befehlen `scan dragto` verwendet. In der Regel ist dieser Befehl mit einem Mausklick im Widget verknüpft, wobei `x` und `y` die Koordinaten der Maus darstellen. Er gibt eine leere Zeichenkette zurück.

`pathName scan dragto x y ?gain?`
Dieser Befehl berechnet die Differenz zwischen seinen Argumenten `x` und `y` (bei denen es sich in der Regel um Mauskoordinaten handelt) und den Argumenten `x` und `y` des letzten `scan mark`-Befehls für das Widget. Anschließend passt er die Ansicht um das `gain`-fache der Differenz der Koordinaten an, wobei `gain` standardmäßig auf `10` gesetzt ist, außer auf Android, wo der Standardwert `2` ist, da dieser Wert besser zur Emulation von Ereignissen der

Maustaste 2 mit Ein-Finger-Gesten geeignet ist. Dieser Befehl wird typischerweise mit Mausbewegungsereignissen im Widget verknüpft, um den Effekt zu erzeugen, dass der Inhaltsrahmen mit hoher Geschwindigkeit durch das Fenster gezogen wird. Der Rückgabewert ist eine leere Zeichenkette.

`pathName see widget ?corner?`
Dieser Befehl passt die Ansicht im Fenster des ScrollableFrame so an, dass `widget` darin sichtbar wird. Das angegebene Widget muss ein Nachkomme des Inhaltsrahmen sein und dieselbe Toplevel-Fenster-Ebene haben. Außerdem muss es von einem Geometriemanager verwaltet werden (wie beispielsweise `grid`, `pack`, `place`, `text` oder `canvas`) verwaltet werden. Das optionale Argument `corner` legt fest, welche Ecke von `widget` sichtbar werden soll, wenn es zu groß ist, um in das Fenster zu passen. Mögliche Werte sind `nw` (Nordwesten), `ne` (Nordosten), `sw` (Südwesten) und `se` (Südosten). Die Standardecke ist `nw`.

HINWEIS: Mit diesem Unterbefehl können Sie die Tastaturnavigation im Inhaltsrahmen benutzerfreundlicher gestalten, wie im folgenden Beispiel, in dem `$w` ein untergeordneter Element des Inhaltsrahmens des ScrollableFrame-Widgets `$sf` ist (es wird davon ausgegangen, dass `$sf` keine Zeichen von `%` enthält):

```
bind $w <<TraverseIn>> [list $sf see %W]
```

Wenn das Widget `$w` über dessen Unterbefehl `setwidget` in einen scrollarea eingebettet wurde und der Scrollbereich wiederum ein Nachkomme des Inhaltsrahmens ist, dann ist es benutzerfreundlicher, den Scrollbereich in den sichtbaren Bereich zu bringen, anstatt nur das Widget:

```
bind $w <<TraverseIn>> [list seeScrollarea $sf %W]

proc seeScrollarea {sf w} { $sf see [scrollutil::getscrollarea $w] }
```

`pathName seerect x1 y1 x2 y2 ?corner?`
Dieser Befehl passt die Ansicht im Fenster des ScrollableFrame so an, dass das durch `x1`, `y1`, `x2` und `y2` darin sichtbar werden. Diese Koordinaten beziehen sich auf den Inhaltsrahmen und legen zwei diagonal gegenüberliegende Ecken des Rechtecks fest (das seine obere und linke Kante, nicht jedoch seine untere oder rechte Kante umfasst). Das optionale Argument `corner` legt fest, welche Ecke des Rechtecks sichtbar werden soll, wenn es zu groß ist, um in das Fenster zu passen. Die möglichen Werte sind `nw` (Nordwesten), `ne` (Nordosten), `sw` (Südwesten) und `se` (Südosten). Die Standardecke ist `nw`.

`pathName unsetattrib name`
Hebt das Attribut `name` auf. Gibt eine leere Zeichenfolge zurück.

`pathName xview ?args?`
Dieser Befehl dient dazu, die horizontale Position der im Fenster des Widgets angezeigten Informationen abzufragen und zu ändern. Er kann eine der folgenden Formen annehmen:

`pathName xview`
Gibt eine Liste mit zwei Elementen zurück. Jedes Element ist ein reeller Bruch zwischen `0` und `1`; zusammen beschreiben sie den horizontalen Bereich, der im Fenster sichtbar ist. Wenn beispielsweise das erste Element `0.2` und das zweite Element `0.6` ist, liegen 20 % des

Inhaltsrahmens links außerhalb des Bildschirms, die mittleren 40 % sind im Fenster sichtbar, und 40 % des Inhaltsrahmens befinden sich rechts außerhalb des Bildschirms. Dies sind dieselben Werte, die über die Option `-xscrollcommand` an die Bildlaufleisten übergeben werden.

`pathName xview units`
Passt die Ansicht im Fenster so an, dass die durch `units` angegebene Position des Inhaltsrahmens am linken Rand des Fensters angezeigt wird. Dieses Argument legt die Anzahl der Einheiten der Option `xScrollIncrement` fest, wenn der Wert größer als Null ist; andernfalls entspricht es einem Zehntel der inneren Breite des Fensters.

`pathName xview moveto fraction`
Passt die Ansicht im Fenster so an, dass `fraction` der Gesamtbreite des Inhaltsrahmens links außerhalb des Bildschirms liegt. `fraction` muss ein Bruchwert zwischen `0` und `1` sein.

`pathName xview scroll number what`
Dieser Befehl verschiebt die Ansicht im Fenster entsprechend den Werten `number` und `what` nach links oder rechts. `number` muss eine Ganzzahl oder eine Gleitkommazahl sein; wenn es sich um eine Gleitkommazahl handelt, wird sie in eine Ganzzahl umgewandelt und dabei von `0` weg gerundet. `what` muss entweder `units` oder `pages` oder eine Abkürzung eines dieser Werte sein. Wenn `what` gleich `units` ist, passt sich die Ansicht nach links oder rechts in Einheiten der Option `xScrollIncrement` an, sofern diese größer als Null ist, andernfalls in Einheiten von einem Zehntel der inneren Breite des Fensters. Wenn `what` gleich `pages` ist, so wird die Ansicht in Einheiten von neun Zehnteln der inneren Breite des Fensters angepasst. Wenn `number` negativ ist, werden Informationen weiter links sichtbar; wenn der Wert positiv ist, werden Informationen weiter rechts sichtbar.

`pathName yview ?args?`
Dieser Befehl dient dazu, die vertikale Position der im Fenster des Widgets angezeigten Informationen abzufragen und zu ändern. Er kann in einer der folgenden Formen vorliegen:

`pathName yview`
Gibt eine Liste mit zwei Elementen zurück. Jedes Element ist ein reeller Bruch zwischen `0` und `1`; zusammen beschreiben sie den vertikalen Bereich, der im Fenster sichtbar ist. Wenn beispielsweise das erste Element `0.2` und das zweite Element `0.6` ist, liegen 20 % des Inhaltsrahmens oben außerhalb des Bildschirms, die mittleren 40 % sind im Fenster sichtbar, und 40 % des Inhaltsrahmens liegen am unteren Rand außerhalb des Bildschirms. Dies sind dieselben Werte, die über die Option `-yscrollcommand` an die Bildlaufleisten übergeben werden.

`pathName yview units`
Passt die Ansicht im Fenster so an, dass die durch `units` angegebene Position des Inhaltsrahmens am oberen Rand des Fensters angezeigt wird. Dieses Argument legt eine Anzahl von Einheiten der Option `yScrollIncrement` fest, wenn diese größer als Null ist; andernfalls legt es ein Zehntel der inneren Höhe des Fensters fest.

`pathName yview moveto fraction`
Passt die Ansicht im Fenster so an, dass `fraction` der Gesamthöhe des Inhaltsrahmens oben außerhalb des Bildschirms liegt. `fraction` muss ein Bruch zwischen `0` und `1` sein.

`pathName yview scroll number what`
Dieser Befehl verschiebt die Ansicht im Fenster entsprechend den Werten `number` und `what` nach oben oder unten. `number` muss eine Ganzzahl oder eine Gleitkommazahl sein; wenn es

sich um eine Gleitkommazahl handelt, wird diese in eine Ganzzahl umgewandelt und dabei von `0` weg gerundet. `what` muss entweder `units` oder `pages` oder eine Abkürzung eines dieser Werte sein. Wenn `what` gleich `units` ist, passt sich die Ansicht in Einheiten der Option `yScrollIncrement` an, sofern diese größer als Null ist, andernfalls in Einheiten von einem Zehntel der inneren Höhe des Fensters. Wenn `what` gleich `pages` ist, so wird die Ansicht in Einheiten von neun Zehnteln der inneren Höhe des Fensters angepasst. Wenn `number` negativ ist, werden Informationen weiter oben sichtbar; wenn der Wert positiv ist, werden Informationen weiter unten sichtbar.

VORGABEBINDUNGEN

Mausknopf 2 kann zum Scrollen verwendet werden. Wenn er gedrückt und über einen freien Bereich des "scrollableframe"-Fensters gezogen wird, bewegt sich der Rahmen des Inhalts mit hoher Geschwindigkeit in die Richtung, in die die Maus bewegt wird. Das Gleiche geschieht, wenn auf ein untergeordnetes Element geklickt wurde, das entweder keine Bindungen für Ereignisse der Maustaste 2 hat oder an den Befehl `scrollutil::adaptBtn2EventHandling` übergeben wurde, und sich der Fokus außerhalb des Fensters `[focusCheckWindow descendant]` befindet.

HINWEIS: Unter Android bedeutet die Unterstützung für das Scannen mit der rechten Maustaste, dass das Scannen auch über Ein-Finger-Gesten möglich ist, da schnelle Wischbewegungen mit einem Finger von AndroWish in die Ereignisse `<Button-2>`, `<B2-Motion>` und `<ButtonRelease-2>` als Ereignisse umgewandelt.

SCHLÜSSELWÖRTER

scrollableframe, widget, frame, scrollable

Der Befehl `scrollutil::adaptBtn2EventHandling`

NAME

`scrollutil::adaptBtn2EventHandling` – Ereignisbehandlung für Maustaste 2 anpassen

ÜBERSICHT

```
scrollutil::adaptBtn2EventHandling ?widget widget ...?
```

BESCHREIBUNG

Für jedes `widget`-Argument führt der Befehl die folgenden Aktionen aus:

- Wenn `widget` der Pfadname eines tablelist-Widgets ist, setzt dieser Befehl dessen Option `-button2window` auf den Pfadnamen des Toplevel-Fensters, vorausgesetzt, die Tablelist-Version ist 7.6 oder höher (bei früheren Tablelist-Versionen ignoriert der Befehl stillschweigend jedes Tablelist-Widget, das ihm als Argument übergeben wird). Infolgedessen wird ein an den Hauptbereich oder das Bearbeitungsfenster der Tablelist gesendetes Ereignis vom Typ `<Button-2>`, `<B2-Motion>` oder `<ButtonRelease-2>` (genauer gesagt: ein Ereignis der Maustaste 2, das an eine beliebige Komponente der Tablelist gesendet wird, die das Tag `TablelistBody` oder `TablelistEdit` trägt), wie folgt behandelt:
- Andernfalls wird das (erste) Bindungs-Tag gesucht, das Ereignisbindungen für Maustaste 2 enthält und sich sowohl vom Pfadnamen des Toplevel-Fensters als auch von `all` unterscheidet. Wenn die Suche nach diesem Tag erfolgreich ist, ändert der Befehl die Liste der Bindungs-Tags des Widgets, indem er diesem Bindungs-Tag das Tag `Btn2EventRedir`

voranstellt und das Tag `Btn2EventBreak` anhängt. Infolgedessen wird ein an dieses Widget gesendetes Ereignis vom Typ `<Button-2>`, `<B2-Motion>` oder `<ButtonRelease-2>` wie folgt behandelt:

- Wenn der Fokus innerhalb des "tablelist"-Widgets liegt, löst das Ereignis eine Überprüfung des Inhalts der "tablelist" oder ihres Bearbeitungsfensters aus, und es erfolgt keine weitere Verarbeitung des Ereignisses.
- Wenn der Fokus außerhalb des "tablelist"-Widgets liegt, findet kein Scanvorgang innerhalb des Hauptbereichs oder des Bearbeitungsfensters der "tablelist" statt. Stattdessen wird das Ereignis über `event generate` an das übergeordnete Toplevel-Fenster weitergeleitet. Dies löst wiederum eine Überprüfung des Inhalts des (innersten) "scrollableframe"-Widgets aus, das ein Vorfahr von `widget` ist und das gleiche Toplevel-Fenster hat (sofern ein solches "scrollableframe"-Widget vorhanden ist).
- Wenn der Fokus auf oder innerhalb des Fensters `[focusCheckWindow widget]` ist, wird das Ereignis durch das diesem Tag zugeordnete Skript der Bindung verarbeitet, und es erfolgt keine weitere Verarbeitung des Ereignisses.
- Wenn der Fokus außerhalb des Fensters `[focusCheckWindow widget]` ist, wird das Ereignis über `event generate` an das Toplevel-Fenster weitergeleitet, anstatt vom Skript der Bindung verarbeitet zu werden, das dem oben genannten Tag zugeordnet ist. Dies löst wiederum eine Überprüfung des Inhalts des (innersten) "scrollableframe"-Widgets aus, das ein Vorfahr von `widget` ist und das gleiche Toplevel-Fenster hat (wenn ein solches "scrollableframe"-Widget vorhanden ist).

ANMERKUNG 1: Dieser Befehl ist dafür vorgesehen, für Widgets aufgerufen zu werden, die Bindungen für Ereignisse der Maustaste 2 haben und Nachkommen eines scrollableframe sind. Die Tk- und Tile-Widgets, die Klassenbindungen für Ereignisse der Maustaste 2 haben, sind: Tk-Core-Entry, ttk::entry, ttk::combobox, listbox, PanedWindow, Tk-Core-Scale, ttk::scale, Tk-Core-Spinbox, ttk::spinbox, Tk-Core-Scrollbar, ttk::scrollbar und Text. Beispiele für Widgets mit anderen Bindungs-Tags als ihren Namen der Klasse, die Ereignisse für die rechte Maustaste aufweisen, sind ctext- und tablelist-Widgets. (Im Fall eines ctext-Widgets `w` beziehen sich die in der obigen BESCHREIBUNG genannten Bindungs-Tags auf das untergeordnete Haupttext-Widget des Widgets `w.t` und nicht auf das Widget selbst.)

ANMERKUNG 2: Wie oben erwähnt, verfügen die Tk-Core-Scrollbar- und ttk::scrollbar-Widgets über Klassenbindungen für Ereignisse der Maustaste 2; daher sollte dieser Befehl für sie aufgerufen werden, falls sie Nachkommen eines ScrollableFrame-Widgets sind. Da diese Aufgabe mühsam werden kann, sorgt Scrollutil dafür, dass, wenn Sie diesem Befehl ein Widget übergeben und dieses Widget in ein scrollarea über dessen Unterbefehl `setwidget` eingebettet ist, wird dieser Befehl automatisch auch für die Bildlaufleisten dieses scrollarea aufgerufen.

ANMERKUNG 3: Bei der Verarbeitung eines Mausklick-2-Ereignisses, das an eine Tk-Core- oder Tile-Bildlaufleiste gesendet wird, deren Pfadname an diesen Befehl übergeben wurde, wird das Ereignis von der Bildlaufleiste verarbeitet, anstatt an das übergeordnete Toplevel-Fenster weitergeleitet zu werden, sofern sich der Fokus auf oder innerhalb des zugehörigen Widgets befindet – genau so, als befände sich der Fokus auf der Bildlaufleiste selbst.

SCHLÜSSELWÖRTER

mouse button 2 event, binding, event handling, scanning, scrollableframe, focus

Vergleich mit BWidget ScrollableFrame und iwidgets::scrolledframe

Das Widget "scrollutil::scrollableframe" wurde als schlanker, an das jeweilige Thema anpassbarer und voll ausgestatteter Ersatz für "BWidget ScrollableFrame" und "iwidgets::scrolledframe" entwickelt. Die folgende Tabelle enthält einen Vergleich der Optionen und Befehle, die von den drei Widgets bereitgestellt werden:

scrollutil::scrollableframe	**BWidget ScrollableFrame**	**iwidgets::scrolledframe**
-background	-background	-background
-borderwidth		-borderwidth
-cursor		-cursor
-contentheight	-areaheight	[pathName component canvas] \ itemconfigure frameTag -height ...
-contentwidth	-areawidth	[pathName component canvas] \ itemconfigure frameTag -width ...
-fitcontentheight	-constrainedheight	bind [pathName component canvas] <Configure> \ { %W itemconfigure frameTag -height %h }
-fitcontentwidth	-constrainedwidth	bind [pathName component canvas] <Configure> \ { %W itemconfigure frameTag -width %w }
-height (legt die innere Höhe fest, ohne den Rahmen und das Hervorhebungsrechteck, falls vorhanden)	-height (Innenhöhe = Gesamthöhe)	-height (legt die Gesamthöhe fest, einschließlich des Rahmens und der horizontalen Bildlaufleiste, falls vorhanden)
-highlightbackground		
-highlightcolor		
-highlightthickness		
-relief		-relief
-takefocus		

scrollutil::scrollableframe	**BWidget ScrollableFrame**	**iwidgets::scrolledframe**
-width (legt die innere Breite fest, ohne den Rahmen und das Hervorhebungsrechteck, wenn sie vorhanden sind)	-width (Innenbreite = Gesamtbreite)	-width (legt die Gesamtbreite fest, einschließlich des Rahmens und der vertikalen Bildlaufleiste, wenn vorhanden)
-xscrollcommand	-xscrollcommand	
-xscrollincrement	-xscrollincrement	-xscrollincrement für [pathName component canvas]
-yscrollcommand	-yscrollcommand	
-yscrollincrement	-yscrollincrement	-yscrollincrement für [pathName component canvas]
attrib		
autosize		
autofillx		
autofilly		
cget	cget	cget
configure	configure	configure
contentframe	getframe	childsite
hasattrib		
preparescan		
preparescroll		
scan (mit Mausereignis-Bindungen)		
see widget ?nw|ne|sw|se?	see widget ?top|bottom ?left|right??	
seerect x1 y1 x2 y2 ?nw|ne|sw|se?		
unsetattrib		
xview	xview	xview

scrollutil::scrollableframe	BWidget ScrollableFrame	iwidgets::scrolledframe
yview	yview	yview
xview|yview units		
xview|yview moveto 0|1	xview|yview moveto 0|1	justify left|right|top|bottom (Kurzbezeichnung für xview|yview moveto 0|1)

ANMERKUNG 1: Beim Befehl `see` des BWidget-Widgets "ScrollableFrame" kann die horizontale Seite nicht ohne Angabe der vertikalen Seite festgelegt werden. Noch ärgerlicher ist jedoch, dass bei einem positiven Wert für `-xscrollincrement` oder `-yscrollincrement` Widgets, die klein genug sind, um in das Fenster zu passen, häufig nur teilweise sichtbar sind. Zudem funktioniert der Befehl nur für direkte untergeordnete Elemente des Inhaltsrahmens des ScrollableFrame-Widgets.

ANMERKUNG 2: Das Widget iwidgets::scrolledframe verfügt über keine Entsprechungen zu den Optionen scrollutil::scrollableframe, `-contentheight`, `-contentwidth`, `-fitcontentheight` und `-fitcontentwidth`, diese lassen sich jedoch (unter Verwendung undokumentierter interner Informationen!) wie oben gezeigt nachbilden. Es bietet einen Befehl `justify`, der jedoch lediglich eine Abkürzung für `xview|yview moveto 0|1` ist. Was diesem Widget wirklich fehlt, ist ein Befehl `see`, um einzelne Widgets im Fenster sichtbar zu machen (was für eine benutzerfreundliche Tastaturnavigation wichtig ist).

Stichwörter: scrollableframe

scrollarea

Das `scrollutil::scrollarea` und `scrollutil::getscrollarea` BEFEHLE

Für Scrollutil Version 2.11

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

- Der Befehl scrollutil::scrollarea – Kurzreferenz
- Der Befehl scrollutil::scrollarea – Ausführliche Referenz
- Der Befehl scrollutil::getscrollarea

Der Befehl `scrollutil::scrollarea` – Kurzreferenz

NAME

`scrollutil::scrollarea` – Erstellen und Bearbeiten von Scrollarea-Widgets

ÜBERSICHT

```
scrollutil::scrollarea pathName ?options?
```

STANDARDOPTIONEN

```
-background   -highlightbackground  -relief
-borderwidth  -highlightcolor
-cursor       -highlightthickness
```

WIDGET-SPEZIFISCHE OPTIONEN

```
-autohidescrollbars boolean

-lockinterval milliseconds

-respectheader boolean

-respecttitlecolumns boolean

-setfocus boolean

-takefocus 0|1|""|command

-xscrollbarmode static|dynamic|none

-yscrollbarmode static|dynamic|none
```

Widget COMMAND

```
pathName attrib ?name ?value name value ...??

pathName cget option

pathName configure ?option ?value option value ...??

pathName hasattrib name

pathName setwidget widget

pathName unsetattrib name

pathName widget
```

SCHLÜSSELWÖRTER

Bildlaufbereich, Widget, Bildlaufleiste

Der Befehl `scrollutil::scrollarea` – Ausführliche Referenz

NAME

`scrollutil::scrollarea` – Erstellen und Bearbeiten von Scrollarea-Widgets

ÜBERSICHT

```
scrollutil::scrollarea pathName ?options?
```

BESCHREIBUNG

Der Befehl `scrollutil::scrollarea` erstellt ein neues Fenster mit dem Namen `pathName` und der Klasse `Scrollarea` und wandelt es in ein **scrollarea**-Widget um. Zusätzliche Optionen, die im Folgenden beschrieben werden, können über die Befehlszeile oder in der Optionsdatenbank angegeben werden, um Aspekte des **scrollarea**-Widgets zu konfigurieren, wie beispielsweise die Rahmenbreite, die Relief-Einstellung und den Anzeigemodus für die Bildlaufleisten. Der Befehl `scrollutil::scrollarea` gibt sein Argument `pathName` zurück. Zum Zeitpunkt des Aufrufs dieses Befehls darf kein Fenster mit dem Namen `pathName` vorhanden sein, aber das übergeordnete Fenster von `pathName` muss vorhanden sein.

Ein Scrollarea ist ein Megawidget, das aus einem scrollbaren Widget – das mithilfe des Unterbefehls `setwidget` des zugehörigen Tcl-Befehls festgelegt wird – sowie zwei mit diesem Widget verbundenen Bildlaufleisten besteht. Diese Komponenten werden innerhalb der Scrollarea mithilfe von `grid` verwaltet. Die Bildlaufleisten mit den Namen `hsb` (zusammen mit `-orient horizontal`) und `vsb` (zusammen mit `-orient vertical`) sind direkte untergeordnete Elemente des scrollarea. Der Anzeigemodus jeder Bildlaufleiste kann `static`, `dynamic` oder `none` lauten (siehe die Konfigurationsoptionen `-xscrollbarmode` und `-yscrollbarmode`). Die Option `-takefocus` beider Bildlaufleisten ist auf `0` gesetzt. Im Paket "Scrollutil_tile" werden die Bildlaufleisten als ttk::scrollbar-Widgets erstellt, außer unter Mac OS X/11+ bei Verwendung einer Tk-Version vor 8.6.10, wo das native ttk::scrollbar-Widget des `aqua`-Themas noch nicht wie erwartet aussah.

Wenn das über `setwidget` in den Scrollbereich eingebettete Widget ein tablelist und Tablelist Version 6.5 oder höher verwendet wird, kann der Scrollbereich auch gleichrangige Elemente des Tablelist-Widgets oberhalb der vertikalen Bildlaufleiste und/oder links von der horizontalen Bildlaufleiste enthalten, was dazu führt, dass die vertikale Bildlaufleiste unterhalb der Kopfzeile der Tablelist angezeigt wird und/oder die horizontale Bildlaufleiste rechts neben dem Titelbereich der Tablelist erscheint, abhängig von den Werten der Konfigurationsoptionen `-respectheader` und `-respecttitlecolumns`.

Das folgende Beispiel zeigt die typischen Schritte zum Erstellen eines Widgets innerhalb eines Scrollbereichs:

```
set sa [scrollutil::scrollarea ...]
set lb [listbox $sa.lb ...]
$sa setwidget $lb

pack $sa -expand yes -fill both
```

STANDARDOPTIONEN

```
-background   -highlightbackground  -relief
-borderwidth  -highlightcolor
-cursor       -highlightthickness
```

Weitere Informationen zu den Standardoptionen für Tk-Widgets finden Sie im Handbucheintrag zu den **options**. Die Optionen `-background`, `-highlightbackground`, `-highlightcolor` und `-highlightthickness` werden nur vom Scrollutil-Paket unterstützt, nicht jedoch von Scrollutil_tile. Sie haben dieselben Standardwerte wie die gleichnamigen Optionen für Tk-Frame-Widgets. Die Standardwerte der übrigen Standardoptionen lauten:

```
-borderwidth 1 -cursor "" -relief sunken
```

HINWEIS: Wenn Sie die Option `-borderwidth` oder `-relief` konfigurieren und die Scrollarea infolge dieser Aktion einen positiven `-borderwidth`-Wert aufweist (z. B. der Standardwert `1`) und einen `-relief`-Wert aufweist, der nicht `flat` ist (z. B. den Standardwert `sunken`), dann wird die Option `-borderwidth` des Widgets, das über den Unterbefehl `setwidget` des zugehörigen Tcl-Befehls in den Scrollbereich eingebettet wurde, automatisch auf `0` gesetzt, vorausgesetzt, das eingebettete Widget unterstützt diese Option.

WIDGET-SPEZIFISCHE OPTIONEN

Befehlszeilenname:	-autohidescrollbars
Name der Datenbank:	autoHideScrollbars
Datenbankklasse:	AutoHideScrollbars

Legt einen booleschen Wert fest, der festlegt, ob die Bildlaufleisten ausgeblendet werden sollen, wenn sich der Fokus außerhalb des Toplevel-Fensters befindet, das den Bildlaufbereich enthält, oder wenn sich der Mauszeiger außerhalb des Bildlaufbereichs-Widgets befindet. Der Standardwert ist `0`.

ANMERKUNG 1: Der einfachste Weg, den Standardwert für alle "scrollarea"-Instanzen Ihrer Anwendung zu überschreiben, besteht darin, die folgende Zeile einzufügen:

```
option add *Scrollarea.autoHideScrollbars 1
```

irgendwo am Anfang Ihres Skripts (bevor Sie Scrollarea-Widgets erstellen).

ANMERKUNG 2: Um eine Bildlaufleiste auszublenden, legt Scrollutil einfach einen Rahmen darüber, wodurch der von ihr eingenommene Bereich reserviert bleibt (wenn er zuvor zugeordnet war). Dies unterscheidet sich vom Aufheben der Zuordnung einer dynamischen Bildlaufleiste, wenn in dem in den Bildlaufbereich eingebetteten Widget nichts zum Scrollen vorhanden ist. Das Ausblenden oder Einblenden einer Bildlaufleiste ändert nichts an ihrem Status "zugeordnet" bzw. "nicht zugeordnet".

ANMERKUNG 3: Unabhängig vom Fokus und dem Wert dieser Option werden die Bildlaufleisten sichtbar, wenn sich der Mauszeiger innerhalb des Bildlaufbereichs befindet und das über `setwidget` darin eingebettete Widget gescrollt wird (z. B. mithilfe des Mausrads oder des Touchpads).

Befehlszeilenname:	-lockinterval

Name der Datenbank:	lockInterval
Datenbankklasse:	LockInterval

Legt das Zeitintervall in Millisekunden fest, für das die Bildlaufleisten mit dem Anzeigemodus `dynamic` (siehe die Optionen `-xscrollbarmode` und `-yscrollbarmode`) vor dem Entmappen geschützt sind, nachdem sie gemappt wurden, um Shimmering-Effekte zu vermeiden. Dies ist auch das Zeitintervall, während dessen sich die dynamischen Bildlaufleisten gegenseitig davor schützen, neu zugeordnet zu werden, nachdem eine von ihnen neu zugeordnet wurde. Ohne diesen Sperrmechanismus könnte eine dynamische Bildlaufleiste unter bestimmten seltenen Umständen in einer Endlosschleife immer wieder neu zugeordnet und wieder entfernt werden, was zu einem störenden und oft gefährlichen Flackereffekt führen würde. Das gleiche Problem kann aufgrund eines zu kleinen Werts für `-lockinterval` auftreten. Der Standardwert ist `1`, der in den allermeisten Fällen wie erwartet funktioniert. Sollten Sie in einem Ihrer Scrollarea-Widgets ein Shimmering feststellen, setzen Sie diese Option für die betreffende Scrollarea einfach auf einen ausreichend großen Wert (z. B. `100`). HINWEIS: Dieser Wert sollte nicht größer als `300` sein, da aus implementierungsspezifischen Gründen ein Wert von `-lockinterval`, der größer als `300` ist, dazu führen kann, dass eine dynamische Bildlaufleiste, die ausgeblendet werden sollte, nicht entfernt wird, oder dass eine, die angezeigt werden sollte, nicht eingeblendet wird.

Befehlszeilenname:	-respectheader
Name der Datenbank:	respectHeader
Datenbankklasse:	RespectHeader

Diese Option ist nur relevant, wenn es sich bei dem Widget, das mithilfe des Unterbefehls `setwidget` des zugehörigen Tcl-Befehls in den Scrollbereich eingebettet wurde, um ein tablelist handelt und die verwendete Tablelist-Version 6.5 oder höher ist. Sein Wert muss ein Boolescher Wert sein, der festlegt, ob die vertikale Bildlaufleiste unterhalb der Kopfzeile des Tablelist-Widgets angezeigt werden soll, wodurch das native Look & Feel unter Mac OS X/11+ und auf vielen modernen Linux-Systemen beibehalten wird. Der Standardwert ist `1` auf den Fenstersystemen `aqua` und `x11`, und `0` auf `win32`.

Befehlszeilenname:	-respecttitlecolumns
Name der Datenbank:	respectTitleColumns
Datenbankklasse:	RespectTitleColumns

Diese Option ist nur relevant, wenn es sich bei dem Widget, das mithilfe des Unterbefehls `setwidget` des zugehörigen Tcl-Befehls in den Scrollbereich eingebettet wurde, um ein tablelist handelt und die verwendete Tablelist-Version 6.5 oder höher ist. Ihr Wert muss ein Boolescher Wert sein, der festlegt, ob die horizontale Bildlaufleiste rechts neben der nicht scrollbaren Spalte des Titels des Tablelist-Widgets beginnen soll, wenn der Wert der Tablelist-Option

`-titlecolumns` positiv ist. Der Standardwert ist `1`.

Befehlszeilenname:	-setfocus
Name der Datenbank:	setFocus
Datenbankklasse:	SetFocus

Legt einen booleschen Wert fest, der bestimmt, ob ein Klick mit Maustaste 1 oder 2 auf eine der Bildlaufleisten den Fokus mithilfe des Unterbefehls `setwidget` auf das in den Bildlaufbereich eingebettete Widget setzen soll, sofern sich dieses Widget nicht im deaktivierten Zustand befindet. Der Standardwert ist `0`.

ANMERKUNG 1: Wenn der Wert der Option `1` lautet und es sich bei dem eingebetteten Widget um ein Text- oder Ctext-Widget handelt, wird durch einen Klick mit Maustaste 1 oder 2 auf eine der Bildlaufleisten der Fokus auf dieses Widget gesetzt, unabhängig vom Status des Widgets, vorausgesetzt, das Fenstersystem ist `win32` oder die Tk-Version ist mindestens 8.6.11 oder 8.7a4. Dies entspricht dem Verhalten, das die VORGABEBINDUNGEN dieser Widgets bei Ereignissen vom Typ `<Button-1>` zeigen. (Im Falle eines ctext-Widgets `w` wird der Fokus auf das untergeordnete Haupttext-Widget `w.t` gesetzt und nicht auf das Widget selbst.)

ANMERKUNG 2: Wenn der Wert der Option `1` lautet und das eingebettete Widget ein ttk::treeview-Widget ist, wird durch einen Klick mit Maustaste 1 oder 2 auf eine der Bildlaufleisten der Fokus auf dieses Widget gesetzt, unabhängig vom Zustand des Widgets, und zwar auf allen Fenstersystemen und für alle Tk-Versionen. Auch hier handelt es sich um dasselbe Verhalten, das die VORGABEBINDUNGEN dieses Widgets für `<Button-1>`-Ereignisse zeigen.

ANMERKUNG 3: Der einfachste Weg, den Standardwert für alle "scrollarea"-Instanzen Ihrer Anwendung zu überschreiben, besteht darin, die folgende Zeile einzufügen:

```
option add *Scrollarea.setFocus 1
```

irgendwo am Anfang Ihres Skripts (bevor Sie Scrollarea-Widgets erstellen).

Befehlszeilenname:	-takefocus
Name der Datenbank:	takeFocus
Datenbankklasse:	TakeFocus

Diese Option legt fest, ob der Scrollbereich während der Tastaturnavigation den Fokus annimmt. Sie ist nahezu identisch mit der gleichnamigen Standardoption (Einzelheiten finden Sie im Handbucheintrag **options**). Der einzige Unterschied besteht darin, dass nicht der Scrollbereich selbst, sondern das Widget, das über den Unterbefehl `setwidget` des zugehörigen Tcl-Befehls in ihn eingebettet wurde, den Fokus bei der Tastaturnavigation mit den Standardtasten erhält (`Tab` und `Shift-Tab`). Die Standardeinstellung ist `0`, da ein Scrollbereich im Wesentlichen ein Container ist, der das oben genannte Widget und zwei Bildlaufleisten enthält.

Befehlszeilenname:	-xscrollbarmode

Name der Datenbank:	xScrollbarMode
Datenbankklasse:	ScrollbarMode

Legt den Anzeigemodus fest, der für die horizontale Bildlaufleiste verwendet werden soll. Die zulässigen Werte sind `static`, `dynamic` und `none`. Im Modus `static` wird die Bildlaufleiste stets angezeigt. Im Modus `dynamic` (dem Standardmodus) wird die Bildlaufleiste je nach Bedarf ein- und ausgeblendet. Der Anzeigemodus `none` deaktiviert die Anzeige der Bildlaufleiste.

Befehlszeilenname:	-yscrollbarmode
Name der Datenbank:	yScrollbarMode
Datenbankklasse:	ScrollbarMode

Legt den Anzeigemodus fest, der für die vertikale Bildlaufleiste verwendet werden soll. Die zulässigen Werte sind `static`, `dynamic` und `none`. Im Modus `static` wird die Bildlaufleiste stets angezeigt. Im Modus `dynamic` (dem Standardmodus) wird die Bildlaufleiste je nach Bedarf ein- und ausgeblendet. Der Anzeigemodus `none` deaktiviert die Anzeige der Bildlaufleiste.

WIDGET-BEFEHL

Der Befehl `scrollutil::scrollarea` erstellt einen neuen Tcl-Befehl mit dem Namen `pathName`. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget auszuführen. Er hat die folgende allgemeine Form:

```
pathName option ?arg arg ...?
```

`option` und `arg` legen das genaue Verhalten des Befehls fest. Für Scrollarea-Widgets sind folgende Befehle möglich:

`pathName attrib ?name ?value name value ...??`

Fragt die Attribute des Widgets ab oder ändert sie. Wenn kein `name` angegeben ist, gibt der Befehl eine Liste von Paaren zurück, von denen jedes den Namen und den Wert eines Attributs für `pathName` enthält. Wenn `name` ohne `value` festgelegt wird, gibt der Befehl den Wert des genannten Attributs zurück oder eine leere Zeichenkette, falls kein entsprechender Wert vorhanden ist (mit dem Unterbefehl `hasattrib` können Sie diesen Fall von dem unterscheiden, bei dem der Wert eines *vorhandenen* Attributs eine leere Zeichenkette ist). Wenn ein oder mehrere Paare aus `name` und `value` angegeben werden, legt der Befehl das/die angegebene(n) Widget-Attribut(e) auf den/die angegebenen Wert(e) fest; in diesem Fall ist der Rückgabewert eine leere Zeichenkette. Jedes `name` kann eine beliebige Zeichenkette sein.

`pathName cget option`

Gibt den aktuellen Wert der durch `option` angegebenen Konfigurationsoption zurück, die einen beliebigen der vom Befehl `scrollutil::scrollarea` akzeptierten Werte annehmen kann.

`pathName configure ?option ?value option value ...??`

Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn kein `option` angegeben ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für `pathName` beschreibt (Informationen zum Format dieser Liste finden Sie unter `Tk_ConfigureInfo`). Wenn `option` ohne `value` festgelegt wird, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste entspricht der entsprechenden Unterliste des gegebenen Werts, wenn kein `option` festgelegt wurde). Wenn ein oder mehrere Paare aus `option` und `value` festgelegt werden, ändert der Befehl die festgelegten Widget-Optionen so, dass sie die festgelegten Werte annehmen; in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. `option` kann jeden der vom Befehl `scrollutil::scrollarea` akzeptierten Werte annehmen.

`pathName hasattrib name`

Gibt `1` zurück, wenn das Attribut `name` vorhanden ist, andernfalls `0`.

`pathName setwidget widget`

Das Argument `widget` muss der Pfadname eines vorhandenen Widgets oder eine leere Zeichenkette sein. Im ersten Fall sorgt der Befehl dafür, dass das Widget den oberenlinken oberen Teil des Scrollbereichs ausfüllt und es mit den Bildlaufleisten verbindet, indem er die Optionen `-xscrollcommand` und `-yscrollcommand` auf entsprechende Wrapper für den Befehl `set` der beiden Bildlaufleisten setzt und die Option `-command` Option der Bildlaufleisten auf `[list widget xview]` bzw. `[list widget yview]`. Wenn `widget` eine leere Zeichenkette ist, ist das an den letzten `setwidget`-Aufruf (falls vorhanden) übergebene Widget nicht verwaltet und nicht mit den Bildlaufleisten verbunden. Der Wert wird als Argument an den letzten erfolgreichen Aufruf dieses Unterbefehls übergeben, oder als leere Zeichenkette, wenn zuvor kein erfolgreicher Aufruf von `setwidget` stattgefunden hat.

ANMERKUNG 1: Wenn `widget` nicht leer ist und der Wert der Option `-xscrollbarmode` sich von dem der Option `none` unterscheidet, muss `widget` ein horizontal scrollbares Widget sein, d. h., es muss die Konfigurationsoption `-xscrollcommand` unterstützen und der zugehörige Tcl-Befehl muss den Unterbefehl `xview` enthalten. Ebenso wenn `widget` nicht leer ist und der Wert der Option `-yscrollbarmode` sich von `none` unterscheidet, muss `widget` ein vertikal scrollbares Widget sein, was bedeutet, dass es die Konfigurationsoption `-yscrollcommand` unterstützen muss und der zugehörige Tcl-Befehl den Unterbefehl `yview` enthalten muss. Folglich wenn `widget` ein Eintrag oder ttk::entry ist, ist dieser Unterbefehl nur dann erfolgreich, wenn die Option `-yscrollbarmode` zuvor auf `none` gesetzt wurde.

ANMERKUNG 2: Das durch das Argument `widget` identifizierte Widget muss ein untergeordnetes Element des scrollarea oder eines seiner übergeordneten Elemente sein. Diese geringfügige Einschränkung wird durch den Geometriemanager `grid` auferlegt.

ANMERKUNG 3: Wenn das Widget, dessen Pfadname an `setwidget` übergeben wurde, zerstört wird, wird dieser Unterbefehl automatisch mit einer leeren Zeichenkette als Argument aufgerufen.

ANMERKUNG 4: Dieser Unterbefehl setzt die Option `-highlightthickness` von `widget` auf `0`, wenn `widget` diese Konfigurationsoption unterstützt. Hat der Scrollbereich außerdem

einen positiven Wert für `-borderwidth` (zum Beispiel die Vorgabe `1`) und einen Wert für `-relief` ungleich `flat` (zum Beispiel die Vorgabe `sunken`), setzt dieser Unterbefehl die Option `-borderwidth` von `widget` auf `0`, sofern `widget` diese Option unterstützt.

ANMERKUNG 5: Dieser Unterbefehl ist die einzige unterstützte Möglichkeit, ein Widget in einen Scrollbereich einzubetten. Jeder Versuch, weitere Widgets mithilfe eines beliebigen Geometriemanagers einzubetten, führt zu unvorhersehbaren Ergebnissen!

`pathName unsetattrib name`
Hebt das Attribut `name` auf. Gibt eine leere Zeichenfolge zurück.

`pathName widget`
Gibt das Argument zurück, das beim letzten erfolgreichen Aufruf des Unterbefehls `setwidget` übergeben wurde, oder eine leere Zeichenkette, falls dieser Unterbefehl noch nicht erfolgreich aufgerufen wurde.

VORGABEBINDUNGEN
Wenn ein neuer Scrollbereich erstellt wird, verfügt dieser über keine VORGABEBINDUNGEN für Ereignisse: Scrollbereiche sind nicht für interaktive Funktionen vorgesehen.

SCHLÜSSELWÖRTER
scrollarea, widget, scrollbar

Der Befehl `scrollutil::getscrollarea`

NAME
`scrollutil::getscrollarea` – Abfrage des Bildlaufbereichs, der ein bestimmtes Widget enthält

ÜBERSICHT

```
scrollutil::getscrollarea widget
```

BESCHREIBUNG
Gibt den Pfadnamen des Scrollbereichs zurück, in den das durch das Argument `widget` angegebene Widget über den Unterbefehl `setwidget` des Scrollbereichs eingebettet ist, oder eine leere Zeichenkette, falls kein solches Scrollbereich-Widget vorhanden ist.

SCHLÜSSELWÖRTER
scrollarea, widget

Stichwörter: scrollarea

scrollednotebook

Der Befehl `scrollutil::scrollednotebook`

und das Stilelement `closetab`

Für Scrollutil Version 2.11

von

Csaba Nemethi

csaba.nemethi@t-online.de

Inhalt

- Der Befehl scrollutil::scrollednotebook – Kurzreferenz
- Der Befehl scrollutil::scrollednotebook – Ausführliche Referenz
- Das "Closetab"-Style-Element
- Der Befehl scrollutil::addclosetab
- Der Befehl scrollutil::removeclosetab
- Der Befehl scrollutil::closetabstate

Der Befehl `scrollutil::scrollednotebook` – Kurzreferenz

NAME

`scrollutil::scrollednotebook` – Erstellen und Bearbeiten von "scrollednotebook"-Widgets

ÜBERSICHT

```
package require scrollutil_tile
scrollutil::scrollednotebook pathName ?options?
```

STANDARDOPTIONEN

```
-class  -cursor  -style
```

WIDGET-SPEZIFISCHE OPTIONEN

```
-forgetcommand command

-height screenDistance

-leavecommand command

-movabletabs boolean

-padding paddingSpec

-takefocus 0|1|""|command

-width screenDistance
```

Widget COMMAND

```
pathName add window ?options?

pathName adjustsize

pathName attrib ?name ?value name value ...??

pathName cget option
```

```
pathName closetabstate tabId ?normal|disabled?

pathName configure ?option ?value option value ...??

pathName forget tabId

pathName hasattrib name

pathName hastabattrib tabId name

pathName hide tabId

pathName identify component x y

pathName index tabId

pathName insert pos window ?options?

pathName instate stateSpec ?script?

pathName notebookpath

pathName see tabId

pathName select ?tabId?

pathName state ?stateSpec?

pathName style

pathName tab tabId ?option ?value option value ...??

pathName tabattrib tabId ?name ?value name value ...??

pathName tabs

pathName unsetattrib name

pathName unsettabattrib tabId name
```

SCHLÜSSELWÖRTER

scrollednotebook, widget, notebook, scrolled

Der Befehl `scrollutil::scrollednotebook` – Ausführliche Referenz

NAME

`scrollutil::scrollednotebook` – "ScrolledNotebook"-Widgets erstellen und bearbeiten

ÜBERSICHT

```
package require scrollutil_tile
scrollutil::scrollednotebook pathName ?options?
```

Es gibt einen HINWEIS, dass der Befehl `scrollutil::scrollednotebook` vom Paket "Scrollutil_tile" bereitgestellt wird, nicht jedoch von Scrollutil.

BESCHREIBUNG

Der Befehl `scrollutil::scrollednotebook` erstellt ein neues Fenster mit dem Namen `pathName` und wandelt es in ein **scrollednotebook**-Widget um. Zusätzliche Optionen, die im Folgenden beschrieben werden, können über die Befehlszeile oder in der Optionsdatenbank angegeben werden, um Aspekte des "scrollednotebook"-Widgets wie dessen Breite, Höhe und Stil zu konfigurieren. Der Befehl `scrollutil::scrollednotebook` gibt sein Argument `pathName` zurück. Zum Zeitpunkt der Ausführung dieses Befehls darf kein Fenster mit dem Namen `pathName` vorhanden sein, das übergeordnete Fenster von `pathName` muss jedoch vorhanden sein.

Ein "scrollednotebook" ist ein Megawidget, das ein ttk::notebook innerhalb eines scrollableframe enthält und eine beliebige Anzahl von nicht komprimierten Registerkarten unterstützt. Derzeit nicht sichtbare Registerkarten können durch Navigation mit dem Mausrad, dem Touchpad oder der Tastatur sowie durch Scrollen mithilfe der beiden Pfeiltasten, die je nach Bedarf in den oberen linken und rechten bzw. unteren linken und rechten Ecken platziert werden (abhängig vom style). Die Größe der Pfeile hängt vom Skalierungsfaktor von Tk ab, während ihre Farbe themenspezifisch ist und bei jedem Wechsel des Themas aktualisiert wird. Ein einzelner Reiter kann programmgesteuert mithilfe des Unterbefehls `see` sichtbar gemacht werden. Im Gegensatz zum Widget ttk::notebook, dessen Option `-width` häufig durch die Gesamtbreite der Registerkarten überschrieben wird, beachtet das "scrollednotebook"-Widget den Wert seiner Option `-width`, unabhängig vom Platzbedarf der Registerkarten.

Die API des "scrollednotebook"-Widgets ist nahezu identisch mit der von ttk::notebook. Abgesehen von einigen zusätzlichen, für "scrollednotebook" spezifischen Unterbefehlen bietet der mit einem "scrollednotebook"-Widget verknüpfte Tcl-Befehl dieselben Unterbefehle wie der mit einem "ttk::notebook" verknüpfte.

STANDARDOPTIONEN

```
-class  -cursor  -style
```

Einzelheiten zu den Standardoptionen finden Sie im Handbucheintrag zu **ttk_widget**. Der Wert der Option `-class` lautet `Scrollednotebook`, der der Option `-cursor` ist eine leere Zeichenkette. Die Option `-style`, deren Wert `TNotebook` ist, bezieht sich auf den Stil des im "scrollednotebook" enthaltenen Widgets ttk::notebook. Der Wert der Option `-tabposition` für den durch diese Option festgelegten Stil muss einer der folgenden Werte sein: `nw` (Standard), `n` (Standardwert für das Thema `aqua`), `ne`, `sw`, `s` oder `se`. Diese Werte führen zu einem horizontalen Registerlayout (das "scrollednotebook"-Widget unterstützt keine vertikalen Registerlayouts).

WIDGET-SPEZIFISCHE OPTIONEN

Befehlszeilenname:	-forgetcommand
Name der Datenbank:	forgetCommand
Datenbankklasse:	ForgetCommand

Legt einen Befehl fest, der aufgerufen werden soll, wenn versucht wird, eine Registerkarte zu entfernen und das zugehörige Fenster mithilfe des Unterbefehls `forget` aus der Verwaltung zu nehmen. Wenn der Wert der Option eine nicht leere Zeichenkette ist, wird diese mit dem Pfadnamen des "scrollednotebook"-Widgets und dem des Fensters, dessen Verwaltung aufgehoben werden soll, verkettet, und das resultierende Skript, das einen booleschen Wert geben muss, wird im globalen Geltungsbereich ausgewertet. Wenn der Wert des Skripts "false" ist, wird der Unterbefehl `forget` abgebrochen, d. h., die Registerkarte wird nicht entfernt und das zugehörige Fenster wird nicht aus der Verwaltung entlassen. Der Standardwert ist eine leere Zeichenkette.

Befehlszeilenname:	-height
Name der Datenbank:	height
Datenbankklasse:	Height

Falls vorhanden und größer als Null, legt dieser Wert die gewünschte Höhe der Teilfläche des Fensters in einem der von `Tk_GetPixels` akzeptierten Formate fest. Andernfalls wird die Höhe des Widgets auf die angeforderte Höhe des darin enthaltenen ttk::notebook gesetzt (die wiederum von der maximal angeforderten Höhe aller Teilflächen und Registerkarten sowie dem zusätzlichen vertikalen Platz abhängt, der sich aus dem Wert der Option `-padding` ergibt), unmittelbar nachdem das "scrollednotebook" zugeordnet wurde. Der Standardwert ist `0`.

Befehlszeilenname:	-leavecommand
Name der Datenbank:	leaveCommand
Datenbankklasse:	LeaveCommand

Legt einen Befehl fest, der aufgerufen werden soll, wenn versucht wird, das aktuell ausgewählte Fenster zu verlassen, indem ein anderes ausgewählt wird. Wenn der Wert der Option eine nicht leere Zeichenkette ist, wird diese mit dem Pfadnamen des "scrollednotebook"-Widgets und dem des aktuell ausgewählten Fensters verkettet, und das resultierende Skript, das einen booleschen Wert geben muss, wird im globalen Geltungsbereich ausgewertet. Wenn der Wert des Skripts "false" ist, wird der Vorgang der Auswahl eines anderen Fensters abgebrochen. Der Standardwert ist eine leere Zeichenkette.

Befehlszeilenname:	-movabletabs
Name der Datenbank:	movableTabs

Datenbankklasse:	MovableTabs

Legt einen booleschen Wert fest, der bestimmt, ob die Registerkarten mit der Maus verschoben werden können. Weitere Informationen finden Sie im Abschnitt WEITERE BINDUNGEN weiter unten. Der Standardwert ist `1`.

Befehlszeilenname:	-padding
Name der Datenbank:	padding
Datenbankklasse:	Padding

Legt fest, wie viel zusätzlicher Abstand um das im "scrollednotebook" enthaltene Widget "ttk::notebook" herum hinzugefügt werden soll. Weitere Informationen finden Sie im Handbucheintrag zu **ttk_notebook**. Der Standardwert ist eine leere Zeichenkette. Beachten Sie, dass bei Verwendung dieses Standardwerts das Widget ttk::notebook mit dem äußeren Innenabstand gezeichnet wird, der durch die Option `-padding` des Widget-Stils festgelegt ist.

Befehlszeilenname:	-takefocus
Name der Datenbank:	takeFocus
Datenbankklasse:	TakeFocus

Diese Option legt fest, ob das "scrollednotebook"-Widget während der Tastaturnavigation den Fokus annimmt. Sie ist nahezu identisch mit der gleichnamigen Standardoption (Einzelheiten finden Sie im Handbucheintrag **options**). Der einzige Unterschied besteht darin, dass nicht das "scrollednotebook"-Widget selbst, sondern das darin enthaltene "ttk::notebook" den Fokus erhält, wenn die Standardtasten (`Tab` und `Shift-Tab`) bei der Tastaturnavigation gedrückt werden. Die Standardeinstellung ist `"ttk::takefocus"` (genau wie bei ttk::notebook und einigen anderen Widgets im Tk-Thema).

Befehlszeilenname:	-width
Name der Datenbank:	width
Datenbankklasse:	Width

Falls vorhanden und größer als Null, legt dieser Wert die gewünschte Breite des im Widget enthaltenen ttk::notebook fest, und zwar in einer der von `Tk_GetPixels` akzeptierten Formen. Andernfalls wird die Breite des Widgets auf die angeforderte Breite des darin enthaltenen ttk::notebook im Container gesetzt (die wiederum von der maximal angeforderten Breite aller Teilflächen, der Gesamtbreite der Registerkarten und dem zusätzlichen horizontalen Platz abhängt, der sich aus dem Wert der Option `-padding` ergibt), unmittelbar nachdem das "scrollednotebook" zugeordnet wurde. Der Standardwert ist `10c`, der durch einen geeigneten anwendungsspezifischen Wert überschrieben werden sollte.

REGISTEROPTIONEN

Siehe den Handbucheintrag zu **ttk_notebook**.

REGISTERKENNUNGEN

Siehe den Handbucheintrag zu **ttk_notebook**.

WIDGET-BEFEHL

Der Befehl `scrollutil::scrollednotebook` erstellt einen neuen Tcl-Befehl mit dem Namen `pathName`. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget auszuführen. Er hat die folgende allgemeine Form:

```
pathName option ?arg arg ...?
```

`option` und die `args` legen das genaue Verhalten des Befehls fest. Für "scrollednotebook"-Widgets sind folgende Befehle möglich:

`pathName add window ?options?`

Siehe den Handbucheintrag zu **ttk_notebook**.

`pathName adjustsize`

Setzt die Option `-height` des "scrollednotebook" auf die maximal angeforderte Höhe aller Teilflächen und die Option `-width` auf die maximale angeforderte Breite aller Teilflächen, erhöht um den zusätzlichen horizontalen Platz, der sich aus dem Wert der Option `-padding` ergibt.

`pathName attrib ?name ?value name value ...??`

Fragt die Attribute des Widgets ab oder ändert sie. Wenn kein `name` angegeben ist, gibt der Befehl eine Liste von Paaren zurück, von denen jedes den Namen und den Wert eines Attributs für `pathName` enthält. Wenn `name` ohne `value` festgelegt wird, gibt der Befehl den Wert des genannten Attributs zurück oder eine leere Zeichenkette, falls kein entsprechender Wert vorhanden ist (mit dem Unterbefehl `hasattrib` können Sie diesen Fall von dem unterscheiden, bei dem der Wert eines *vorhandenen* Attributs eine leere Zeichenkette ist). Wenn ein oder mehrere Paare aus `name` und `value` angegeben werden, legt der Befehl das/die angegebene(n) Widget-Attribut(e) auf den/die angegebenen Wert(e) fest; in diesem Fall ist der Rückgabewert eine leere Zeichenkette. Jedes `name` kann eine beliebige Zeichenkette sein.

`pathName cget option`

Gibt den aktuellen Wert der durch `option` angegebenen Konfigurationsoption zurück, die einen der vom Befehl `scrollutil::scrollednotebook` akzeptierten Werte annehmen kann.

`pathName closetabstate tabId ?normal|disabled?`

Legt den Status des Elements `closetab` der durch `tabId` angegebenen Registerkarte fest oder fragt ihn ab.

HINWEIS: Der Aufruf dieses Unterbefehls ist lediglich eine Abkürzung, um die Parameter `pathName`, `tabId` und das optionale Argument (falls vorhanden) an den Befehl `scrollutil::closetabstate` zu übergeben.

`pathName configure ?option ?value option value ...??`

Ruft die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn kein `option` festgelegt ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für `pathName` beschreibt (Informationen zum Format dieser Liste finden Sie unter `Tk_ConfigureInfo`).

Wenn `option` ohne `value` festgelegt wird, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste entspricht der entsprechenden Unterliste des gegebenen Werts, wenn kein `option` festgelegt wurde). Wenn ein oder mehrere Paare aus `option` und `value` angegeben werden, ändert der Befehl die angegebenen Widget-Optionen so, dass sie die angegebenen Werte annehmen; in diesem Fall ist der Rückgabewert eine leere Zeichenkette. `option` kann jeden der vom Befehl `scrollutil::scrollednotebook` akzeptierten Werte annehmen.

`pathName forget tabId`
Siehe den Handbucheintrag zu **ttk_notebook**.

`pathName hasattrib name`
Gibt `1` zurück, wenn das Attribut `name` vorhanden ist, andernfalls `0`.

`pathName hastabattrib tabId name`
Gibt `1` zurück, wenn das Attribut `name` für die durch `tabId` angegebene Registerkarte vorhanden ist, andernfalls `0`.

`pathName hide tabId`
Siehe den Handbucheintrag zu **ttk_notebook**.

`pathName identify component x y`
Siehe den Handbucheintrag zu **ttk_notebook**.

`pathName index tabId`
Siehe den Handbucheintrag zu **ttk_notebook**.

`pathName insert pos window ?options?`
Siehe den Handbucheintrag zu **ttk_notebook**.

`pathName instate stateSpec ?script?`
Siehe den Handbucheintrag zu **ttk_widget**.

`pathName notebookpath`
Gibt den Pfadnamen des im "scrollednotebook" enthaltenen Widgets "ttk::notebook" zurück. Einer der äußerst seltenen Fälle, in denen dieser Unterbefehl benötigt wird, hängt mit der Einstellung eines Tooltips für eine der Registerkarten zusammen. Wenn `nb` den von diesem Unterbefehl gegebenen Pfad ttk::notebook bezeichnet, können Sie einen Tooltip `message` für eine seiner Registerkarten festlegen, indem Sie entweder die Option `-tab` des Befehls `tooltip::tooltip` aus dem in tklib enthaltenen Tooltip-Paket verwenden:

```
tooltip::tooltip nb -tab tabId message
```

oder die Option `-nbktab` des Befehls `baltip::tip` aus dem baltip-Paket von Alex Plotnikov:

```
baltip::tip nb message -nbktab tabWindow ...
```

Die Unterstützung für die Option `-tab` des Befehls `tooltip::tooltip` wurde in Version 1.6 des Tooltip-Pakets hinzugefügt. Der Befehl `baltip::tip` akzeptiert zudem eine Vielzahl weiterer Optionen. Das baltip-Paket kann unter folgendem Link heruntergeladen werden:

https://chiselapp.com/user/aplsimple/repository/baltip/download/

HINWEIS: Abgesehen von der oben genannten Ausnahme ist es in den allermeisten Fällen nicht erforderlich, direkt auf das im "scrollednotebook"-Widget enthaltene "ttk::notebook"-Widget zuzugreifen. Darüber hinaus führt die Verwendung der Optionen oder

Unterbefehle von ttk::notebook anstelle derjenigen des "scrollednotebook"-Widgets in den meisten Fällen zu unvorhersehbarem Verhalten.

`pathName see tabId`
Dieser Unterbefehl passt die Ansicht im Fenster von "scrollednotebook" so an, dass die durch `tabId` angegebene Registerkarte darin sichtbar wird. Beachten Sie, dass dieser Befehl automatisch aufgerufen wird, sobald eine Registerkarte ausgewählt wird, um diese Registerkarte sichtbar zu machen.

`pathName select ?tabId?`
Siehe den Handbucheintrag zu **ttk_notebook**.

`pathName state ?stateSpec?`
Siehe den Handbucheintrag zu **ttk_widget**.

`pathName style`
Gibt den Wert der Konfigurationsoption `-style` zurück (d. h. den Stil, der vom Widget ttk::notebook verwendet wird, das im Widget "scrollednotebook" enthalten ist). Für Widgets mit Tk-Thema wurde dieser Unterbefehl in Tk 8.7a4 eingeführt, das "scrollednotebook"-Widget stellt ihn jedoch für alle unterstützten Tk-Versionen bereit.

`pathName tab tabId ?option ?value option value ...??`
Siehe den Handbucheintrag zu **ttk_notebook**.

`pathName tabattrib tabId ?name ?value name value ...??`
Fragt die Attribute der durch `tabId` angegebenen Registerkarte ab oder ändert sie. Wenn kein `name` festgelegt ist, gibt der Befehl eine Liste von Paaren zurück, von denen jedes den Namen und den Wert eines Attributs der Registerkarte enthält. Wenn `name` ohne `value` festgelegt wird, gibt der Befehl den Wert des einen benannten Registerkartenattributs zurück oder eine leere Zeichenkette, falls kein entsprechender Wert vorhanden ist (Sie können den Unterbefehl `hastabattrib` verwenden, um diesen Fall von dem zu unterscheiden, bei dem der Wert eines *vorhandenen* Registerkartenattributs eine leere Zeichenkette ist). Wenn ein oder mehrere Paare aus `name` und `value` gelegt werden, setzt der Befehl das/die angegebene(n) Registerkartenattribut(e) auf den/die angegebenen Wert(e); in diesem Fall gibt der Befehl eine leere Zeichenkette zurück. Jedes `name` kann eine beliebige Zeichenkette sein.

`pathName tabs`
Siehe den Handbucheintrag zu **ttk_notebook**.

`pathName unsetattrib name`
Hebt das Attribut `name` auf. Gibt eine leere Zeichenfolge zurück.

`pathName unsettabattrib tabId name`
Hebt das Attribut `name` für die durch `tabId` angegebene Registerkarte auf. Gibt eine leere Zeichenkette zurück.

TASTATURNAVIGATION
Siehe den Handbucheintrag zu **ttk_notebook**.

VIRTUELLE EREIGNISSE
Wie im Handbucheintrag zu **ttk_notebook** beschrieben, generiert das Widget ein virtuelles Ereignis vom Typ `<<NotebookTabChanged>>`, nachdem eine neue Registerkarte eines ttk::notebook (programmgesteuert oder interaktiv) ausgewählt wurde. Dies gilt auch für "scrollednotebook"-Widgets.

Das Paket "Scrollutil_tile" definiert das virtuelle Ereignis `<<Button3>>` für alle Fenstersysteme als `<Button-3>` und zusätzlich für Mac OS X/11+ als `<Control-Button-1>`. Wenn dieses Ereignis über einer Registerkarte eines ttk::notebook oder eines scrollednotebook auftritt, dessen Widget-Statusflag `disabled` nicht gesetzt ist, und die Tk-Version 8.5a2 oder höher ist, dann erzeugt das Widget ein virtuelles Ereignis `<<MenuItemsRequested>>`, indem es `event generate` aufruft, wobei die Option `-data` auf eine Liste gesetzt ist, die aus dem Pfadnamen eines leeren Popup-Menüs und dem numerischen Index der betreffenden Registerkarte besteht. (Die Unterstützung für diese Option wurde in Tk 8.5a2 eingeführt.) Wenn die Anwendung eine Bindung für dieses virtuelle Ereignis erstellt, kann das Bindungsskript auf die Benutzerdaten als Wert des Feldes `%d` zugreifen, um das Menü mit anwendungsspezifischen Einträgen zu füllen. HINWEIS: Das Popup-Menü wird mit einer Verzögerung von 100 ms angezeigt, um sicherzustellen, dass es bereits von der Anwendung ausgefüllt erscheint.

Das folgende Beispiel zeigt einen Popup-Menüpunkt zum Schließen der Registerkarten eines ttk::notebook- oder eines scrollednotebook-Widgets (`$nb`):

```
bind $nb <<MenuItemsRequested>> { populateMenu %W %d }

proc populateMenu {nb data} {
    foreach {menu tabIdx} $data {}
    $menu add command -label "Close Tab" -command [list $nb forget
    $tabIdx]
}
```

Eine BESCHREIBUNG der VIRTUELLEN EREIGNISSE `<<NotebookTabMoved>>`, `<<NotebookTabClosed>>` und `<<CloseTabRequested>>` finden Sie im nächsten Abschnitt.

WEITERE BINDUNGEN

Das Paket "Scrollutil_tile" erstellt die folgenden zusätzlichen Bindungen für die Widgets "ttk::notebook" und "scrollednotebook":

1. **Navigation zwischen den Registerkarten eines "scrollednotebook"- oder "ttk::notebook"-Widgets über das Mausrad oder das Touchpad:**Ein Klick mit dem Mausrad oder eine Zwei-Finger-Geste auf dem Touchpad wählt zyklisch die Registerkarte im Zustand `normal` aus, die der aktuell ausgewählten vorausgeht bzw. ihr folgt. (HINWEIS: Der gleiche Effekt kann auch mit den Tasten `Right`, `Left`, `Control-Tab` und `Control-Shift-Tab` erzielt werden.) Im Fenstersystem `aqua` wird, wenn die Taste `Option` gedrückt ist, während das Mausrad gedreht wird, durch einen Mausrad-Tick zyklisch die 10. Registerkarte des Zustands `normal` ausgewählt, die auf die aktuell ausgewählte folgt bzw. ihr vorausgeht (modulo der Anzahl der Registerkarten). Das Gleiche gilt für die Fenstersysteme `x11` und `win32`, wenn die Tk-Version 8.7a4 oder höher ist und das Mausrad bei gedrückter Taste `Alt` gedreht wird.
2. **Blättern in der Registerkarten-Zeile eines "scrollednotebook"-Widgets mithilfe der Pfeiltasten:**Wenn die Registerkarte am rechten Rand des "scrollednotebook"-Widgets nur teilweise sichtbar ist, passt ein Klick mit der linken Maustaste auf den Rechtspfeil die Ansicht in der Registerkarten-Zeile so an, dass die Registerkarte vollständig sichtbar wird; andernfalls

wird durch den Klick die nächste nicht ausgeblendete Registerkarte eingeblendet. Ähnlich verhält es sich, wenn die Registerkarte am linken Rand des "scrollednotebook"-Widgets nur teilweise sichtbar ist: Durch Drücken der linken Maustaste über dem Pfeil nach links wird die Ansicht in der Registerkarten-Zeile so angepasst, dass die Registerkarte vollständig sichtbar wird; andernfalls wird durch den Klick die vorherige, nicht ausgeblendete Registerkarte eingeblendet. Wenn die Maustaste mindestens 300 Millisekunden lang gedrückt gehalten wird, wiederholen sich die oben beschriebenen Aktionen automatisch alle 100 Millisekunden.

3. **Verschieben der Registerkarten eines "scrollednotebook"- oder "ttk::notebook"-Widgets mit der Maus**: Wenn Maustaste 1 über einer Registerkarte, jedoch außerhalb des `closetab`-Elements (sofern vorhanden) gedrückt und anschließend außerhalb dieser Registerkarte gezogen wird, dann (im Falle eines "scrollednotebook", vorausgesetzt, der Wert der Option `-movabletabs` des Widgets ist "true") nimmt der Mauszeiger die Form eines Links-/Rechtspfeils an, was anzeigt, dass die betreffende Registerkarte an eine andere Position verschoben wird. Dieser Vorgang endet, wenn Maustaste 1 losgelassen wird, und kann durch Drücken der Taste `Escape` abgebrochen werden. In beiden Fällen wird der Mauszeiger auf seinen ursprünglichen Wert zurückgesetzt, der durch die Konfigurationsoption `-cursor` festgelegt ist. Nach dem Loslassen der Maustaste 1 wird die Quellregisterkarte an die Position verschoben, auf die zuvor der Pfeil zeigte (vorausgesetzt, die Aktion wurde nicht über `Escape` abgebrochen). Wenn die Tk-Version 8.5a2 oder höher ist, generiert das Widget nach dem Verschieben der Registerkarte an ihre neue Position ein virtuelles Ereignis `<<NotebookTabMoved>>`, indem es `event generate` aufruft, wobei die Option `-data` auf eine Liste gesetzt ist, die aus dem Pfadnamen des im Quellbereich enthaltenen Widgets und dem numerischen Index der Zielposition der Registerkarte besteht. (Die Unterstützung für diese Option wurde in Tk 8.5a2 eingeführt.) All das gilt auch für ein ttk::notebook mit vertikalem Registerkarten-Layout, mit dem einzigen Unterschied, dass der Cursor für die Dauer des Ziehvorgangs die Form eines Aufwärts-/Abwärtspfeils annimmt.
4. **Schließen eines Reiters eines "scrollednotebook"-Widgets mithilfe des "closetab"-Elements**: Wenn die Maustaste 1 über dem Element `closetab` (sofern vorhanden) eines Reiters gedrückt und später über demselben Element wieder losgelassen wird, vorausgesetzt, dass sowohl der Wert der Option `-state` der Registerkarte als auch der Status ihres Elements `closetab normal` ist, wird die Registerkarte mittels des Unterbefehls `forget` des Widgets geschlossen. Beachten Sie, dass dieser Vorgang durch den als Wert der Option `-forgetcommand` angegebenen Befehl abgebrochen werden kann; war dies nicht der Fall und handelt es sich um die Tk-Version 8.5a2 oder höher, dann erzeugt das Widget nach dem Schließen der Registerkarte ein virtuelles Ereignis vom Typ `<<NotebookTabClosed>>`, indem es `event generate` aufruft, wobei die Option `-data` auf den Pfadnamen des Widgets gesetzt ist, das sich auf der Teilfläche dieser Registerkarte befindet.
5. **Schließen eines Reiters eines ttk::notebook-Widgets mithilfe des "closetab"-Elements**: Wenn die linke Maustaste über dem `closetab`-Element (sofern vorhanden) eines Reiters gedrückt und später über demselben Element wieder losgelassen wird, vorausgesetzt, dass sowohl der Wert der Option `-state` der Registerkarte als auch der Status ihres Elements `closetab` den Wert `normal` haben, wird eine von der Tk-Version abhängige Aktion ausgeführt:
   - Wenn die Tk-Version 8.5a2 oder höher ist, wird die Registerkarte (noch) nicht geschlossen. Stattdessen generiert das Widget ein virtuelles Ereignis vom Typ

<<CloseTabRequested>>, indem es `event generate` aufruft, wobei die Option `-data` auf den numerischen Index der betreffenden Registerkarte gesetzt ist. Es liegt in der Verantwortung der Anwendung, eine Bindung für dieses virtuelle Ereignis zu erstellen und die Registerkarte aus dem Bindungsskript heraus zu schließen, indem gegebenenfalls der Unterbefehl `forget` des Widgets aufgerufen wird, wie im folgenden Beispiel gezeigt:

```
bind $nb <<CloseTabRequested>> { closeTab %W %d }

proc closeTab {nb tabIdx} {
    set btn [tk_messageBox -title "Close Tab?" -icon question \
             -message "Do you really want to close the tab?" -type
    yesno
    if {$btn eq "yes"} {
        $nb forget $tabIdx

        # Optionally:
        set widget [lindex [$nb tabs] $tabIdx]
        event generate $nb <<NotebookTabClosed>> -data $widget
    }
}
```

- Wenn die Tk-Version älter als 8.5a2 ist, wird die Registerkarte sofort über den Unterbefehl `forget` des Widgets geschlossen. (Zur Erinnerung: Die Option `-data` für VIRTUELLE EREIGNISSE wurde in Tk 8.5a2 eingeführt.)

Wenn das Statusflag `disabled` des Widgets gesetzt ist, findet keine der oben genannten Aktionen statt: Die Registerkarten sind vollständig deaktiviert.

SCHLÜSSELWÖRTER

scrollednotebook, widget, notebook, scrolled

Das Stilelement `closetab`

Das Paket "Scrollutil_tile" erstellt automatisch ein neues Stilelement mit dem Namen `closetab`, dessen Form einem "x" ähnelt. Die Größe des Elements hängt vom Skalierungsfaktor von Tk ab, während seine Vordergrundfarbe themenspezifisch ist und bei jedem Wechsel des Themas aktualisiert wird.

Sie können dieses Element zu den Registerkarten eines Stile vom Typ ttk::notebook (wie `TNotebook` oder `My.TNotebook`) hinzufügen, mithilfe des Befehls `scrollutil::addclosetab`, wodurch die Registerkarten jedes ttk::notebook- oder scrollednotebook-Widgets dieses Stils vom Benutzer geschlossen werden können.

Für den umgekehrten Vorgang stellt das Paket "Scrollutil_tile" den Befehl `scrollutil::removeclosetab` zur Verfügung.

Der Status (`normal` oder `disabled`) der Elemente `closetab` einer Registerkarte vom Typ ttk::notebook oder scrollednotebook kann mithilfe des Befehls `scrollutil::closetabstate` festgelegt und abgefragt werden.

Der Befehl `scrollutil::addclosetab`

NAME

`scrollutil::addclosetab` – Fügen Sie das Element `closetab` zu den Registerkarten des Stile ttk::notebook hinzu

ÜBERSICHT

```
package require scrollutil_tile
scrollutil::addclosetab notebookStyle
```

HINWEIS: Der Befehl `scrollutil::addclosetab` wird vom Paket "Scrollutil_tile" bereitgestellt, nicht jedoch von Scrollutil.

BESCHREIBUNG

Passen Sie den Stil `notebookStyle.Tab` an einen bestimmten Stil "ttk::notebook" an (wie z. B. `TNotebook` oder `My.TNotebook`), indem das Element `closetab` hinzugefügt wird. Dadurch können die Registerkarten aller ttk::notebook- oder scrollednotebook-Widgets, bei denen die Option `-style` auf `notebookStyle` gesetzt wurde, vom Benutzer geschlossen werden. Wenn das Element `closetab` bereits im Stil `notebookStyle.Tab` vorhanden ist, gibt der Befehl zurück, ohne eine Aktion auszuführen.

SCHLÜSSELWÖRTER

ttk::notebook style, closetab

Der Befehl `scrollutil::removeclosetab`

NAME

`scrollutil::removeclosetab` – Entfernen Sie das Element `closetab` aus den Registerkarten eines Stils vom Typ ttk::notebook

ÜBERSICHT

```
package require scrollutil_tile
scrollutil::removeclosetab notebookStyle
```

HINWEIS: Der Befehl `scrollutil::removeclosetab` wird vom Paket "Scrollutil_tile" bereitgestellt, nicht jedoch von Scrollutil.

BESCHREIBUNG

Ändert den Stil `notebookStyle.Tab`, der zu einem gegebenen ttk::notebook-Stil gehört (etwa `TNotebook` oder `My.TNotebook`), indem das Element `closetab` daraus entfernt wird. War das Element `closetab` im Stil `notebookStyle.Tab` nicht vorhanden, kehrt der Befehl zurück, ohne etwas zu tun.

SCHLÜSSELWÖRTER

ttk::notebook style, closetab

Der Befehl `scrollutil::closetabstate`

NAME

`scrollutil::closetabstate` – Den Status des Elements `closetab` eines ttk::notebook oder einer Registerkarte eines ScrolledNotebook festlegen oder abfragen

ÜBERSICHT

```
package require scrollutil_tile
scrollutil::closetabstate notebook tabId ?normal|disabled?
```

Es gibt einen HINWEIS, dass der Befehl `scrollutil::closetabstate` vom Paket "Scrollutil_tile" bereitgestellt wird, nicht jedoch von Scrollutil.

BESCHREIBUNG

Setzt oder fragt den Status des Elements `closetab` der Registerkarte ab, die durch `tabId` des Widgets "ttk::notebook" oder "scrollednotebook" (`notebook`) angegeben wird. Das optionale Argument legt den Status als `normal` oder `disabled` fest. Ohne dieses Argument gibt der Befehl den aktuellen Status des Elements `closetab` der Registerkarte zurück.

Sofern das Statusflag `disabled` des Widgets nicht gesetzt ist und der Wert der Option `-state` der Registerkarte `normal` lautet, bestehen folgende Unterschiede zwischen den beiden `closetab`-Zuständen:

- Im Standardzustand `normal` wird das Elemente `closetab` bei Mauszeigerüberführung in weißer Farbe auf hellrotem Hintergrund angezeigt und bei Betätigung mit Maustaste 1 in weißer Farbe auf dunkelrotem Hintergrund. Wird die Maustaste 1 über demselben Element losgelassen, wird die unter den Punkten 4 und 5 des obigen Abschnitts WEITERE BINDUNGEN beschriebene Aktion ausgelöst.
- Im Zustand `disabled` wird das Elemente `closetab` sowohl beim Bewegen des Mauszeigers darüber als auch beim Klicken in derselben themenspezifischen Vordergrundfarbe wie deaktivierter Text angezeigt, und das Klicken und anschließende Loslassen der Maustaste 1 darüber hat keine Auswirkung.

Beachten Sie, dass der Unterbefehl "`closetabstate` scrollednotebook" eine praktische Abkürzung für diesen Befehl ist.

SCHLÜSSELWÖRTER

tab, closetab, state

Stichwörter: scrollednotebook

scrollsync

Das `scrollutil::scrollsync` und `scrollutil::getscrollsync` BEFEHLE

Für Scrollutil Version 2.11

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

- Der Befehl scrollutil::scrollsync – Kurzreferenz
- Der Befehl scrollutil::scrollsync – Ausführliche Referenz
- Der Befehl scrollutil::getscrollsync

Der Befehl `scrollutil::scrollsync` – Kurzreferenz

NAME

`scrollutil::scrollsync` – Erstellen und Bearbeiten von ScrollSync-Widgets

ÜBERSICHT

```
scrollutil::scrollsync pathName ?options?
```

STANDARDOPTIONEN

```
-background   -highlightbackground  -relief
-borderwidth  -highlightcolor       -xscrollcommand
-cursor       -highlightthickness   -yscrollcommand
```

WIDGET-SPEZIFISCHE OPTIONEN

`-takefocus 0|1|""|command`

Widget COMMAND

`pathName attrib ?name ?value name value ...??`

`pathName cget option`

`pathName configure ?option ?value option value ...??`

`pathName hasattrib name`

`pathName setwidgets widgetList`

`pathName unsetattrib name`

`pathName widgets`

`pathName xview ?args?`

`pathName xview`

`pathName xview moveto fraction`

`pathName xview scroll number units|pages`

`pathName yview ?args?`

`pathName yview`

`pathName yview moveto fraction`

`pathName yview scroll number units|pages`

SCHLÜSSELWÖRTER

Scrollsync, Widget, Bildlauf

Der Befehl `scrollutil::scrollsync` – Ausführliche Referenz

NAME

`scrollutil::scrollsync` – Erstellen und Bearbeiten von ScrollSync-Widgets

ÜBERSICHT

```
scrollutil::scrollsync pathName ?options?
```

BESCHREIBUNG

Der Befehl `scrollutil::scrollsync` erstellt ein neues Fenster mit dem Namen `pathName` und der Klasse `Scrollsync` und wandelt es in ein **scrollsync**-Widget um. Zusätzliche Optionen, die im Folgenden beschrieben werden, können über die Befehlszeile oder in der Optionsdatenbank festgelegt werden, um Aspekte des **scrollsync**-Widgets wie beispielsweise dessen Rahmenbreite und Relief zu konfigurieren. Der Befehl `scrollutil::scrollsync` gibt sein Argument `pathName` zurück. Zum Zeitpunkt der Ausführung dieses Befehls darf kein Fenster mit dem Namen `pathName` vorhanden sein, das übergeordnete Fenster von `pathName` muss jedoch vorhanden sein.

Ein "Scrollsync" ist ein Megawidget, das dafür konzipiert ist, mehrere Widgets gleichzeitig zu scrollen. Es lässt sich sowohl horizontal als auch vertikal scrollen, was bedeutet, dass es die Konfigurationsoptionen `-xscrollcommand` und `-yscrollcommand` unterstützt und der zugehörige Tcl-Befehl über die Unterbefehle `xview` und `yview` verfügt. Immer wenn sich die horizontale/vertikale Position der Ansicht im Fenster eines der horizontal/vertikal scrollbaren Widgets ändert, die in der an den Unterbefehl `setwidgets` des zugehörigen Tcl-Befehls übergebenen Liste enthalten sind, wird die Ansicht in den Fenstern aller anderen horizontal/vertikal scrollbaren Elemente dieser Liste automatisch entsprechend angepasst, wodurch sichergestellt wird, dass die Position der Ansicht in diesen Fenstern synchron bleibt.

Ein ScrollSync-Widget wird in der Regel innerhalb eines scrollarea erstellt, wie im folgenden Beispiel gezeigt:

```
set sa [scrollutil::scrollarea ...]
set ss [scrollutil::scrollsync $sa.ss ...]
$sa setwidget $ss

set lb1 [listbox $ss.lb1 ...]
set lb2 [listbox $ss.lb2 ...]
$ss setwidgets [list $lb1 $lb2]

grid $lb1 $lb2 -sticky news -padx {0 2}
grid rowconfigure    $ss 0   -weight 1
grid columnconfigure $ss all -weight 1

pack $sa -expand yes -fill both
```

STANDARDOPTIONEN

```
-background   -highlightbackground  -relief
-borderwidth  -highlightcolor       -xscrollcommand
-cursor       -highlightthickness   -yscrollcommand
```

Weitere Informationen zu den Standardoptionen für Tk-Widgets finden Sie im Handbucheintrag zu den **options**. Die Optionen `-background`, `-highlightbackground`, `-highlightcolor` und `-highlightthickness` werden nur vom Scrollutil-Paket unterstützt, nicht jedoch von Scrollutil_tile. Sie haben dieselben Standardwerte wie die gleichnamigen Optionen für Tk-Frame-Widgets. Die Standardwerte der übrigen Standardoptionen lauten:

```
-borderwidth 0 -cursor "" -relief flat -xscrollcommand ""
    -yscrollcommand ""
```

WIDGET-SPEZIFISCHE OPTIONEN

Befehlszeilenname:	-takefocus
Name der Datenbank:	takeFocus
Datenbankklasse:	TakeFocus

Diese Option legt fest, ob das Scrollsync-Widget während der Tastaturnavigation den Fokus annimmt. Sie ist nahezu identisch mit der gleichnamigen Standardoption (Einzelheiten finden Sie im Handbucheintrag **options**). Der einzige Unterschied besteht darin, dass nicht das "scrollsync"-Widget selbst, sondern das erste Element der Widget-Liste, die an den Unterbefehl `setwidgets` des zugehörigen Tcl-Befehls übergeben wird, den Fokus während der Tastaturnavigation mit den Standardtasten (`Tab` und `Shift-Tab`). Die Standardeinstellung ist `0`, da ein "scrollsync"-Widget im Wesentlichen ein Container ist, der die oben genannten Widgets enthält.

WIDGET-BEFEHL

Der Befehl `scrollutil::scrollsync` erstellt einen neuen Tcl-Befehl mit dem Namen `pathName`. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget auszuführen. Er hat die folgende allgemeine Form:

```
pathName option ?arg arg ...?
```

`option` und die `arg`-Elemente legen das genaue Verhalten des Befehls fest. Für Scrollsync-Widgets sind die folgenden Befehle möglich:

`pathName attrib ?name ?value name value ...??`

Fragt die Attribute des Widgets ab oder ändert sie. Wenn kein `name` angegeben ist, gibt der Befehl eine Liste von Paaren zurück, von denen jedes den Namen und den Wert eines Attributs für `pathName` enthält. Wenn `name` ohne `value` festgelegt wird, gibt der Befehl den Wert des genannten Attributs zurück oder eine leere Zeichenkette, falls kein entsprechender Wert vorhanden ist (mit dem Unterbefehl `hasattrib` können Sie diesen Fall von dem unterscheiden, bei dem der Wert eines *vorhandenen* Attributs eine leere Zeichenkette ist). Wenn ein oder mehrere Paare aus `name` und `value` angegeben werden, legt der Befehl das/die angegebene(n) Widget-Attribut(e) auf den/die angegebenen Wert(e)

fest; in diesem Fall ist der Rückgabewert eine leere Zeichenkette. Jedes `name` kann eine beliebige Zeichenkette sein.

`pathName cget option`
Gibt den aktuellen Wert der durch `option` angegebenen Konfigurationsoption zurück, die einen der vom Befehl `scrollutil::scrollsync` akzeptierten Werte annehmen kann.

`pathName configure ?option ?value option value ...??`
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Wenn kein `option` festgelegt ist, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für `pathName` beschreibt (Informationen zum Format dieser Liste finden Sie unter `Tk_ConfigureInfo`). Wenn `option` ohne `value` festgelegt wird, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste entspricht der entsprechenden Unterliste des gegebenen Werts, wenn kein `option` festgelegt wurde). Wenn ein oder mehrere Paare aus `option` und `value` festgelegt werden, ändert der Befehl die festgelegten Widget-Optionen so, dass sie die festgelegten Werte annehmen; in diesem Fall ist der Rückgabewert eine leere Zeichenkette. `option` kann jeden der vom Befehl `scrollutil::scrollsync` akzeptierten Werte annehmen.

`pathName hasattrib name`
Gibt `1` zurück, wenn das Attribut `name` vorhanden ist, andernfalls `0`.

`pathName setwidgets widgetList`
Legt die Widgets fest, in deren Fenstern die Position der Ansicht synchronisiert werden soll. Das Argument `widgetList` muss eine gültige Liste sein, die aus Pfadnamen vorhandener Widgets besteht. Immer wenn sich die horizontale/vertikale Position der Ansicht im Fenster eines der in dieser Liste enthaltenen, horizontal/vertikal scrollbaren Widgets ändert, wird die Ansicht in den Fenstern aller anderen horizontal/vertikal scrollbaren Elemente der Liste automatisch entsprechend angepasst, wodurch sichergestellt wird, dass die Position der Ansicht in diesen Fenstern synchron bleibt. Der Wert wird als Argument beim letzten erfolgreichen Aufruf dieses Unterbefehls übergeben, oder eine leere Liste, falls zuvor kein erfolgreicher Aufruf von `setwidgets` stattgefunden hat.

HINWEIS: Wenn eines der Widgets, dessen Pfadname in `widgetList` enthalten ist, zerstört wird, wird dieses Widget automatisch aus der internen Liste der Widgets entfernt, in deren Fenstern die Position der Ansicht synchronisiert wird.

`pathName unsetattrib name`
Hebt das Attribut `name` auf. Gibt eine leere Zeichenfolge zurück.

`pathName widgets`
Gibt das Argument zurück, das beim letzten erfolgreichen Aufruf des Unterbefehls `setwidgets` übergeben wurde, oder eine leere Liste, falls dieser Unterbefehl noch nicht erfolgreich aufgerufen wurde.

`pathName xview ?args?`

```
pathName xview

pathName xview moveto fraction

pathName xview scroll number units|pages
```

Dieser Befehl übergibt seine Argumente an den Befehl `xview` des aktuellen **Master-Widgets für die x-Achse** und gibt das Ergebnis dieses Befehlsaufrufs zurück. Das Master-Widget für die x-Achse ist das Element der an den Unterbefehl `setwidgets` übergebenen Widget-Liste, das unter den horizontal scrollbaren Widgets in dieser Liste die geringste relative Ansichtsbreite aufweist, d. h., die geringste Differenz zwischen dem letzten und dem ersten Element der aus dem Befehl `xview` zurückgegebenen Liste mit zwei Elementen. Dieses Master-Widget kann sich während der Programmausführung ändern (bei Text-Widgets kann es sich sogar in Abhängigkeit von der aktuellen vertikalen Ansichtsposition ändern).

`pathName yview ?args?`

```
pathName yview

pathName yview moveto fraction

pathName yview scroll number units|pages
```

Dieser Befehl übergibt seine Argumente an den Befehl `yview` des aktuellen **Master-Widgets für die y-Achse** und gibt das Ergebnis dieses Befehlsaufrufs zurück. Das Master-Widget für die y-Achse ist das Element der an den Unterbefehl `setwidgets` übergebenen Widget-Liste, das unter den vertikal scrollbaren Widgets in dieser Liste die geringste relative Ansichtshöhe aufweist, d. h. die geringste Differenz zwischen dem letzten und dem ersten Element der aus zwei Elementen bestehenden Liste, die von seinem Befehl `yview` gegeben wird. Dieses Master-Widget kann sich während der Programmausführung ändern.

VORGABEBINDUNGEN

Wenn ein neues "scrollsync"-Widget erstellt wird, verfügt es über keine VORGABEBINDUNGEN für Ereignisse: "scrollsync"-Widgets sind nicht dafür vorgesehen, interaktiv zu sein.

SCHLÜSSELWÖRTER

scrollsync, widget, scrolling

Der Befehl `scrollutil::getscrollsync`

NAME

`scrollutil::getscrollsync` – Abfrage des ScrollSync, der ein bestimmtes Widget enthält

ÜBERSICHT

```
scrollutil::getscrollsync widget
```

BESCHREIBUNG

Gibt den Pfadnamen des Scrollsync zurück, in das das durch das Argument `widget` angegebene Widget über den Unterbefehl `setwidgets` des Scrollsync eingebettet ist, oder eine leere Zeichenkette, falls kein solches Scrollsync-Widget vorhanden ist.

SCHLÜSSELWÖRTER

scrollsync, widget

Stichwörter: scrollsync

scrollutil

Scrollutil-Programmierhandbuch

Für Scrollutil Version 2.11

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

Übersicht
- Was ist Scrollutil?
- So erhalten Sie es:
- Wie wird es installiert?
- Anleitung
- Mehr zu scrollutil::scalingpct
- Die Prozedur scrollutil::getScalingPct und der Skalierungsfaktor von Tk
- Mehr zu scrollutil::svgfmt

Beispiele
- Das Hilfsskript styleUtil.tcl
- Ein scrollbares Tablelist-Widget
- Ein scrollbares Text-Widget
- Ein Widget mit scrollbarer Canvas
- Synchronisierung von zwei listbox-Widgets
- Synchronisierung von drei Tablelist-Widgets
- Ein Skript, das das Widget scrollutil::scrollableframe verwendet
- Ein Skript, das ein BWidget-ScrollableFrame-Widget verwendet
- Ein Skript, das ein iwidgets::scrolledframe-Widget verwendet
- Ein Skript, das zwei scrollutil::scrollableframe-Widgets verwendet:
- Ein Skript, das zwei BWidget-ScrollableFrame-Widgets verwendet
- Ein Skript, das zwei iwidgets::scrolledframe-Widgets und ein-Widget verwendet]
- A scrollutil::scrollednotebook Demo
- A scrollutil::plainnotebook Demo
- A scrollutil::pagesman Demo

Überblick

Was ist Scrollutil?

Scrollutil ist ein Paket für Tcl/Tk ab Version 8.4, das vollständig in Tcl/Tk geschrieben ist. Es enthält:

- die Implementierung der Megawidgets **scrollarea**, **scrollsync**, **scrollableframe**, **scrollednotebook**, **plainnotebook** und **pagesman**, einschließlich eines allgemeinen

Hilfsmoduls für Megawidgets. Die Widgets **scrollednotebook** und **plainnotebook** erfordern Tile 0.8 oder höher;
- Der Befehl `scrollutil::addMouseWheelSupport`, der Mausrad-Ereignis-Bindungen für ein bestimmtes Tag erstellt;
- Befehle für die *benutzerfreundliche* Verarbeitung von Mausrad-Ereignissen in **scrollbaren Widget-Containern** wie scrollutil::scrollableframe, BWidget ScrollableFrame und iwidgets::scrolledframe. Diese Befehle erfordern Tcl/Tk-Versionen 8.4 oder höher unter X11 und Mac OS X/11+ sowie Tk 8.6b2 oder höher unter Windows;
- Befehle für die *benutzerfreundliche* Ereignisbehandlung bei Betätigung der Maustaste 2 in scrollutil::scrollableframe-Widgets;
- Beispielskripte, die die Verwendung des Scrollutil-Pakets in Verbindung mit verschiedenen scrollbaren Widgets und den oben genannten Containern für scrollbare Widgets veranschaulichen;
- Demo-Skripte, die die Verwendung der Widgets "scrollednotebook", "plainnotebook" und "pagesman" veranschaulichen;
- dieses Tutorial;
- Referenzseiten im HTML-Format.

Das Megawidget scrollutil::scrollarea vereinfacht die Erstellung beliebiger scrollbarer Widgets erheblich. Es besteht aus einem scrollbaren Widget und zwei mit diesem Widget verbundenen Bildlaufleisten. Der Anzeigemodus jeder Bildlaufleiste kann `static`, `dynamic` oder `none` sein. Diese Implementierung eines scrollbaren Fensters unterstützt auch Widgets, die nur in eine Richtung scrollbar sind (z. B. "entry" und ttk::entry), und berücksichtigt die Kopfzeilenkomponente sowie die Titelspalten von tablelist-Widgets (dies ist frei konfigurierbar).

Das Widget scrollutil::scrollarea ähnelt dem BWidget ScrolledWindow und dessen auf snit basierendem Äquivalent widget::scrolledwindow, das von Jeffrey Hobbs beigesteuert wurde und in tklib enthalten ist. Das snit-basierte Paket scrodget von Aldo Buratti und sein TclOO-basiertes Äquivalent scrolledwidget, beigesteuert von Johann Oberdorfer, sind weitere Implementierungen von Bildlauf-Fenstern. Eine *vollständige* Tablelist-Unterstützung bietet jedoch nur das Scrollarea-Widget, das frei von externen Abhängigkeiten wie BWidget, snit oder (für Tcl 8.5) TclOO ist. Es ist zudem frei von dem Shimmering-Problem im Zusammenhang mit Text-Widgets, das die oben genannten Implementierungen von Bildlauf-Fenstern entweder gemeinsam mit dem autoscroll-Paket (enthalten in tklib) aufweisen oder auf suboptimale Weise umgehen.

Das Megawidget scrollutil::scrollsync ist dafür ausgelegt, mehrere Widgets gleichzeitig zu scrollen. Sobald sich die horizontale/vertikale Position der Ansicht im Fenster eines seiner Widgets ändert, wird die Ansicht in den Fenstern aller anderen Widgets automatisch entsprechend angepasst, wodurch sichergestellt wird, dass die Position der Ansicht in diesen Fenstern synchron bleibt. Dieses Megawidget ist horizontal und vertikal scrollbar und kann daher über den Unterbefehl `setwidget` des scrollutil::scrollarea-Widgets in dieses eingebettet werden.

Das Megawidget scrollutil::scrollableframe ist ein scrollbarer Widget-Container. Es enthält einen Inhaltsrahmen, dessen Abmessungen in der Regel größer sind als die des Widgets selbst. Beliebige Bereiche dieses Rahmens können durch Scrollen in den sichtbaren Bereich gebracht werden, und das Widget bietet zudem einen Befehl, mit dem einzelne, im Inhaltsrahmen enthaltene Widgets im Fenster des scrollbaren Rahmens sichtbar gemacht werden können.

Das Widget scrollutil::scrollableframe ähnelt den Widgets BWidget ScrollableFrame und

iwidgets::scrolledframe. Im Gegensatz zu diesen Widgets, die ein canvas zum Scrollen des Inhaltsrahmens verwenden, passt es die Ansicht mithilfe des Geometriemanagers `place` an, genau wie der Befehl `scrolledframe::scrolledframe` aus dem Scrolledframe-Paket von Maurice Bredelet (ulis) und dessen von Keith Nash beigesteuerte optimierte und erweiterte Version. Einzelheiten zu diesen Befehlen finden Sie auf der Wiki-Seite

https://wiki.tcl-lang.org/page/A+scrolled+frame

Der Ansatz ohne Canvas ist schlanker und lässt sich besser in Anwendungen integrieren, die Tile-Widgets verwenden.

Das Megawidget scrollutil::scrollednotebook enthält ein ttk::notebook in einem scrollbaren Rahmen und unterstützt eine beliebige Anzahl von nicht komprimierten Registerkarten. Derzeit nicht sichtbare Registerkarten können durch Navigation mit dem Mausrad, dem Touchpad oder der Tastatur sowie durch Scrollen mithilfe der beiden Pfeiltasten, die je nach Stile des Notizbuchs auf Wunsch in den oberen linken und rechten bzw. unteren linken und rechten Ecken platziert werden, in den sichtbaren Bereich gebracht werden. Das Widget bietet außerdem einen Befehl, um einzelne Registerkarten im "scrollednotebook"-Fenster sichtbar zu machen. Im Gegensatz zum ttk::notebook-Widget, dessen `-width`-Option häufig durch die Gesamtbreite der Registerkarten überschrieben wird, beachtet das "scrollednotebook"-Widget den Wert seiner Option `-width`, unabhängig vom Platzbedarf der Registerkarten.

Das Megawidget scrollutil::plainnotebook erweitert ein ttk::notebook mit einer beliebigen Anzahl von Seiten und unsichtbaren Registerkarten um ein ttk::frame links oder rechts davon, das unter anderem einen scrollableframe, dessen Inhaltsrahmen der übergeordnete Rahmen einer Reihe von Widgets ist, die die Rolle vertikal angeordneter Notizbuchregisterkarten übernehmen. Derzeit nicht sichtbare "Registerkarten" können durch Navigation mit dem Mausrad, dem Touchpad oder der Tastatur sowie durch Scrollen mit der vertikalen Bildlaufleiste des scrollarea, der den scrollableframe enthält, in den sichtbaren Bereich gebracht werden. Das Widget bietet zudem einen Befehl, um einzelne "Registerkarten" im PlainNotebook-Fenster sichtbar zu machen. Im Gegensatz zu einem ttk::notebook-Widget mit vertikal ausgerichteten Registerkarten, das in den meisten Themen nicht optimal aussieht und dessen Option `-height` häufig durch die Gesamthöhe der Registerkarten überschrieben wird, beachtet das "plainnotebook"-Widget den Wert seiner Option `-height`, unabhängig vom Platzbedarf der "Registerkarten".

Das Megawidget scrollutil::pagesman bietet die Grundfunktionalität eines Seitenmanagers, d. h., es verwaltet eine Liste von Fenstern, die als **Seiten** bezeichnet werden, von denen jeweils nur eine sichtbar ist. Durch die Verwendung von "plainnotebook"-Widgets als Seiten ist es recht einfach, Anwendungen zu schreiben, in denen der Benutzer mit einem einzigen Mausklick von einem "plainnotebook" zu einem anderen wechseln und auf dieselbe Weise wieder zum ursprünglichen zurückkehren kann. Alles, was für diese Navigation benötigt wird, wird durch entsprechende Optionen und Unterbefehle des "plainnotebook"-Widgets bereitgestellt.

Aus Sicht der **Befehle im Zusammenhang mit dem Mausrad und der Ereignisbehandlung von <TouchpadScroll>**, die vom Scrollutil-Paket bereitgestellt werden, bedeutet die Scrollbarkeit eines Widgets oder eines Widget-Container-Fensters, dass der zugehörige Tcl-Befehl die Unterbefehle `xview scroll number units` und `yview scroll number units` unterstützt. Der Grund dafür, dass unter Windows mindestens Tk-Version 8.6b2 für die Befehle im Zusammenhang mit scrollbaren Widget-Containern erforderlich ist, liegt darin, dass in früheren Tk-Versionen auf dieser Plattform die Mausrad-Ereignisse an das Widget mit

Fokus gesendet wurden und nicht an das Widget, das sich unter dem Mauszeiger befand.

Um das benutzerfreundliche Mausrad und die Ereignisbehandlung von `<TouchpadScroll>` in scrollbaren Widget-Containern mithilfe des Scrollutil-Pakets zu nutzen, führen Sie die folgenden Schritte aus:

- Erstellen Sie Mausrad- und `<TouchpadScroll>`-Ereigniszuordnungen für das Bindungs-Tag `"all"` oder für die Toplevel-Fenster (einschließlich `"."`), die scrollbare Widget-Container enthalten, indem Sie den Befehl `scrollutil::createWheelEventBindings` aufrufen. Registrieren Sie außerdem Ihre scrollbaren Widget-Container mithilfe des Befehls `scrollutil::enableScrollingByWheel` für das Scrollen über diese Bindungen. Beachten Sie, dass dieser Befehl für das Widget scrollutil::scrollableframe automatisch bei der Erstellung aufgerufen wird. Die oben genannten Bindungen verarbeiten Mausrad- und `<TouchpadScroll>`-Ereignisse, indem sie den (innersten) registrierten scrollbaren Widget-Container scrollen, der ein Vorfahr des Widgets unter dem Mauszeiger ist und in dessen Toplevel-Fenster enthalten ist.
- Rufen Sie den Befehl `scrollutil::adaptWheelEventHandling` für jene Widgets auf, die in registrierten scrollbaren Widget-Containern enthalten sind und über Ereignis-(Klasse-)Bindungen für das Mausrad oder `<TouchpadScroll>` verfügen. Dieser Schritt ändert die Ereignisbehandlung für das Mausrad und das Ereignis `<TouchpadScroll>` wie folgt: Wenn der Fokus auf dem Widget unter dem Mauszeiger liegt, werden die oben genannten Ereignisse ausschließlich von den (Klassenbindungen des) Widgets verarbeitet, andernfalls von den mit dem Befehl `scrollutil::createWheelEventBindings` erstellten Bindungen. Beachten Sie, dass dieser Befehl als optionales Argument den Schalter `-ignorefocus` akzeptiert, der festlegt, dass die oben genannten Ereignisse (ausschließlich) vom Widget unter dem Mauszeiger verarbeitet werden sollen, unabhängig davon, ob dieses Widget den Fokus hat oder nicht.
- Bei einigen Widgets kann es wünschenswert sein, die Prüfung des Fokus innerhalb dieser modifizierten Ereignisbehandlung weniger restriktiv zu gestalten. Wenn es sich bei dem Widget unter dem Mauszeiger beispielsweise um eine Eingabekomponente eines mentry vom Typ `"Date"`, `"Time"`, `"DateTime"`, `"IPAddr"`, oder `"IPv6Addr"` ist und der Fokus auf einem seiner Geschwisterelemente liegt, sollten die an dieses Eintrags-Widget gesendeten Mausrad- und `<TouchpadScroll>`-Ereignisse vom Eintrags-Widget selbst verarbeitet werden, anstatt den Widget-Container zu scrollen, der ein übergeordnetes Element des mentry ist. Der Befehl `scrollutil::setFocusCheckWindow` deckt genau solche Fälle ab.

Die Funktionsweise des Mausrads und der Ereignisbehandlung von `<TouchpadScroll>` mithilfe des Scrollutil-Pakets wurde ebenfalls in Verbindung mit dem Befehl `scrolledframe::scrolledframe` aus dem Scrolledframe-Paket von Maurice Bredelet (ulis) und dessen optimierter und erweiterter Version, die von Keith Nash beigesteuert wurde, sowie mit dem von Paul Walton implementierten Befehl `sframe`. Einzelheiten zu diesen Befehlen (die weitere Implementierungen von scrollbaren Widget-Containern bereitstellen) finden Sie auf der oben genannten Wiki-Seite.

Die Befehle zur Ereignisbehandlung für Mausklick 2 sind auf das Widget scrollutil::scrollableframe beschränkt, das über die Befehle `scan mark` und `scan dragto` Unterstützung für das Scannen bietet. Die Bindungen für die rechte Maustaste des von Scrollutil automatisch erstellten Bindungs-Tags `"all"` verarbeiten die Ereignisse `<Button-2>`, `<B2-Motion>` und `<ButtonRelease-2>`, indem sie den Inhalt des (innersten)

"scrollableframe"-Widgets abtasten, das ein Vorfahr des angeklickten Widgets ist und in dessen Toplevel-Fenster enthalten ist. Durch Aufrufen des Befehls `scrollutil::adaptBtn2EventHandling` für die Widgets, die in einem "scrollableframe" enthalten sind und über Mausknopf-2-(Klassen-)Bindungen verfügen, wird die Behandlung dieser Ereignisse wie folgt geändert: Wenn der Fokus auf dem angeklickten Widget liegt, werden die Ereignisse ausschließlich von den (Klassenbindungen des) Widgets verarbeitet, andernfalls von den für das `"all"`-Tag erstellten Bindungen. Dies ist insbesondere auf Android nützlich, wo es eine benutzerfreundliche Scan-Unterstützung für "scrollableframe"-Widgets über Ein-Finger-Gesten bietet.

Wie bekomme ich es?

Scrollutil steht auf der Seite zum kostenlosen Download bereit

https://www.nemethi.de

Die Distributionsdatei lautet `scrollutil2.11.tar.gz` für UNIX und `scrollutil2_11.zip` für Windows. Diese Dateien enthalten dieselben Informationen, mit Ausnahme des zusätzlichen Wagenrücklaufzeichens, das in den Textdateien für Windows am Ende jeder Zeile vor dem Zeilenvorschub steht.

Scrollutil ist ebenfalls in tklib enthalten, dessen Adresse lautet

https://core.tcl-lang.org/tklib

Wie wird es installiert?

Installieren Sie das Paket als Unterverzeichnis eines der Verzeichnisse, die durch die Variable `auto_path` angegeben sind. Sie können es beispielsweise als Unterverzeichnis des Verzeichnisses `lib` innerhalb Ihrer Tcl/Tk-Installation installieren.

Um Scrollutil *unter UNIX* zu installieren, wechseln Sie mit `cd` in das gewünschte Verzeichnis und entpacken Sie die Distributionsdatei `scrollutil2.11.tar.gz`:

```
gunzip -c scrollutil2.11.tar.gz | tar -xf -
```

Auf den meisten UNIX-Systemen kann dies durch Folgendes ersetzt werden:

```
tar -zxf scrollutil2.11.tar.gz
```

Beide Befehle erstellen ein Verzeichnis namens `scrollutil2.11` mit den Unterverzeichnissen `demos`, `doc` und `scripts`.

Unter Windows verwenden Sie WinZip oder ein anderes Programm, mit dem Sie die Distributionsdatei `scrollutil2_11.zip` in das Verzeichnis `scrollutil2.11` entpacken können, einschließlich der Unterverzeichnisse `demos`, `doc` und `scripts`.

Beachten Sie, dass in tklib das Verzeichnis "Scrollutil `demos`" durch das Unterverzeichnis `scrollutil` des Verzeichnisses `examples` ersetzt wird. Bitte berücksichtigen Sie dies beim Lesen der folgenden Beispiele.

Wie wird es angewendet?

Die Scrollutil-Distribution enthält zwei Pakete: **Scrollutil** und **Scrollutil_tile**. Der Hauptunterschied zwischen den beiden besteht darin, dass **Scrollutil_tile** die kachelbasierte, themenspezifische Darstellung der Widgets **scrollarea**, **scrollsync** und **scrollableframe** ermöglicht und die themenbezogenen Widgets **scrollednotebook** und **plainnotebook**

bereitstellt; dieses Paket erfordert **tile** 0.8 oder höher. Es ist nicht möglich, beide Pakete in ein und derselben Anwendung zu verwenden, da beide im selben Namespace `scrollutil` implementiert sind und identische Befehle bereitstellen (mit Ausnahme der Befehle `scrollutil::scrollednotebook`, `scrollutil::plainnotebook`, `scrollutil::addclosetab`, `scrollutil::removeclosetab` und `scrollutil::closetabstate`, die ausschließlich vom Paket "Scrollutil_tile" bereitgestellt werden).

Um auf die Befehle und Variablen des Scrollutil-Pakets zugreifen zu können, müssen Ihre Skripte eine der folgenden Zeilen enthalten:

```
package require scrollutil ?version?
package require Scrollutil ?version?
```

Sie können eine der beiden oben genannten Anweisungen verwenden, da die Datei `scrollutil.tcl` beide Zeilen enthält.

```
package provide scrollutil ...
package provide Scrollutil ...
```

Um auf die Befehle und Variablen des Pakets "Scrollutil_tile" zugreifen zu können, müssen Ihre Skripte ebenfalls eine der folgenden Zeilen enthalten:

```
package require scrollutil_tile ?version?
package require Scrollutil_tile ?version?
```

Auch hier können Sie eine der beiden oben genannten Anweisungen verwenden, da die Datei `scrollutil_tile.tcl` beide Zeilen enthält.

```
package provide scrollutil_tile ...
package provide Scrollutil_tile ...
```

Es steht Ihnen frei, jeweils eine dieser beiden Zeilen aus `scrollutil.tcl` bzw. `scrollutil_tile.tcl` zu entfernen, wenn Sie verhindern möchten, dass die entsprechenden Pakete jeweils unter zwei verschiedenen Namen angezeigt werden. Dadurch beschränken Sie natürlich das Argument von `package require` auf einen einzigen Namen.

Da die Pakete "Scrollutil" und "Scrollutil_tile" im Namespace `scrollutil` implementiert sind, müssen Sie entweder die

```
namespace import scrollutil::pattern ?scrollutil::pattern ...?
```

Befehl zum Importieren der benötigten *Prozeduren* oder verwenden Sie qualifizierte Namen wie `scrollutil::scrollarea`. Im folgenden Beispiel examples haben wir uns für den letzteren Ansatz entschieden.

Um auf Scrollutil-*Variablen* zuzugreifen, *müssen* Sie qualifizierte Namen verwenden. Es gibt nur 5 Scrollutil-Variablen, die für den Zugriff außerhalb des Namespaces `scrollutil` vorgesehen sind:

- Die Variable `scrollutil::version` enthält die aktuelle Versionsnummer des Scrollutil-Pakets.
- Die Variable `scrollutil::library` enthält den Pfad zum Verzeichnis der Scrollutil-Installation.

- Die schreibgeschützte Variable `scrollutil::scalingpct` wird beim Laden des Pakets "Scrollutil" oder "Scrollutil_tile" über `package require` auf den Skalierungsprozentsatz gesetzt, der der DPI-Skalierungsstufe des Bildschirms entspricht. Scrollutil passt unter anderem die Standardbreite der Tk-Core-Bildlaufleisten unter X11 sowie die des Widgets ttk::scrollbar für die integrierten Themen `alt`, `clam`, `classic` und `default` an den Wert dieser Variable an. Die derzeit unterstützten Werte sind `100`, `125`, `150`, `175` und `200`. Sie können diese Variable beispielsweise verwenden, wenn Sie je nach DPI-Skalierungsstufe Bilder unterschiedlicher Größe erstellen möchten. Wenn Ihre Anwendung beispielsweise auf einem nicht skalierten Display Bilder der Größe 16 x 16 verwendet und `scrollutil::scalingpct` den Wert `150` hat, sollte die Bildgröße für dieses Display 24 x 24 betragen.
- Wenn die Tk-Version entweder mindestens 8.7 (mit integrierter SVG-Unterstützung) ist oder 8.6 und die Erweiterung "tksvg" in den Interpreter geladen werden kann, dann wird die Variable `scrollutil::svgfmt` auf eine Tcl-Liste gesetzt, die Sie an die Befehle zur Erstellung oder Bearbeitung von SVG-Bildern als Wert ihrer Option `-format` übergeben können, um sicherzustellen, dass Ihre Bilder korrekt skaliert werden.
- Die schreibgeschützte Variable `scrollutil::usingTile` hat im Paket "Scrollutil" den Wert `0` und im Paket "Scrollutil_tile" den Wert `1`.

Das Paket "Scrollutil_tile" prüft, ob die erforderlichen Tk- und Tile-Versionen vorhanden sind, indem es die folgenden Befehle ausführt:

```
package require Tk 8.4-
if {$::tk_version < 8.5 || [regexp {^8\.5a[1-5]$} $::tk_patchLevel]} {
    package require tile 0.8-
}
```

Der zweite Befehl oben spiegelt die Tatsache wider, dass "Tile" ab Tk 8.5a6 in den Tk-Kern integriert ist und daher nur bei Verwendung einer früheren Tk-Version explizit geladen werden sollte.

Mehr zu `scrollutil::scalingpct`

Der Scrollutil-Code setzt die Variable `scrollutil::scalingpct` auf den Wert, der von der öffentlichen Prozedur `scaleutil::scalingPercentage` des Pakets "scaleutil" gegeben wird, das im Lieferumfang von Scrollutil enthalten ist. Die Art und Weise, wie dieser Wert berechnet wird, hängt vom Fenstersystem ab:

Unter Windows und Mac OS X/11+ wird der Skalierungsprozentsatz anhand des von Tk vorgegebenen Skalierungsfaktors `[tk scaling]` berechnet. HINWEIS: Unter Mac OS X/11+ ist das Ergebnis unabhängig von der Skalierungsstufe des Bildschirms immer `100`. Auf diesem System skaliert die Desktop-Engine alles automatisch nach Bedarf.

Unter X11 erfolgt die Berechnung des Skalierungsprozentsatzes aus `[tk scaling]` nur als Ausweichmethode, da die Implementierung der Bildschirmskalierung stark von der Desktop-Umgebung abhängt und größtenteils Systemressourcen nutzt, die außerhalb von Xlib liegen, auf der Tk basiert. (Auch herkömmliche X-Anwendungen wie `bitmap` und `xmag` sind davon betroffen.) Mit der teilweisen Ausnahme von Xfce und MATE (siehe unten) berechnet die Prozedur den Skalierungsprozentsatz anhand des Werts der X-Ressource `Xft.dpi`, indem die Anwendung `xrdb` ausgeführt wird. Auf GNOME-basierten Systemen, auf denen `xrdb` nicht standardmäßig installiert ist (z. B. Solus GNOME und Solus Budgie), werden stattdessen die Anwendung `xrandr` und die Datei `~/.config/monitors.xml` verwendet.

- Unter *Xfce* kann der Anzeigeskalierungsmodus, der als *Fensterskalierung* bezeichnet wird, auf "1x" oder "2x" eingestellt werden und im Dialogfeld *Darstellung* ausgewählt werden. Xfce-Versionen ab 4.16 unterstützen zudem eine fraktionierte Skalierung, deren Parameter im Dialogfeld *Anzeige* konfiguriert werden können. Wenn der Modus "2x" ausgewählt wurde, wird der Skalierungsprozentsatz durch die Prozedur `scaleutil::scalingPercentage` bis `200` festgelegt; ansonsten wird er anhand des Werts der Schriftart-DPI berechnet, der durch die X-Ressource `Xft.dpi` vorgegeben wird. Der Wert der Schriftart-DPI kann auf der Registerkarte *Schriftarten* des Dialogs *Darstellung* oder (unter Linux Lite 5+) über den Dialog *HiDPI-Einstellungen* festgelegt werden.
- Unter *MATE* kann der Anzeigenskalierungsmodus, der als *Fenster-Skalierungsfaktor* bezeichnet wird, auf "Automatische Erkennung", "Normal" oder "HiDPI" eingestellt werden und im Dialogfeld *MATE Tweak* ausgewählt werden. Der gleiche Effekt lässt sich erzielen, indem Sie im Dialogfeld *Monitoreinstellungen* einen der möglichen Werte "Auto-Detect", "100 %" und "200 %" auswählt. Wenn Bedarf besteht, erkennt die Prozedur `scaleutil::scalingPercentage` automatisch, welcher der Modi "Normal" ("100 %") und "HiDPI" ("200 %") dem voreingestellten Modus "Auto-detect" ("Auto-Erkennung") entspricht. Wenn der Modus "HiDPI" ("200 %") vom Benutzer ausgewählt wurde oder von der Prozedur automatisch erkannt wurde, wird der Skalierungsprozentsatz auf `200` gesetzt; andernfalls wird er anhand des Werts der Schriftart-DPI berechnet, der durch die X-Ressource `Xft.dpi` vorgegeben ist. Der Wert der Schriftart-DPI kann über das Dialogfeld *Font Rendering Details* eingestellt werden, das wiederum über die Registerkarte *Schriftarten* im Dialogfeld *Darstellungseinstellungen* geöffnet werden kann.
- Bei *GNOME unter Xorg, Budgie und Cinnamon in Versionen vor 4.6* kann die Bildschirmskalierung entweder 100 % oder 200 % betragen, und die Prozedur legt den Skalierungsprozentsatz entsprechend auf `100` oder `200` fest. In neueren GNOME- und Budgie-Versionen unter Ubuntu können Sie `x11-randr-fractional-scaling` als experimentelles Feature aktivieren, wodurch (mindestens) 125 %, 150 % und 175 % zur Liste der unterstützten Skalierungsprozentsätze hinzugefügt werden. HINWEIS: Aufgrund der Art und Weise, wie diese fraktionierte Skalierung implementiert ist, gibt `scaleutil::scalingPercentage` einen Wert zurück, der unabhängig von der gewählten Bildschirmskalierung `200` lautet. Cinnamon ab Version 4.6 unterstützt ebenfalls die fraktionierte Skalierung; ist diese aktiviert, lautet der von der Prozedur zurückgegebene Skalierungsprozentsatz `200`, unabhängig von der Auswahl der Bildschirmskalierung.
- *GNOME auf Wayland* unterstützt traditionell die Werte für die Bildschirmskalierung 100 % und 200 %, und der Skalierungsprozentsatz wird entsprechend auf `100` oder `200` festgelegt. In neueren GNOME-Versionen können Sie die experimentelle Funktion `scale-monitor-framebuffer` aktivieren, die (mindestens) 125 %, 150 % und 175 % zur Liste der unterstützten Skalierungsprozentsätze hinzufügt. Ist diese Funktion aktiviert, gibt `scaleutil::scalingPercentage` den Wert `100` bei älteren GNOME-Versionen zurück und `200` bei neueren Versionen, unabhängig vom Skalierungsgrad, da in diesem Fall nicht der Fensterinhalt, sondern die Monitor-Framebuffer in einem logischen Pixelkoordinatenraum skaliert werden.
- *KDE Plasma unter Xorg* bietet Unterstützung für die fraktionierte Skalierung. Auf diesem Desktop gibt die Prozedur je nach Skalierungsstufe des Bildschirms `100`, `125`, `150`, `175` oder `200` zurück.
- *KDE Plasma auf Wayland* unterstützt ebenfalls die fraktionelle Skalierung. In diesem Fall gibt

die Prozedur bei älteren KDE-Versionen immer den Wert `100` zurück, unabhängig vom Skalierungsgrad des Bildschirms, während sie bei neueren KDE-Versionen den Wert `100` gibt, `125`, `150`, `175` oder `200`.

Unter GNOME und Budgie lässt sich die Bildschirmskalierung auf der Seite *Bildschirme* des Dialogfelds *Einstellungen* einstellen, der Wert der X-Ressource `Xft.dpi`, die von der Prozedur `scaleutil::scalingPercentage` verwendet wird, kann jedoch auch durch die Einstellung des Skalierungsfaktors der Schriftarten über die Anwendung *Tweaks* angepasst werden. Ebenso lässt sich unter Cinnamon die Bildschirmskalierung auf der Seite *Bildschirm* des Dialogfelds *Systemeinstellungen* einstellen, doch der Wert der X-Ressource `Xft.dpi` kann auch durch Einstellen des Textskalierungsfaktors über die Seite *Schriftartauswahl* des Dialogfelds *Systemeinstellungen* angepasst werden. Schließlich kann unter KDE Plasma die Bildschirmskalierung auf der Seite *Bildschirmkonfiguration* des Dialogs *Systemeinstellungen* eingestellt werden, und unter KDE Plasma auf Xorg kann der Wert der Schriftart-DPI, der durch die X-Ressource `Xft.dpi` vorgegeben wird, ebenfalls über die Seite *Schriftarten* des Dialogs *Systemeinstellungen* eingestellt werden.

Neben der Berechnung des Skalierungsprozentsatzes führt die Prozedur `scaleutil::scalingPercentage` eine Reihe weiterer Aufgaben aus. Unter anderem ändert es die Standardhöhe der ttk::treeview-Zeilen vom fest codierten Wert von 20 Pixeln auf einen sinnvolleren Wert, basierend auf den Schriftmetriken der vom `Treeview`-Stil verwendeten Schriftart (in der Regel `TkDefaultFont`), und stellt sicher, dass dieser Schritt jedes Mal wiederholt wird, wenn das virtuelle Ereignis `<<ThemeChanged>>` empfangen wird (z. B. weil sich der Wert der Option `-font` des Stils `Treeview` geändert hat), oder das virtuelle Ereignis `<<TkWorldChanged>>` empfangen wird, bei dem das Feld `user_data` (%d) auf `FontChanged` gesetzt ist. Wenn die Tk-Version höher als 8.6.11 oder 8.7a5 ist, wird dieses virtuelle Ereignis von der Tk-Engine an alle Widgets gesendet, wenn eine Schriftart geändert wird, beispielsweise durch den Aufruf von `font configure`(siehe TIP 608).

Die im weiteren Verlauf dieses Abschnitts beschriebenen zusätzlichen Schritte werden nur ausgeführt, wenn der Skalierungsprozentsatz größer ist als `100`.

Unter X11 synchronisiert die Prozedur den von Tk verwendeten Skalierungsfaktor für die Umrechnung zwischen physikalischen Einheiten und Pixeln mit dem Skalierungsprozentsatz, indem sie einen aus letzterem abgeleiteten Wert an den Befehl `tk scaling` übergibt. Während dies in den allermeisten Anwendungen das gewünschte und empfohlene Verhalten ist, kann es Fälle geben, in denen Sie diesen Schritt unterdrücken möchten, da Ihre Anwendung möglicherweise über eine eigene Logik zur Bestimmung des Werts des Skalierungsfaktors von Tk verfügt. Dies können Sie erreichen, indem Sie die Zeile

```
namespace eval ::scaleutil { set keepTkScaling 1 }
```

bevor Scrollutil oder Scrollutil_tile über `package require` geladen wird.

Die Prozedur `scaleutil::scalingPercentage` korrigiert bei Bedarf auch die Größen der Standardschriften. Diese Schriftarten (`TkDefaultFont`, `TkTextFont` usw.) sind in der Datei `$tk_library/ttk/fonts.tcl` definiert. Lange Zeit waren die in dieser Datei angegebenen Schriftarten für X11 in Pixeln angegeben, was für die Verwendung auf HiDPI-Bildschirmen nicht geeignet war. Dies führte dazu, dass mehrere Linux-Distributionen gepatchte Versionen dieser Datei mitlieferten, in denen die Größenangaben in Pixeln durch Größenangaben in Punkten ersetzt wurden. Die gleiche Korrektur wurde im Februar 2020 in das Tk-Core-Repository Commit

und ist nun in Tk 8.7a5 und späteren Versionen enthalten. Um sicherzustellen, dass die Schriftgrößen unabhängig von der Tk-Version zum Skalierungsgrad des Bildschirms passen, überprüft die Prozedur diese Bibliotheksdatei und setzt, wenn diese Größen in Pixeln enthalten sind, die Option `-size` der Standardschriften auf entsprechende Größen in Punkten (ohne die Datei zu verändern). Zudem verdoppelt die Prozedur für den "2x"-Modus unter Xfce die Größen (in Punkten) der Standardschriften (die Funktionsweise der Bildschirmskalierung auf diesem Desktop macht dies erforderlich).

Die Prozedur skaliert außerdem:

- die Standardparameter des Trenngriffs (sash) des Tk-Core-panedwindow;
- die Standardgröße des Tk-Core-Scale-Widgets und seines Schiebereglers;
- die Standardbreite der Tk-Core-Bildlaufleisten unter X11;
- die Standardlänge der Widgets ttk::scale und ttk::progressbar;
- für die eingebauten Themen `alt` und `default` einige Stiloptionen der Widgets ttk::scrollbar, ttk::scale, ttk::progressbar, ttk::combobox, ttk::spinbox, ttk::button, toolbutton, ttk::menubutton, ttk::checkbutton, ttk::radiobutton, ttk::notebook, ttk::panedwindow und ttk::treeview;
- für das eingebaute Thema `clam` einige Stiloptionen der Widgets ttk::scrollbar, ttk::scale, ttk::progressbar, ttk::combobox, ttk::spinbox, ttk::button, toolbutton, ttk::menubutton, ttk::checkbutton, ttk::radiobutton, ttk::notebook, ttk::panedwindow, ttk::treeview und ttk::labelframe;
- für das eingebaute Thema `classic` einige Stiloptionen der Widgets ttk::scrollbar, ttk::scale, ttk::progressbar, ttk::combobox, ttk::spinbox, ttk::button, toolbutton, ttk::menubutton, ttk::checkbutton, ttk::radiobutton, ttk::notebook, ttk::panedwindow und ttk::treeview;
- für das eingebaute Thema `vista` einige Stiloptionen der Widgets ttk::combobox, ttk::button, toolbutton, ttk::menubutton, ttk::checkbutton, ttk::radiobutton, ttk::notebook, ttk::panedwindow und ttk::treeview;
- für das eingebaute Thema `winnative` einige Stiloptionen der Widgets ttk::scale, ttk::progressbar, ttk::combobox, ttk::spinbox, toolbutton, ttk::menubutton, ttk::checkbutton, ttk::radiobutton, ttk::notebook, ttk::panedwindow und ttk::treeview;
- für das eingebaute Thema `xpnative` einige Stiloptionen der Widgets ttk::combobox, ttk::spinbox, ttk::button, toolbutton, ttk::menubutton, ttk::checkbutton, ttk::radiobutton, ttk::notebook, ttk::panedwindow und ttk::treeview.

Darüber hinaus stellt die Prozedur `scaleutil::scalingPercentage` sicher, dass in den Themen `vista` und `xpnative` die Indikatoren der Widgets ttk::checkbutton und ttk::radiobutton korrekt skaliert angezeigt werden, unabhängig von der verwendeten Tk-Version. (Ein seit Langem bestehender Fehler in der Implementierung dieser Widgets wurde im Mai 2020 behoben und ist nun sowohl in Tk 8.6.11 und höher als auch in 8.7a5 und höher enthalten; die Prozedur bietet jedoch eine automatische Abhilfe für die Tk-Versionen, die noch von diesem Fehler betroffen sind.)

HINWEIS: In Tk-Versionen vor Version 9 sind die Tk-Core-Widgets "checkbutton" und "radiobutton" enthalten. sowie die Indikatoren der Widgets "ttk::checkbutton" und "ttk::radiobutton" der Themen `alt` und `winnative` nicht skalierbar sind (in Tk 9 und höher werden sie automatisch skaliert). Das Gleiche gilt für das Widget "ttk::sizegrip" in den integrierten Themen.

Die Prozedur `scrollutil::getScalingPct` **und der Skalierungsfaktor von Tk**

Das Scrollutil-Paket stellt die Hilfsprozedur `scrollutil::getScalingPct` bereit, die keine Argumente erwartet und den Skalierungsprozentsatz der Anwendung gibt, der aus dem *aktuellen* Skalierungsfaktor von Tk abgeleitet wird, der durch `[tk scaling]` angegeben wird. Der Wert, der von der Prozedur zurückgegeben wird, ist auf Vielfache von `25` beschränkt, beträgt mindestens `100` und kann größer als `200` sein, insbesondere wenn AndroWish auf einem Tablet oder Smartphone ausgeführt wird. Scrollutil nutzt diese Prozedur, um sicherzustellen, dass die Größen seiner GUI-Komponenten dem aktuellen Skalierungsfaktor von Tk entsprechen (der möglicherweise über `tk scaling` geändert wurde), und es wird empfohlen, dies auch im Anwendungscode zu tun.

Wenn die Tk-Version 9 oder höher ist, ist die im vorherigen Abschnitt beschriebene Funktionalität der Prozedur `scaleutil::scalingPercentage` in den Kern integriert. Als Teil dieser Funktionalität setzt Tk 9+ die Variable `tk::scalingPct` bei der Initialisierung auf den Skalierungsprozentsatz, der der DPI-Skalierungsstufe des Bildschirms entspricht, und die Prozedur `scaleutil::scalingPercentage` gibt lediglich das Minimum aus dem aktuellen Wert von `tk::scalingPct` und `200` zurück. Zur Erinnerung: In Tk 9+ wird der Wert der Variablen `tk::scalingPct` automatisch aktualisiert, sobald der Skalierungsfaktor über `tk scaling` geändert wird. Folglich gilt: Wenn die Anwendung den Skalierungsfaktor von Tk *vor dem Laden von Scrollutil oder Scrollutil_tile* über `package require` ändert, wird die Variable `scrollutil::scalingpct` auf den kleinsten Wert aus dem automatisch aktualisierten Wert von `tk::scalingPct` und `200` gesetzt. Dies führt zu folgendem Ergebnis

Empfehlung: Wenn Ihre Anwendung den Skalierungsfaktor von Tk durch Aufruf von `tk scaling` ändert, sollte dies *vor dem Laden von Scrollutil oder Scrollutil_tile* geschehen.

Mehr zu `scrollutil::svgfmt`

Wenn die Tk-Version entweder mindestens 8.7 (mit integrierter SVG-Unterstützung) oder 8.6 ist und das Paket "tksvg" in den Interpreter geladen werden kann, setzt Scrollutil die Variable `scrollutil::svgfmt` auf

```
[list svg -scale [expr {[scrollutil::getScalingPct] / 100.0}]]
```

Typische Werte sind `{svg -scale 1.0}`, `{svg -scale 1.25}`, `{svg -scale 2.0}` usw.

Es wird empfohlen, den Wert der Variablen `scrollutil::svgfmt` an die Befehle `image create photo`, `imageName configure`, `imageName put`und `imageName read` als Wert für deren Option `-format` zu übergeben, um sicherzustellen, dass deren Größen dem aktuellen Skalierungsprozentsatz der Anwendung entsprechen.

Beispiele

Das Hilfsskript `styleUtil.tcl`

Alle Beispiele im Verzeichnis `demos` verwenden Tile-Widgets (TTK) und enthalten die folgenden Zeilen

```
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]
```

Das Skript `styleUtil.tcl` beginnt mit einem Kommentar zu den Konfigurationsoptionen

"scrollarea" für `-autohidescrollbars` und `-setfocus`:

```
#
# To set the "-autohidescrollbars" or "-setfocus" option of all
    scrollarea
# widgets in all demo scripts to true, uncomment the corresponding
    line below:
#
# option add *Scrollarea.autoHideScrollbars 1
# option add *Scrollarea.setFocus           1
```

Es steht Ihnen frei, diesen Hinweis zu befolgen oder die Demo-Skripte mit den Standardeinstellungen für die Bildlaufbereiche `-autohidescrollbars 0` und `-setfocus 0` auszuführen.

Das Skript passt einige TTK-Widget-Stile an und definiert den Stil `Small.Toolbutton`.

Der Patch für den Stil `TCombobox` stellt sicher, dass die (schreibgeschützten) ttk::combobox-Widgets der Themen `alt`, `clam`, `classic` und `default` anzeigen, ob sie den Fokus haben. Diese grundlegende Anforderung, die die Tastaturnavigation benutzerfreundlicher macht, wird bereits von den Themen `vista`, `xpnative` und `aqua` erfüllt.

Die Widgets "ttk::button" des Stils `Small.Toolbutton`, die durch die in diesem Skript implementierte Prozedur `styleutil::createToolbutton` erstellt wurden, werden hervorgehoben angezeigt, wenn sie den Fokus haben. Auch dies trägt dazu bei, die Tastaturnavigation benutzerfreundlicher zu gestalten.

Eine weitere von diesem Skript bereitgestellte Prozedur ist `styleutil::getCurrentTheme`, die durch Aufruf des Befehls `ttk::style theme use` den Namen des aktuellen Themas gibt. Wenn eine ältere Tile-Version verwendet wird, die diese Methode noch nicht unterstützt, gibt die Prozedur den Wert der Variablen `::ttk::currentTheme` zurück (die ausschließlich durch den Befehl `ttk::setTheme` gesetzt wird). Diese Prozedur wird von fast allen in der Scrollutil-Distribution enthaltenen Demo-Skripten verwendet.

Das Skript passt außerdem das Thema `clam` an, indem es die öffentliche Prozedur `themepatch::patch` des Pakets "themepatch" aufruft, das im Lieferumfang von Scrollutil enthalten ist (das jedoch – im Gegensatz zum "scaleutil"-Paket – vom Scrollutil-Code nicht verwendet wird und daher explizit über `package require themepatch` geladen werden muss):

```
package require scrollutil_tile

. . .

#
# Patch the clam theme's styles TButton, TMenubutton,
# Heading, TCheckbutton, and TRadiobutton
#
package require themepatch
themepatch::patch clam
```

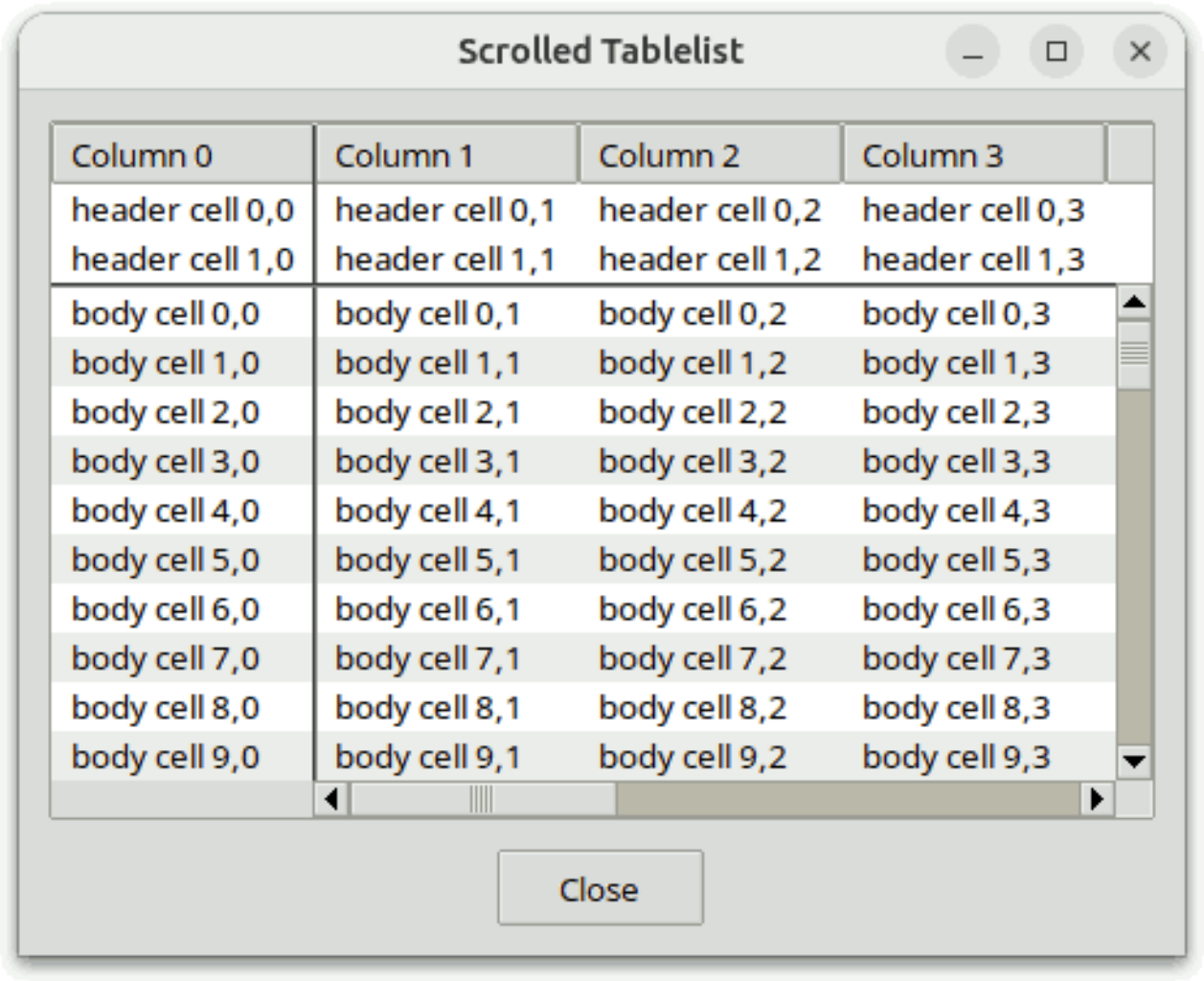
Unter X11 legt das Skript das Thema auf diese gepatchte Variante des Themas `clam` fest, das

über kleinere "ttk::button"-Widgets sowie "ttk::treeview"- und Tablelist-Kopfzeilen verfügt, und dessen skalierbare Widgets ttk::checkbutton und ttk::radiobutton weisen ein deutlich verbessertes Erscheinungsbild und Verhalten auf.

Ein scrollbares Tablelist-Widget

Dieses Beispiel zeigt, wie Sie die Erstellung einer scrollbaren Tablelist mithilfe eines scrollarea -Widgets erheblich vereinfachen können.

Das Skript `ScrolledTablelist1.tcl` im Verzeichnis `demos` erstellt ein Widget vom Typ "Tablelist" mit horizontalem und vertikalem Bildlauf, das über zwei Kopfzeilen und eine Titelspalte verfügt, und verwaltet die beiden Bildlaufleisten so, dass die vertikale Bildlaufleiste unterhalb der Kopfzeile der Tablelist erscheint und die horizontale Bildlaufleiste rechts neben dem Bereich der Spalten des Widgets beginnt:



ScrolledTablelist

Das Skript erfüllt diese Anforderungen mithilfe der herkömmlichen Verwaltung von Bildlaufleisten, die im Folgenden in der Farbe "red" dargestellt ist:

```
package require Tk 8.5-
package require tablelist_tile 6.3-
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Scrolled Tablelist"

#
# Create the tablelist and the scrollbars as children
# of a frame having -borderwidth 1 and -relief sunken
#
set f   [ttk::frame .f]
set frm [ttk::frame $f.frm -borderwidth 1 -relief sunken]
set tbl $frm.tbl
set vsb $frm.vsb
set hsb $frm.hsb
tablelist::tablelist $tbl ... -borderwidth 0 \
    -xscrollcommand [list $hsb set] -yscrollcommand [list $vsb set]
. . .
ttk::scrollbar $vsb -orient vertical   -command [list $tbl yview]
ttk::scrollbar $hsb -orient horizontal -command [list $tbl xview]

. . .

#
# Manage the widgets within the frame
#
grid $tbl -row 0 -rowspan 2 -column 0 -columnspan 2 -sticky news
if {[tk windowingsystem] eq "win32"} {
    grid $vsb -row 0 -rowspan 2 -column 2 -sticky ns
} else {
    grid [$tbl cornerpath] -row 0 -column 2 -sticky ew
    grid $vsb              -row 1 -column 2 -sticky ns
}
grid [$tbl cornerpath -sw] -row 2 -column 0 -sticky ns
grid $hsb                  -row 2 -column 1 -sticky ew
grid rowconfigure    $frm 1 -weight 1
grid columnconfigure $frm 1 -weight 1

. . .

#
# Manage the frame
#
pack $frm -expand yes -fill both -padx 9p -pady 9p
pack $f   -expand yes -fill both
```

Das Skript `ScrolledTablelist2.tcl` im Verzeichnis `demos` ersetzt den oben genannten, recht technischen Code durch nur wenige Zeilen (unten in der Farbe red dargestellt), indem es die Tablelist in ein Scrollarea-Widget einbettet. Dazu ist Tablelist Version 6.5 erforderlich, damit die "Scrollarea"-Optionen `-respectheader` und `-respecttitlecolumns` wie erwartet funktionieren (bei früheren Tablelist-Versionen werden diese Optionen stillschweigend ignoriert). Ein weiterer Vorteil ist, dass die mit dieser Methode erstellten Bildlaufleisten den Standard-Anzeigemodus `dynamic` aufweisen.

```
package require Tk 8.5-
package require tablelist_tile 6.5-
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Scrolled Tablelist"

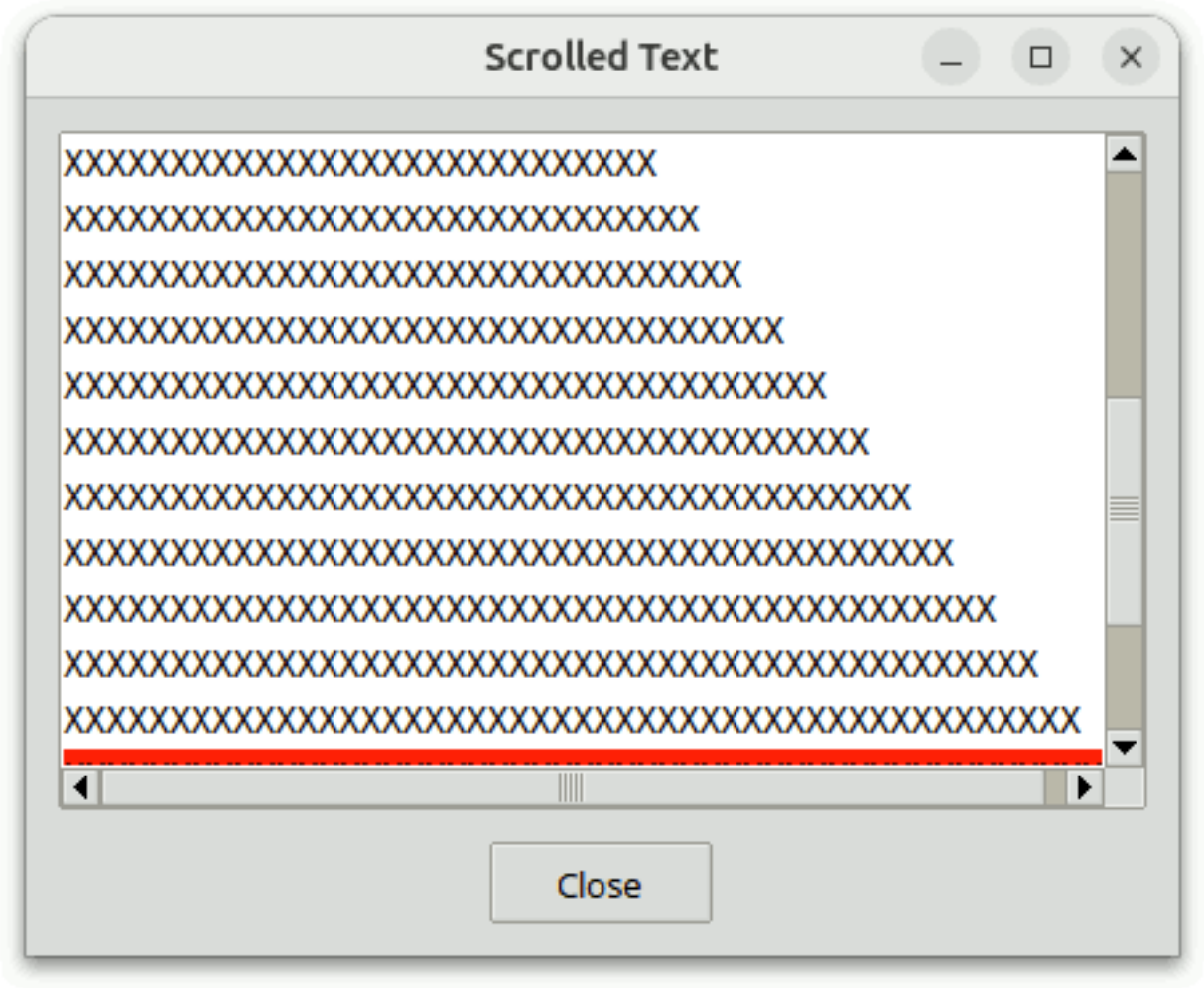
#
# Create the tablelist within a scrollarea
#
set f  [ttk::frame .f]
set sa [scrollutil::scrollarea $f.sa]
set tbl $sa.tbl
tablelist::tablelist $tbl ...
. . .
$sa setwidget $tbl

. . .

#
# Manage the scrollarea
#
pack $sa -expand yes -fill both -padx 9p -pady 9p
pack $f  -expand yes -fill both
```

Ein scrollbares Text-Widget

Das Skript `ScrolledText.tcl` im Verzeichnis `demos` zeigt, wie das Widget "scrollarea" den möglichen Shimmering-Effekt in der Verbindung mit Text-Widgets umgeht.



ScrolledText

Hier ist der entsprechende Code, in dem die Zeilen, die sich auf das "scrollarea"-Widget beziehen, in der Farbe "red" dargestellt sind:

```
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Scrolled Text"

#
# Create a text widget within a scrollarea
#
set f  [ttk::frame .f]
set sa [scrollutil::scrollarea $f.sa -lockinterval 10]
set txt [text $sa.txt -font TkFixedFont -width 49 -height 12 \
         -spacing1 1.5p -spacing3 1.5p -wrap none]
$sa setwidget $txt

#
# Populate the text widget and set the background color of line #25 to
```

```
    red
#
for {set i 1} {$i <= 30} {incr i} {
    set j [expr {2*$i}]
    $txt insert end [string repeat x $j]\n
}
$txt delete 30.end
$txt tag configure bgRed -background red
$txt tag add bgRed 25.0 25.end

. . .

#
# Manage the scrollarea
#
pack $sa -expand yes -fill both -padx 9p -pady 9p
pack $f  -expand yes -fill both

#
# Adjust the vertical view in the text window
# so that line #25 becomes the bottom line
#
tkwait visibility $txt
after 100 [list $txt yview 14.0]
```

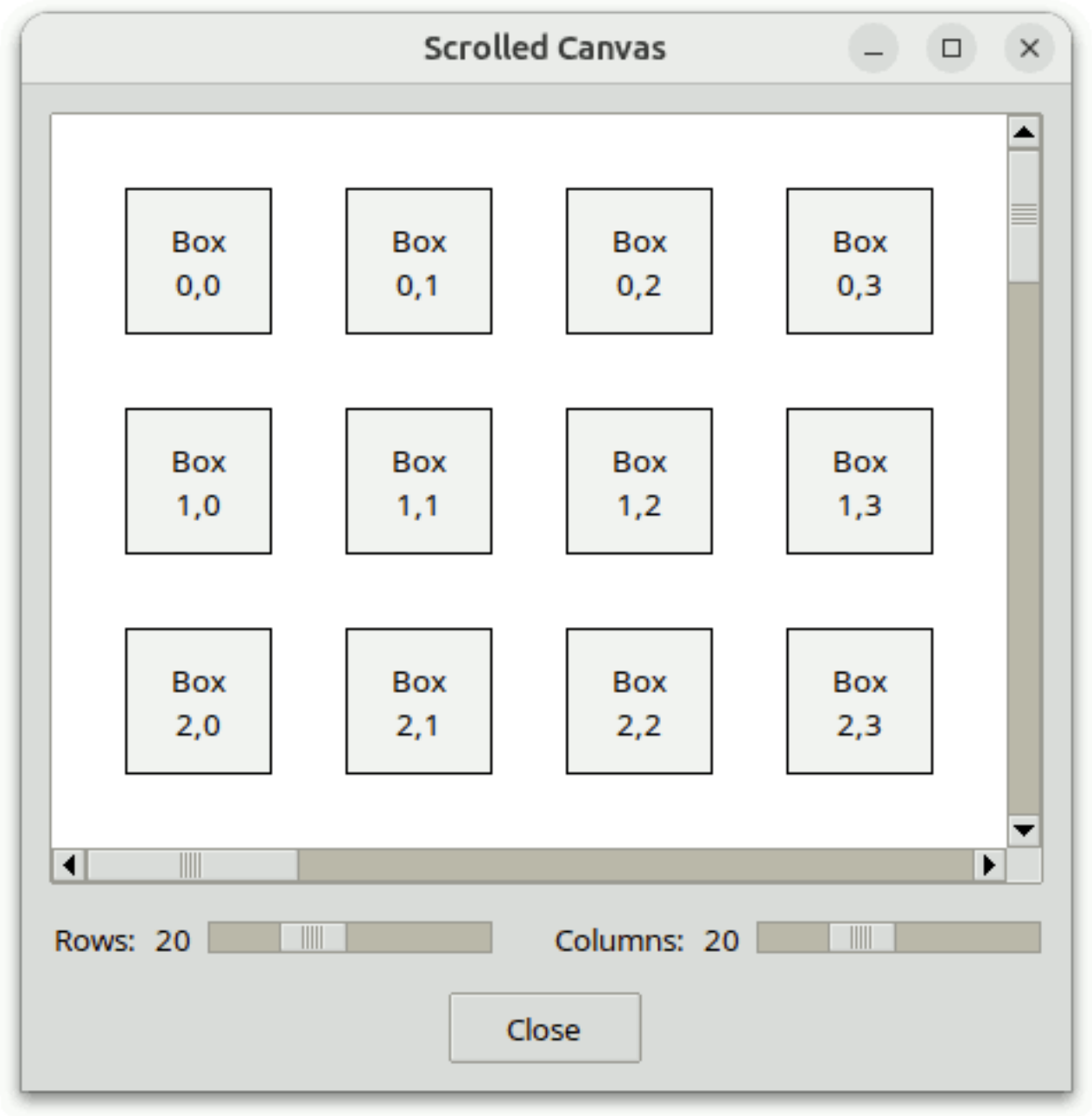
Das Skript erstellt ein Text-Widget mit der Bezeichnung `$txt`, das in einen Scrollbereich eingebettet ist, füllt es mit 30 Zeilen und passt die vertikale Ansicht im Textfenster so an, dass Zeile 25 zur untersten Zeile wird. Diese Zeile hat 50 Zeichen und passt daher nicht vollständig in das Fenster, dessen Breite 49 Zeichen beträgt. Folglich gibt der Befehl `$txt xview` die Liste `{0.0 0.98}` zurück, wodurch die horizontale Bildlaufleiste des Scrollbereichs eingeblendet wird und den größten Teil der untersten Zeile verdeckt. Da diese Zeile den Hintergrund `red` hat, lässt sich leicht erkennen, wie viel davon über den oberen Rand der Bildlaufleiste hinausragt.

Schauen wir uns einmal an, was passiert, wenn die Höhe des Text-Widgets durch Ziehen am oberen oder unteren Rand des Hauptfensters so weit verringert wird, dass die roten Pixel von der horizontalen Bildlaufleiste verdeckt werden. Nach dieser Aktion ist Zeile Nr. 25 vollständig aus dem Sichtbereich verschwunden und die neue unterste Zeile ist Zeile Nr. 24, die 48 Zeichen enthält; daher gibt der Befehl `$txt xview` den Wert `{0.0 1.0}` zurück. Normalerweise würde dies dazu führen, dass die horizontale Bildlaufleiste ausgeblendet wird. Dadurch würde jedoch Zeile Nr. 25 zur untersten Zeile werden, was wiederum dazu führen würde, dass die horizontale Bildlaufleiste erneut eingeblendet wird. Diesmal würde die Bildlaufleiste diese Zeile vollständig verdecken, was dazu führen würde, dass Zeile Nr. 24 zur untersten Zeile wird, wodurch die Bildlaufleiste wieder ausgeblendet würde, und so weiter. Mit anderen Worten: Die horizontale Bildlaufleiste würde in einer Endlosschleife ein- und ausgeblendet werden, was zu einem störenden Flackereffekt führen würde. Der integrierte Sperrmechanismus des "scrollarea"-Widgets schützt vor solchen potenziellen Endlosschleifen. Um sicherzustellen, dass die Sperre wie erwartet funktioniert, haben wir die Scrollarea-Option `-lockinterval` auf `10`

gesetzt (zur Erinnerung: Der Standardwert ist 1).

Ein scrollbares Canvas-Widget

Das Skript `ScrolledCanvas.tcl` im Verzeichnis `demos` zeigt, wie Sie das Scrollutil-Paket verwendet, um einem Canvas-Widget Bildlaufleisten sowie Unterstützung für Mausrad- und `<TouchpadScroll>`-Ereignisse hinzuzufügen. Außerdem wird gezeigt, wie Sie ttk::scale-Widgets um Unterstützung für das Mausrad und `<TouchpadScroll>`-Ereignisse erweitert.



ScrolledCanvas

Hier ist der entsprechende Code, in dem die Zeilen, die sich auf das Scrollutil-Paket beziehen, in der Farbe red dargestellt sind:

```
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Scrolled Canvas"

set scaleFactor [expr {[scrollutil::getScalingPct] / 100.0}]
set width  [expr {13 * 32 * $scaleFactor}]
set height [expr {10 * 32 * $scaleFactor}]
set scrlIncr [expr {16 * $scaleFactor}]

#
# Create a canvas widget within a scrollarea and add mouse wheel
    support to it
#
set f  [ttk::frame .f]
set sa [scrollutil::scrollarea $f.sa]
set c  [canvas $sa.c -background white -width $width -height $height \
        -xscrollincrement $scrlIncr -yscrollincrement $scrlIncr]
bind $c <Configure> { setScrollRegion %W %w %h }
scrollutil::addMouseWheelSupport $c
$sa setwidget $c

set rows 20
set cols 20

#
# Populate the canvas and then rescale the coordinates
# of all of the items by a factor of $scaleFactor
#
proc populate canv {
    $canv delete all
    global rows cols scaleFactor

    for {set row 0; set y 32} {$row < $rows} {incr row; incr y 96} {
        for {set col 0; set x 32} {$col < $cols} {incr col; incr x 96}
    {
            $canv create rectangle $x $y [expr {$x+63}] [expr {$y+63}]
    \
                -fill gray95
            $canv create text [expr {$x+32}] [expr {$y+32}] \
                -text "Box\n$row,$col" -anchor center -justify center
        }
    }

    $canv scale all 0 0 $scaleFactor $scaleFactor
```

```
}
populate $c

#
# Sets the scroll region of the canvas.
#
proc setScrollRegion {canv width height} {
    global cols rows scaleFactor
    set rightX [expr {($cols*96 + 32) * $scaleFactor}]
    set lowerY [expr {($rows*96 + 32) * $scaleFactor}]
    if {$rightX < $width}  { set rightX $width }
    if {$lowerY < $height} { set lowerY $height }
    $canv configure -scrollregion [list 0 0 $rightX $lowerY]
}

. . .

#
# Create a ttk::scale widget for setting the number of box rows
#
set fRows [ttk::frame $f.fRows]
pack [ttk::label $fRows.l1 -text "Rows:"] -side left -padx {0 3p}
pack [ttk::label $fRows.l2 -textvariable rows] -side left -padx {0 3p}
pack [ttk::scale $fRows.scl -from 5 -to 50 -value $rows -command \
      [list setBoxRows $fRows.scl]] -side left -expand yes -fill x

proc setBoxRows {scl val} {
    global rows c
    set val [expr {round($val)}]
    if {$val != $rows} {
        set rows $val
        populate $c
        setScrollRegion $c [winfo width $c] [winfo height $c]
    }
}

#
# Create a ttk::scale widget for setting the number of box columns
#

. . .

#
# Add mouse wheel support to the ttk::scale widgets
#
scrollutil::addMouseWheelSupport TScale
```

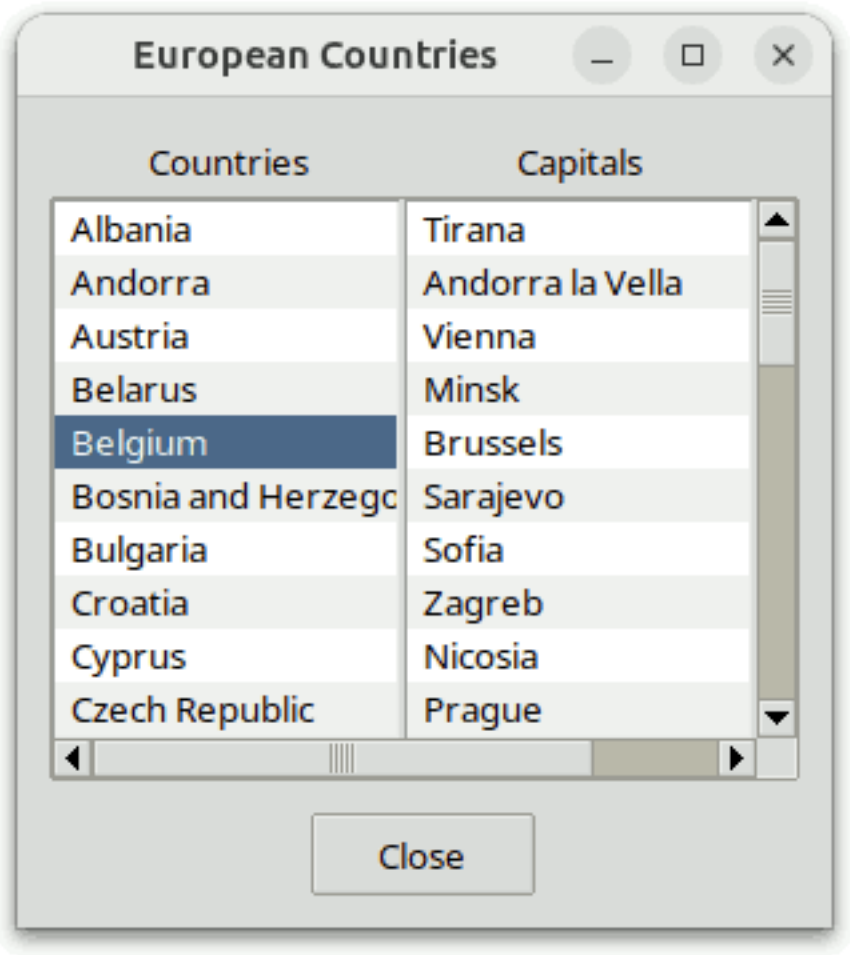
```
. . .
```

Das Skript erstellt ein canvas-Widget, füllt es mit den Elementen `rectangle` und `text`, und fügt ihm Unterstützung für Mausrad- und `<TouchpadScroll>`-Ereignisse hinzu, indem sein Name an den Befehl `scrollutil::addMouseWheelSupport` übergeben wird. HINWEIS: Wir arbeiten im canvas mit Pixeln statt mit Punkten, die Benutzeroberfläche jedoch mithilfe der Variablen `scaleFactor` vollständig skalierbar machen, deren Wert (`1.0`, `1.25`, `1.5` usw.) von `[scrollutil::getScalingPct]` abgeleitet wird.

Das Skript erstellt außerdem zwei ttk::scale-Widgets, die zur Festlegung der Anzahl der Zeilen und Spalten der Felder auf dem Canvas dienen, und erweitert die ttk::scale-Widgets um die Unterstützung für Mausrad- und `<TouchpadScroll>`-Ereignisse, indem die Widget-Klasse `TScale` an den Befehl `scrollutil::addMouseWheelSupport` übergeben wird.

Zwei listbox-Widgets synchronisieren

Das Skript `SyncListboxes.tcl` im Verzeichnis `demos` erstellt zwei listboxes innerhalb eines scrollsync-Widgets, das wiederum in ein scrollarea eingebettet ist.



SyncListboxes

Hier ist der entsprechende Code, in dem die Zeilen, die sich auf die Widgets "scrollarea" und "scrollsync" beziehen, in der Farbe red dargestellt sind:

```
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "European Countries"

. . .

set f  [ttk::frame .f]

. . .

#
# Create a scrollsync widget within a scrollarea
#
set sa [scrollutil::scrollarea $f.sa]
set ss [scrollutil::scrollsync $sa.ss]
$sa setwidget $ss

#
# Populate the scrollsync widget with two listboxes
#

. . .

set lb1 [listbox $ss.lb1 -activestyle none -highlightthickness 0
    -width 16]
set lb2 [listbox $ss.lb2 -activestyle none -highlightthickness 0
    -width 16]
$ss setwidgets [list $lb1 $lb2]

. . .

grid $lb1 $lb2 -sticky news -padx {0 1.5p}
grid rowconfigure    $ss 0 -weight 1
grid columnconfigure $ss 0 -weight 1
grid columnconfigure $ss 1 -weight 1

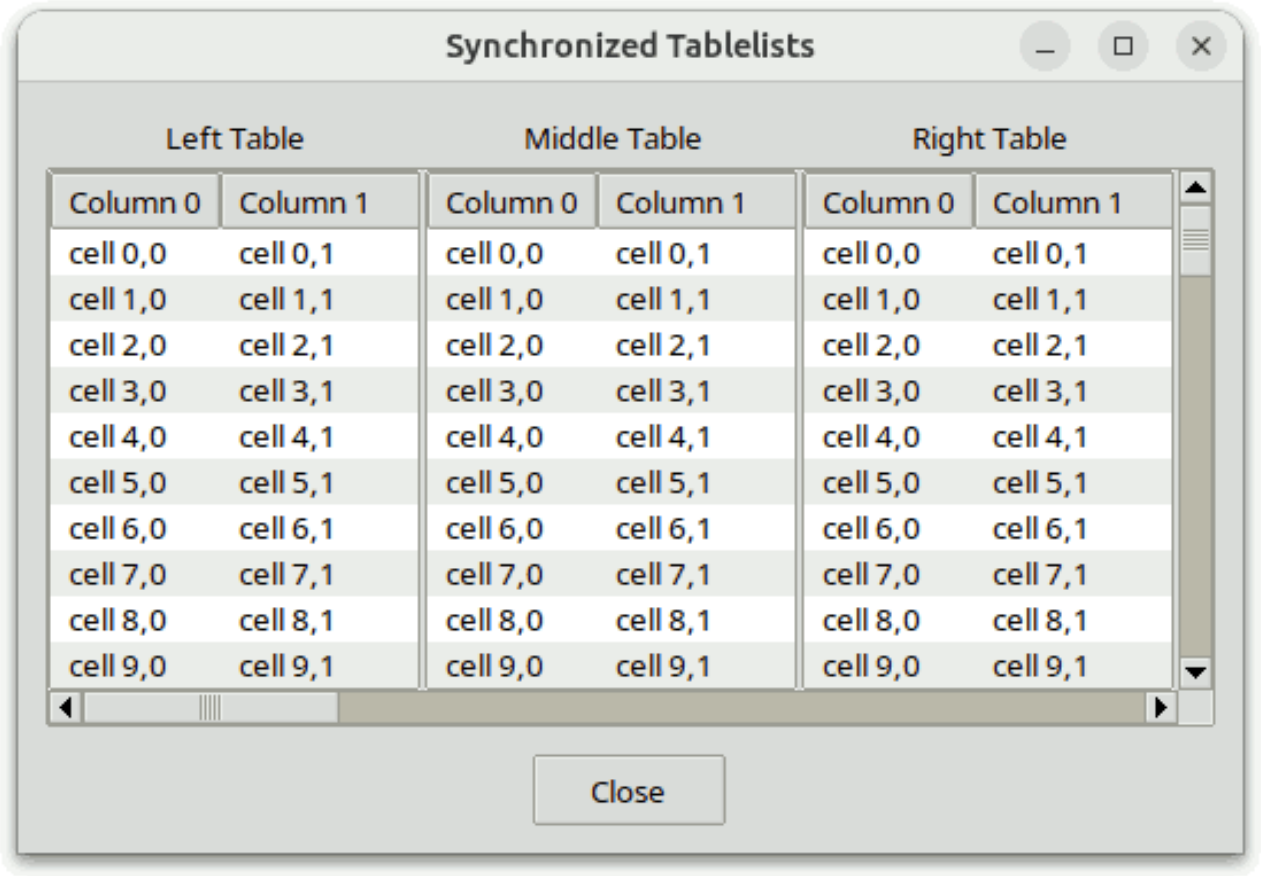
. . .

pack $sa -side top -expand yes -fill both -padx 9p -pady {1.5p 9p}
pack $f  -expand yes -fill both

. . .
```

Synchronisierung von drei Tablelist-Widgets

Das Skript `SyncTablelists.tcl` im Verzeichnis `demos` erstellt drei Tablelistn innerhalb eines scrollsync-Widgets, das wiederum in ein scrollarea eingebettet ist.



SyncTablelists

Der entsprechende Code ähnelt dem im vorherigen Beispiel gezeigten:

```
package require tablelist_tile
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Synchronized Tablelists"

. . .

set f  [ttk::frame .f]

. . .

#
# Create a scrollsync widget within a scrollarea
#
set sa [scrollutil::scrollarea $f.sa]
set ss [scrollutil::scrollsync $sa.ss]
```

```
$sa setwidget $ss

#
# Populate the scrollsync widget with three tablelists
#

tablelist::setThemeDefaults
if {$tablelist::themeDefaults(-stripebackground) eq ""} {
    option add *Tablelist.background            white
    option add *Tablelist.stripeBackground      #f0f0f0
}

for {set n 1; set colWidth 40} {$n <= 3} {incr n; incr colWidth 20} {
    set tbl [tablelist::tablelist $ss.tbl$n \
             -columns [list 0 "Column 0" left  $colWidth "Column 1"
    left]]
    set tbl$n $tbl

    for {set i 0} {$i < 40} {incr i} {
        $tbl insert end [list "cell $i,0" "cell $i,1"]
    }
}
$ss setwidgets [list $tbl1 $tbl2 $tbl3]

grid $tbl1 $tbl2 $tbl3 -sticky news -padx {0 1.5p}
grid rowconfigure    $ss 0 -weight 1
grid columnconfigure $ss 0 -weight 1
grid columnconfigure $ss 1 -weight 1
grid columnconfigure $ss 2 -weight 1

. . .

pack $sa -side top -expand yes -fill both -padx 9p -pady {1.5p 9p}
pack $f  -expand yes -fill both

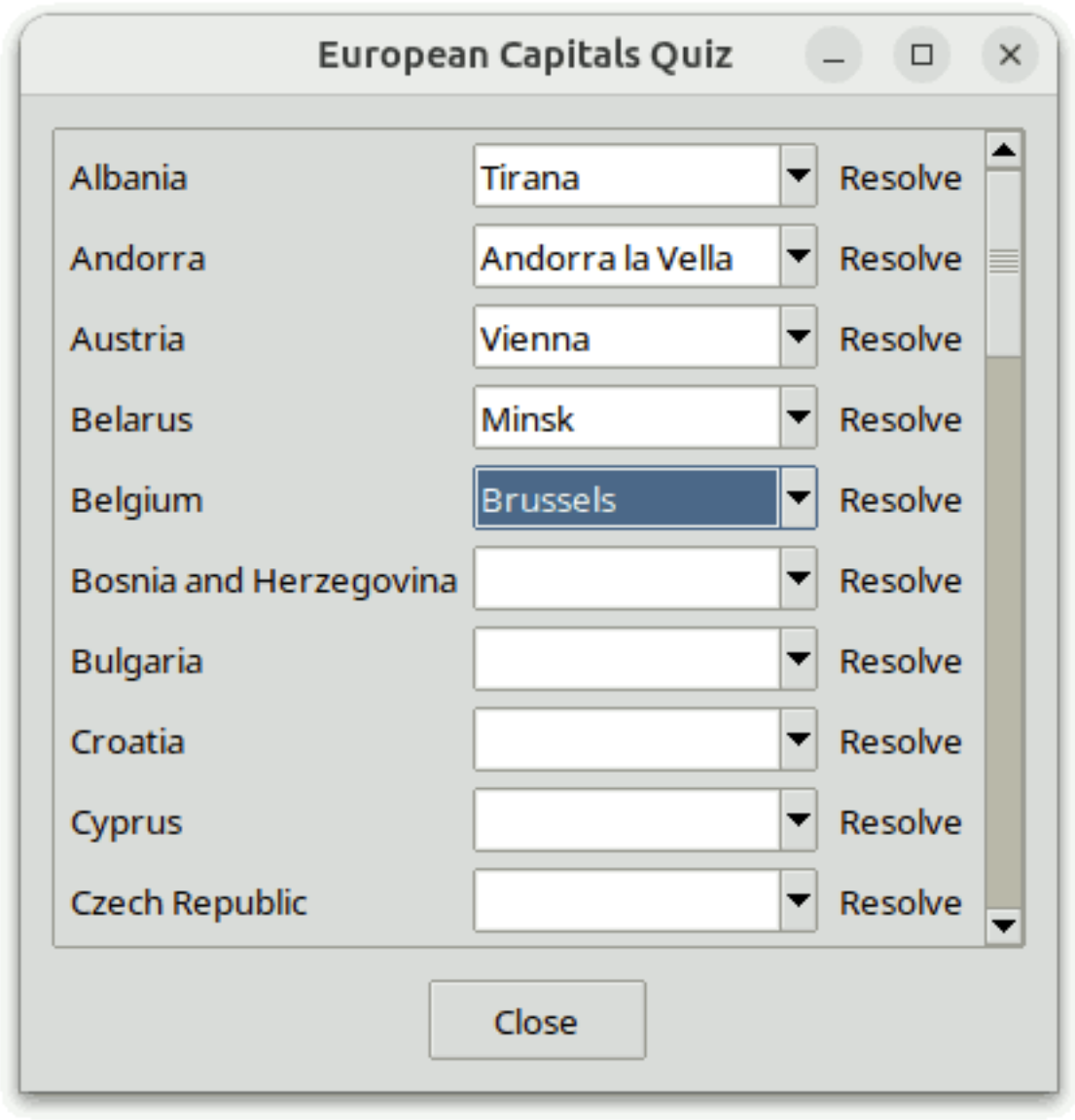
. . .
```

Die oben genannten Datenbank-Einstellungen für die Konfigurationsoptionen der Tablelist-Konfiguration für `-background` und `-stripebackground` sind beim Thema `aqua` nicht vorhanden, da für dieses Thema die Standardwerte dieser Optionen nicht nur spezifisch für aqua sind, sondern unter Mac OS 10.14 (Mojave) und höher zudem vom aktuellen Systemdesign (Hellmodus oder Dunkelmodus) abhängen.

Beachten Sie, dass Spalte Nr. 1 der drei Tablelist-Widgets jeweils 40, 60 und 80 Zeichen breit ist. Aus diesem Grund erreicht beim horizontalen Scrollen nach rechts zuerst die Ansicht der linken Tabelle ihre horizontale Endposition, dann die der mittleren Tabelle und zuletzt die der rechten Tabelle.

Ein Skript, das ein scrollutil::scrollableframe-Widget verwendet

Das Skript `SuScrollableFrmDemo1.tcl` im Verzeichnis `demos` erstellt ein scrollutil::scrollableframe-Widget, das in ein scrollarea eingebettet ist und mit Hilfe des Befehls `scrollutil::createWheelEventBindings` Mausrad- und `<TouchpadScroll>` -Ereignis-Bindungen für das Bindungs-Tag `"all"` erstellt. Zur Erinnerung: Der "scrollableframe" wurde bei seiner Erstellung durch diese Bindungen automatisch für das Scrollen registriert, daher ist es nicht erforderlich, den Befehl `scrollutil::enableScrollingByWheel` dafür erneut aufzurufen. Anschließend füllt das Skript den Inhaltsrahmen des "scrollableframe" mit "ttk::label"-Widgets, die die Namen der europäischen Länder anzeigen, "ttk::combobox"-Widgets zur Auswahl der entsprechenden Hauptstädte sowie "ttk::button"-Widgets im Stil `Small.Toolbutton` (erstellt mithilfe der Prozedur `styleutil::createToolbutton`, implementiert in der Datei `styleUtil.tcl`) für die weniger geduldigen Benutzer, die den Text "Resolve" anzeigen.



ScrollableFrmDemo1

Hier ist der entsprechende Code:

```
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "European Capitals Quiz"

#
# Create a scrollableframe within a scrollarea
#
set f  [ttk::frame .f]
set sa [scrollutil::scrollarea $f.sa]
set sf [scrollutil::scrollableframe $sa.sf]
$sa setwidget $sf

#
# Create mouse wheel event bindings for the binding tag "all"
#
scrollutil::createWheelEventBindings all

#
# Get the content frame and populate it
#

set cf [$sf contentframe]

set countryList {
    Albania Andorra Austria Belarus Belgium "Bosnia and Herzegovina"
    Bulgaria
    . . .
}
set capitalList {
    Tirana "Andorra la Vella" Vienna Minsk Brussels Sarajevo Sofia
    . . .
}

. . .

set capitalList [lsort $capitalList]

. . .

set row 0
foreach country $countryList {
    . . .

    set w [ttk::combobox $cf.cb$row -state readonly -width 14 \
```

```
            -values $capitalList]
    . . .

    #
    # Make the keyboard navigation more user-friendly
    #
    bind $w <<TraverseIn>> [list $sf see %W]

    #
    # Adapt the handling of the mouse wheel and mouse
    # button 2 events for the ttk::combobox widget
    #
    scrollutil::adaptWheelEventHandling $w
    scrollutil::adaptBtn2EventHandling $w

    . . .

    incr row
}

. . .
```

Die Tastaturnavigation gestalten wir mit dem Unterbefehl `see` des scrollableframe-Widgets benutzerfreundlicher, wenn das virtuelle Ereignis `<<TraverseIn>>` für die Widgets ttk::combobox und (oben nicht gezeigt) ttk::button behandelt wird. Zusätzlich rufen wir den Befehl `scrollutil::adaptWheelEventHandling` für jedes ttk::combobox-Widget auf. Das ist für eine benutzerfreundliche Ereignisbehandlung nötig, weil dieses Widget eingebaute Bindungen für das Mausrad und für `<TouchpadScroll>`-Ereignisse hat. Durch diesen Befehl wählen diese Ereignisse über einem der ttk::combobox-Widgets nur dann die nächste oder vorherige Hauptstadt, wenn das Widget den Fokus hat. Andernfalls scrollen sie den scrollableframe.

Für jedes ttk::combobox-Widget rufen wir zusätzlich den Befehl `scrollutil::adaptBtn2EventHandling` auf und stellen so sicher, dass die an das Widget gesendeten Ereignisse der Maustaste 2 nur dann vom Widget selbst verarbeitet werden, wenn es den Fokus hat; andernfalls führen sie dazu, dass der Inhalt des scrollbaren Rahmens gescannt wird. Während des Scannens nimmt der Cursor die Form einer zeigenden Hand an, und durch Ziehen der Maus wird der Rahmen mit dem Inhalt mit hoher Geschwindigkeit in Richtung der Mausbewegung durch das Fenster gezogen.

Ein Skript, das das BWidget-Widget "ScrollableFrame" verwendet

Das Skript `BwScrollableFrmDemo1.tcl` im Verzeichnis `demos` erstellt ein BWidget "ScrollableFrame", das in ein scrollarea-Widget eingebettet ist, erstellt mithilfe des Befehls `scrollutil::createWheelEventBindings` Mausrad- und `<TouchpadScroll>` -Ereigniszuordnungen für das Bindungs-Tag `"all"` und ruft den Befehl `scrollutil::enableScrollingByWheel` für diesen ScrollableFrame auf, wodurch dieser für das Scrollen über diese Bindungen registriert wird. Anschließend füllt es den Inhaltsrahmen des

"ScrollableFrame" mit denselben Widgets wie `SuScrollableFrmDemo1.tcl` im vorherigen Beispiel.

Hier ist der entsprechende Code:

```
package require BWidget
Widget::theme yes
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "European Capitals Quiz"

#
# Create a ScrollableFrame within a scrollarea
#
set f  [ttk::frame .f]
set sa [scrollutil::scrollarea $f.sa]
set sf [ScrollableFrame $sa.sf]
$sa setwidget $sf

. . .

#
# Create mouse wheel event bindings for the binding tag "all" and
# register the ScrollableFrame for scrolling by these bindings
#
scrollutil::createWheelEventBindings all
scrollutil::enableScrollingByWheel $sf

#
# Get the content frame and populate it
#

set cf [$sf getframe]

set countryList {
    Albania Andorra Austria Belarus Belgium "Bosnia and Herzegovina"
    Bulgaria
    . . .
}
set capitalList {
    Tirana "Andorra la Vella" Vienna Minsk Brussels Sarajevo Sofia
    . . .
}

. . .
```

```
set capitalList [lsort $capitalList]

. . .

set row 0
foreach country $countryList {
    . . .

    set w [ttk::combobox $cf.cb$row -state readonly -width 14 \
           -values $capitalList]
    . . .

    #
    # Make the keyboard navigation more user-friendly
    #
    bind $w <<TraverseIn>> [list $sf see %W]

    #
    # Adapt the handling of the mouse wheel events for the
    ttk::combobox widget
    #
    scrollutil::adaptWheelEventHandling $w

    . . .

    incr row
}

. . .
```

Ein Skript, das ein iwidgets::scrolledframe-Widget verwendet

Das Skript `ScrolledFrmDemo1.tcl` im Verzeichnis `demos` erstellt ein iwidgets::scrolledframe-Widget und legt mithilfe des Befehls `scrollutil::createWheelEventBindings` Mausrad- und `<TouchpadScroll>` -Ereigniszuordnungen für das Bindungs-Tag `"all"` an, und ruft den Befehl `scrollutil::enableScrollingByWheel` für diesen ScrolledFrame auf, wodurch dieser für das Scrollen über diese Bindungen registriert wird. Anschließend füllt es den Inhaltsrahmen des Scrolledframes mit denselben Widgets wie `SuScrollableFrmDemo1.tcl` und `BwScrollableFrmDemo1.tcl` in den beiden vorherigen Beispielen.

Hier ist der entsprechende Code:

```
if {[catch {package require iwidgets} result1] != 0 &&
    [catch {package require Iwidgets} result2] != 0} {
    error "$result1; $result2"
}
```

```
set dir [file dirname [info script]]
source [file join $dir scrolledwidgetPatch.itk] ;# adds ttk::scrollbar
    widgets
package require scrollutil_tile
source [file join $dir styleUtil.tcl]

wm title . "European Capitals Quiz"

. . .

#
# Create a scrolledframe
#
set f  [ttk::frame .f]
set sf [iwidgets::scrolledframe $f.sf -borderwidth 1 -relief sunken \
        -scrollmargin 0]
. . .

#
# Create mouse wheel event bindings for the binding tag "all"
# and register the scrolledframe for scrolling by these bindings
#
scrollutil::createWheelEventBindings all
scrollutil::enableScrollingByWheel $sf

#
# Get the content frame and populate it
#

set cf [$sf childsite]
. . .

<exactly as in the previous example, except the stuff related to
    keyboard navigation>

. . .
```

Der Code für die Tastaturnavigation ist in diesem Beispiel nicht enthalten, da das Widget "iwidgets::scrolledframe" keinen Unterbefehl `see` bereitstellt.

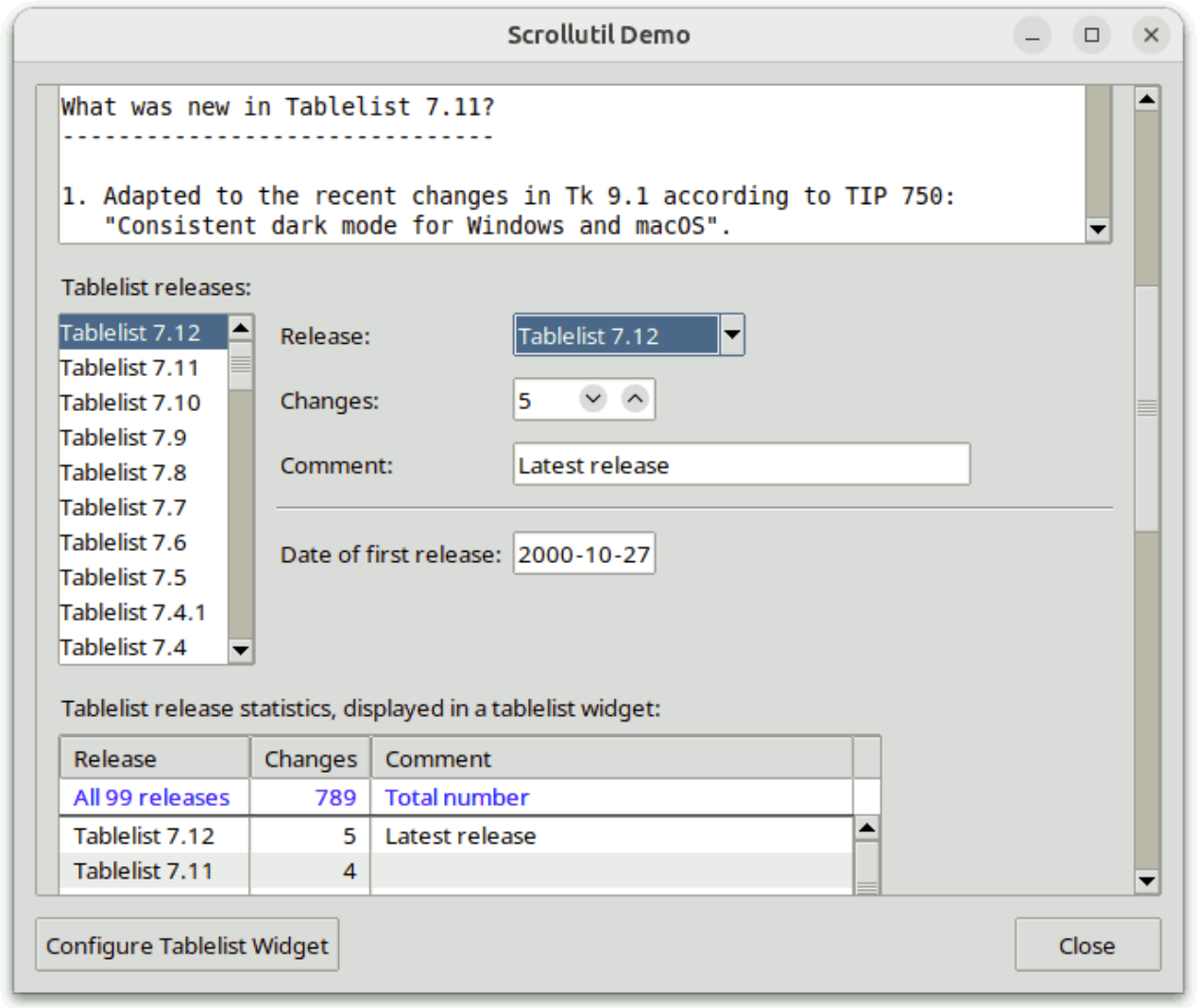
Ein Skript, das zwei scrollutil::scrollableframe-Widgets verwendet

Das Skript `SuScrollableFrmDemo2.tcl` im Verzeichnis `demos` erzeugt ein scrollutil::scrollableframe-Widget, das in einen scrollarea eingebettet ist, und führt dann mit `source` das Skript `SuScrollableFrmContent.tcl` aus, das den Inhaltsrahmen des scrollableframe mit den folgenden Widgets füllt:

- eine Reihe von ttk::label-Widgets;

- ein Widget mit scrollbarem Text `$txt` innerhalb eines Scrollbereichs;
- ein scrollbares listbox-Widget `$lb` innerhalb eines Scrollbereichs;
- ein ttk::combobox-Widget `$cb`;
- ein ttk::spinbox-Widget `$sb`;
- ein ttk::entry-Widget `$e`;
- ein ttk::separator-Widget;
- ein Mentry-Widget `$me` vom Typ `"Date"`;
- ein scrollbares Tablelistn-Widget `$tbl` innerhalb eines Scrollbereichs;
- Ein scrollbares ttk::treeview-Widget `$tv` innerhalb eines Scrollbereichs.

Mit Ausnahme von ttk::label und ttk::separator verfügen alle diese Widgets über integrierte Mausrad- und `<TouchpadScroll>`-Ereignis-Bindungen. Und mit Ausnahme von ttk::label, ttk::separator und ttk::treeview verfügen alle diese Widgets über integrierte Bindungen für das Maustaste 2-Ereignis.



ScrollableFrmDemo2

Hier ist der entsprechende Code:

```
package require Tk 8.5.9-                          ;# for ttk::spinbox
package require mentry_tile 3.2-                   ;# for mouse wheel
    support
package require tablelist_tile 6.5-                ;# for
    -(x|y)mousewheelwindow
                                                   ;# and
    scrollutil::scrollarea
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Scrollutil Demo"

#
# Create a scrollableframe within a scrollarea
#
set tf [ttk::frame .tf]
set sa [scrollutil::scrollarea $tf.sa]
set sf [scrollutil::scrollableframe $sa.sf]
$sa setwidget $sf

#
# Get the content frame and populate it
#
set cf [$sf contentframe]
source [file join $dir SuScrollableFrmContent.tcl]

#
# Make the keyboard navigation more user-friendly
#
foreach w [list $cb $sb $e $me] {
    bind $w <<TraverseIn>> [list $sf see %W]
}
foreach w [list $txt $lb $tbl $tv] {
    bind $w <<TraverseIn>> [list seeScrollarea $sf %W]
}
proc seeScrollarea {sf w} { $sf see [scrollutil::getscrollarea $w] }
```

Immer wenn das virtuelle Ereignis `<<TraverseIn>>` an eines der vier Widgets gesendet wird, die innerhalb von Scrollbereichen erstellt wurden, fragen wir über `scrollutil::getscrollarea` den Pfadnamen des entsprechenden Scrollbereichs ab und bringen diesen Scrollbereich (einschließlich der Bildlaufleisten und des Rahmens) in den sichtbaren Bereich, anstatt nur das betreffende Widget anzuzeigen. Zwar hätten wir *in diesem Skript* stattdessen auch `[winfo parent]` verwenden können, doch wird der Befehl `scrollutil::getscrollarea` empfohlen, da er auch für Widgets funktioniert, die keine untergeordneten Elemente der entsprechenden Scrollbereiche sind.

Hier finden Sie weitere Informationen zum Mausrad und zu den `<TouchpadScroll>` -Ereignissen unter Verwendung der Scrollutil-Befehle, die im Abschnitt Was ist Scrollutil? beschrieben sind:

```
#
# For the entry components of the mentry widget
# set the "focus check window" to the mentry
#
set entryList [$me entries]
scrollutil::setFocusCheckWindow {*}$entryList $me

#
# Create mouse wheel event bindings for the binding tag "all"
#
scrollutil::createWheelEventBindings all

#
# Adapt the handling of the mouse wheel events for the
# text, listbox, ttk::combobox, ttk::spinbox, ttk::entry,
# tablelist, and ttk::treeview widgets, as well as for
# the entry components of the mentry widget of type "Date"
#
### scrollutil::adaptWheelEventHandling $txt $lb $cb $sb $e
    {*}$entryList \
    $tbl $tv
$sf preparescroll
```

Beachten Sie, dass der Aufruf des Befehls `scrollutil::adaptWheelEventHandling` auskommentiert und durch den wesentlich einfacheren und sichereren Aufruf des Unterbefehls `preparescroll` "scrollableframe" ersetzt wurde.

Außerdem gibt es noch weitere Informationen zu den Ereignissen der Maustaste 2:

```
#
# Adapt the handling of the mouse button 2 events for the text,
    listbox,
# ttk::combobox, ttk::spinbox, ttk::entry, and tablelist widgets, as
# well as for the entry components of the mentry widget of type "Date"
#
### scrollutil::adaptBtn2EventHandling $txt $lb $cb $sb $e
    {*}$entryList $tbl
$sf preparescan
```

Auch hier wurde der Aufruf des Befehls `scrollutil::adaptBtn2EventHandling` auskommentiert und durch den wesentlich einfacheren und sichereren Aufruf des Unterbefehls `preparescan` "scrollableframe" ersetzt.

Wie bereits erwähnt, werden in der Datei `SuScrollableFrmContent.tcl` die Widgets "scrolled text", "listbox", "tablelist" und "ttk::treeview" innerhalb von scrollarea erstellt:

```
set _sa [scrollutil::scrollarea ...]
set txt [text $_sa.txt -font TkFixedFont -width 73]
scrollutil::addMouseWheelSupport $txt
$_sa setwidget $txt
grid $_sa ...

. . .

set _sa [scrollutil::scrollarea ...]
set lb [listbox $_sa.lb -width 0]
$_sa setwidget $lb
grid $_sa ...

. . .

set _sa [scrollutil::scrollarea ...]
set tbl [tablelist::tablelist $_sa.tbl ...]
. . .
$_sa setwidget $tbl
grid $_sa ...

. . .

set _sa [scrollutil::scrollarea ... -borderwidth 0]
set tv [ttk::treeview $_sa.tv ...]
. . .
$_sa setwidget $tv
grid $_sa ...
```

Bei den Widgets "text", "listbox" und "tablelist" verwenden wir "scrollarea"-Widgets mit ihren Standardeinstellungen `-borderwidth 1 -relief sunken`, was dazu führt, dass der Unterbefehl `setwidget` der zugehörigen Tcl-Befehle die Option `-borderwidth` der Text-, listbox- und Tablelist-Widgets auf `0` setzt. Für das ttk::treeview hingegen verwenden wir ein Scrollarea-Widget mit `-borderwidth 0`, da das ttk::treeview einen Rahmen mit der Breite `1` hat und die Konfigurationsoption `-borderwidth` nicht unterstützt.

Für unser Text-Widget bevorzugen wir eine Mausrad- und `<TouchpadScroll>` -Ereignisbehandlung, die das Widget zeilenweise statt pixelweise scrollt, wie es bei den `Text` -Klassenbindungen in Tk 8.5 und höher der Fall ist; Dies erreichen wir, indem wir den Pfadnamen `$txt` an den Befehl `scrollutil::addMouseWheelSupport` übergeben.

Das etwas ungewöhnliche Erscheinungsbild des Widgets ttk::spinbox (`$sb`), das die Anzahl der Änderungen in einer Tablelist-Version anzeigt, ist darauf zurückzuführen, dass es in der Datei `SuScrollableFrmContent.tcl` mit der Option `-style`-Option auf `Wide.TSpinbox` statt auf `TSpinbox` gesetzt wurde:

```
catch {package require wsb}

. . .

set sb [ttk::spinbox $cf.sb -style Wide.TSpinbox -from 0 -to 20 \
        -state readonly -width 4]
```

Das kleine **w**ide **s**pin**b**ox-Paket "wsb" ist im Lieferumfang der Scrollutil-Distribution enthalten, und der Versuch, es über `package require wsb` zu laden, ist erfolgreich, wenn die Tk-Version entweder mindestens 8.7 (mit integrierter SVG-Unterstützung) oder 8.6 ist und die Erweiterung "tksvg" in den Interpreter geladen werden kann. Wenn es geladen ist, erstellt das wsb-Paket automatisch das Layout `Wide.TSpinbox`. Andernfalls hat die Einstellung `-style Wide.TSpinbox` für das Widget ttk::spinbox keine Wirkung.

Der Hauptvorteil des Layouts `Wide.TSpinbox` gegenüber dem Stil `TSpinbox` liegt in der benutzerfreundlichen Größe und Anordnung der skalierungsabhängigen Auf- und Ab-Pfeile, was sich insbesondere bei der Nutzung von AndroWish auf einem Tablet oder Smartphone als nützlich erweist:

Die Datei `SuScrollableFrmContent.tcl` realisiert lediglich eine minimale Interaktion zwischen fünf der bereits erwähnten Widgets innerhalb des Inhaltsrahmens: Durch Auswahl einer Tablelist-Version aus der ttk::combobox `$cb` wird die ttk::spinbox `$sb` auf die entsprechende Anzahl von Änderungen gesetzt, der zu dieser Version gehörende Kommentar wird in die ttk::entry `$e` eingefügt, und die entsprechenden Einträge der Tablelist `$tbl` und ttk::treeview `$tv` werden ausgewählt und angezeigt.

Die Datei `SuScrollableFrmContent.tcl` enthält außerdem die Implementierung der Prozedur `configTablelist`, die der Schaltfläche "Configure Tablelist Widget" als Wert ihrer Option `-command` zugeordnet ist. Diese Prozedur öffnet ein Toplevel-Fenster, das ein scrollutil::scrollableframe-Widget enthält, das mit der Einstellung `-fitcontentwidth yes` innerhalb eines scrollarea erzeugt wurde. Anschließend füllt es den Inhaltsrahmen des "scrollableframe" mit den Widgets ttk::label, ttk::combobox, ttk::spinbox, ttk::entry und ttk::checkbutton oder Kippschalter, die zur Anzeige und Bearbeitung der Konfigurationsoptionen des Tablelist-Widgets dienen. Wenn das Widget "ttk::toggleswitch" verfügbar ist (es wurde in Tk 9.1a1 zum Kern hinzugefügt), verwenden wir dieses Widget anstelle von "Checkbuttons"; ansonsten verwenden wir tsw::toggleswitch-Widgets, wenn das Kippschalter-Widget-Paket "Tsw" in den Interpreter geladen werden kann. Die ttk::spinbox-Widgets werden mit der Einstellung `-style Wide.TSpinbox` erstellt.

Die Prozedur verarbeitet das virtuelle Ereignis `<<TraverseIn>>`, das an eines der oben genannten Widgets gesendet wird, mithilfe des Unterbefehls `see` des ScrollableFrame. Immer wenn ein ttk::combobox, ttk::spinbox oder ttk::entry erstellt wird, wird dafür der Befehl `scrollutil::adaptWheelEventHandling` aufgerufen, da diese Widgets über eigene Bindungen für das Mausrad und `<TouchpadScroll>`-Ereignisse verfügen. Ebenso übergeben wir die Widgets ttk::combobox, ttk::spinbox und ttk::entry an den Befehl `scrollutil::adaptBtn2EventHandling` weiter, da diese Widgets über integrierte Bindungen für Ereignisse der Maustaste 2 verfügen.

Die Widgets, die den Inhaltsrahmen füllen, werden über `grid` verwaltet. Bei den ttk::entry-Widgets rufen wir `grid` zusammen mit `-sticky we` auf. Aufgrund dessen und der

Einstellung `-fitcontentwidth yes` für den scrollbaren Rahmen passen sich die Widgets vom Typ ttk::entry an, indem sie sich vergrößern oder verkleinern, sobald sich die Breite des scrollbaren Rahmens infolge einer Größenänderung des Toplevel-Fensters ändert.

Ein Skript, das zwei "BWidget ScrollableFrame"-Widgets verwendet

Das Skript `BwScrollableFrmDemo2.tcl` im Verzeichnis `demos` erzeugt einen BWidget-ScrollableFrame, der in ein scrollarea-Widget eingebettet ist, und führt dann mit `source` das Skript `BwScrollableFrmContent.tcl` aus, das den Inhaltsrahmen des ScrollableFrame mit denselben Widgets füllt wie `SuScrollableFrmContent.tcl` im vorherigen Beispiel.

Hier ist der entsprechende Code:

```
package require Tk 8.5.9-                          ;# for ttk::spinbox
package require BWidget
Widget::theme yes
package require mentry_tile 3.2-                   ;# for mouse wheel
    support
package require tablelist_tile 6.5-                ;# for
    -(x|y)mousewheelwindow
                                                   ;# and
    scrollutil::scrollarea
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Scrollutil Demo"

#
# Create a ScrollableFrame within a scrollarea
#
set tf [ttk::frame .tf]
set sa [scrollutil::scrollarea $tf.sa]
set sf [ScrollableFrame $sa.sf]
$sa setwidget $sf

. . .

#
# Get the content frame and populate it
#
set cf [$sf getframe]
source [file join $dir BwScrollableFrmContent.tcl]
```

```
#
# Make the keyboard navigation more user-friendly
#
foreach w [list $cb $sb $e $me] {
    bind $w <<TraverseIn>> [list $sf see %W]
}
foreach w [list $txt $lb $tbl $tv] {
    bind $w <<TraverseIn>> [list seeScrollarea $sf %W]
}
proc seeScrollarea {sf w} { $sf see [scrollutil::getscrollarea $w] }
```

Der zusätzliche Code bezüglich des Mausrads und der `<TouchpadScroll>`-Ereignisse enthält dieselben Scrollutil-Befehlsaufrufe wie im vorherigen Beispiel, mit dem Unterschied, dass hier zusätzlich der ScrollableFrame für das Scrollen mit dem Mausrad und dem Touchpad registriert und der Befehl `scrollutil::prepareScrollingByWheel` aufgerufen wird:

```
. . .

#
# Create mouse wheel event bindings for the binding tag "all" and
# register the ScrollableFrame for scrolling by these bindings
#
scrollutil::createWheelEventBindings all
scrollutil::enableScrollingByWheel $sf

#
# Adapt the handling of the mouse wheel events for the
# text, listbox, ttk::combobox, ttk::spinbox, ttk::entry,
# tablelist, and ttk::treeview widgets, as well as for
# the entry components of the mentry widget of type "Date"
#
### scrollutil::adaptWheelEventHandling $txt $lb $cb $sb $e
    {*}$entryList \
    $tbl $tv
scrollutil::prepareScrollingByWheel $sf
```

Die Datei `BwScrollableFrmContent.tcl` enthält außerdem die Implementierung der Prozedur `configTablelist`, die der Schaltfläche "Configure Tablelist Widget" als Wert ihrer Option `-command` zugeordnet ist. Diese Prozedur öffnet ein Toplevel-Fenster, das ein BWidget-ScrollableFrame enthält, das mit der Einstellung `-constrainedwidth yes` innerhalb eines scrollarea-Widgets erstellt wurde, und ruft den Befehl `scrollutil::enableScrollingByWheel` für dieses ScrollableFrame auf, wodurch dieser für das Scrollen über die bereits erstellten Mausrad- und `<TouchpadScroll>`-Ereignis-Bindungen für das Bindungs-Tag `"all"` registriert wird. Anschließend füllt es den Inhaltsrahmen des ScrollableFrame mit den Widgets ttk::label, ttk::combobox, ttk::spinbox, ttk::entry und ttk::checkbutton oder Kippschalter, die zur Anzeige und Bearbeitung der Konfigurationsoptionen des Tablelist-Widgets dienen. Die ttk::spinbox-Widgets werden mit der Einstellung `-style`

`Wide.TSpinbox` erstellt.

Die Prozedur verarbeitet das virtuelle Ereignis `<<TraverseIn>>`, das an eines dieser Widgets gesendet wird, mithilfe des Unterbefehls `see` des ScrollableFrame. Immer wenn ein ttk::combobox, ttk::spinbox oder ttk::entry erstellt wird, wird dafür der Befehl `scrollutil::adaptWheelEventHandling` aufgerufen, da diese Widgets über eigene Bindungen für das Mausrad und `<TouchpadScroll>`-Ereignisse verfügen.

Auch hier ist das Ganze fast identisch mit dem, was wir im vorherigen Beispiel gemacht haben.

Ein Skript, das zwei iwidgets::scrolledframe-Widgets verwendet

Das Skript `ScrolledFrmDemo2.tcl` im Verzeichnis `demos` erstellt ein iwidgets::scrolledframe-Widget und führt anschließend `source` auf die Datei `ScrolledFrmContent.tcl`, wodurch der Inhaltsrahmen des "scrolledframe" mit denselben Widgets gefüllt wird wie `SuScrollableFrmContent.tcl` und `BwScrollableFrmContent.tcl` in den beiden vorherigen Beispielen.

Hier ist der entsprechende Code:

```
package require Tk 8.5.9-                       ;# for ttk::spinbox
if {[catch {package require iwidgets} result1] != 0 &&
    [catch {package require Iwidgets} result2] != 0} {
    error "$result1; $result2"
}
set dir [file dirname [info script]]
source [file join $dir scrolledwidgetPatch.itk] ;# adds ttk::scrollbar
    widgets
package require mentry_tile 3.2-                ;# for mouse wheel
    support
package require tablelist_tile 6.5-             ;# for
    -(x|y)mousewheelwindow
                                                ;# and
    scrollutil::scrollarea
package require scrollutil_tile
source [file join $dir styleUtil.tcl]

wm title . "Scrollutil Demo"

. . .

#
# Create a scrolledframe
#
set tf [ttk::frame .tf]
set sf [iwidgets::scrolledframe $tf.sf -borderwidth 1 -relief sunken \
        -scrollmargin 0]
. . .

#
```

```
# Get the content frame and populate it
#
set cf [$sf childsite]
. . .
source [file join $dir ScrolledFrmContent.tcl]
```

Die zusätzlichen Angaben zum Mausrad und zu den `<TouchpadScroll>`-Ereignissen enthalten dieselben Scrollutil-Befehlsaufrufe wie im vorherigen Beispiel.

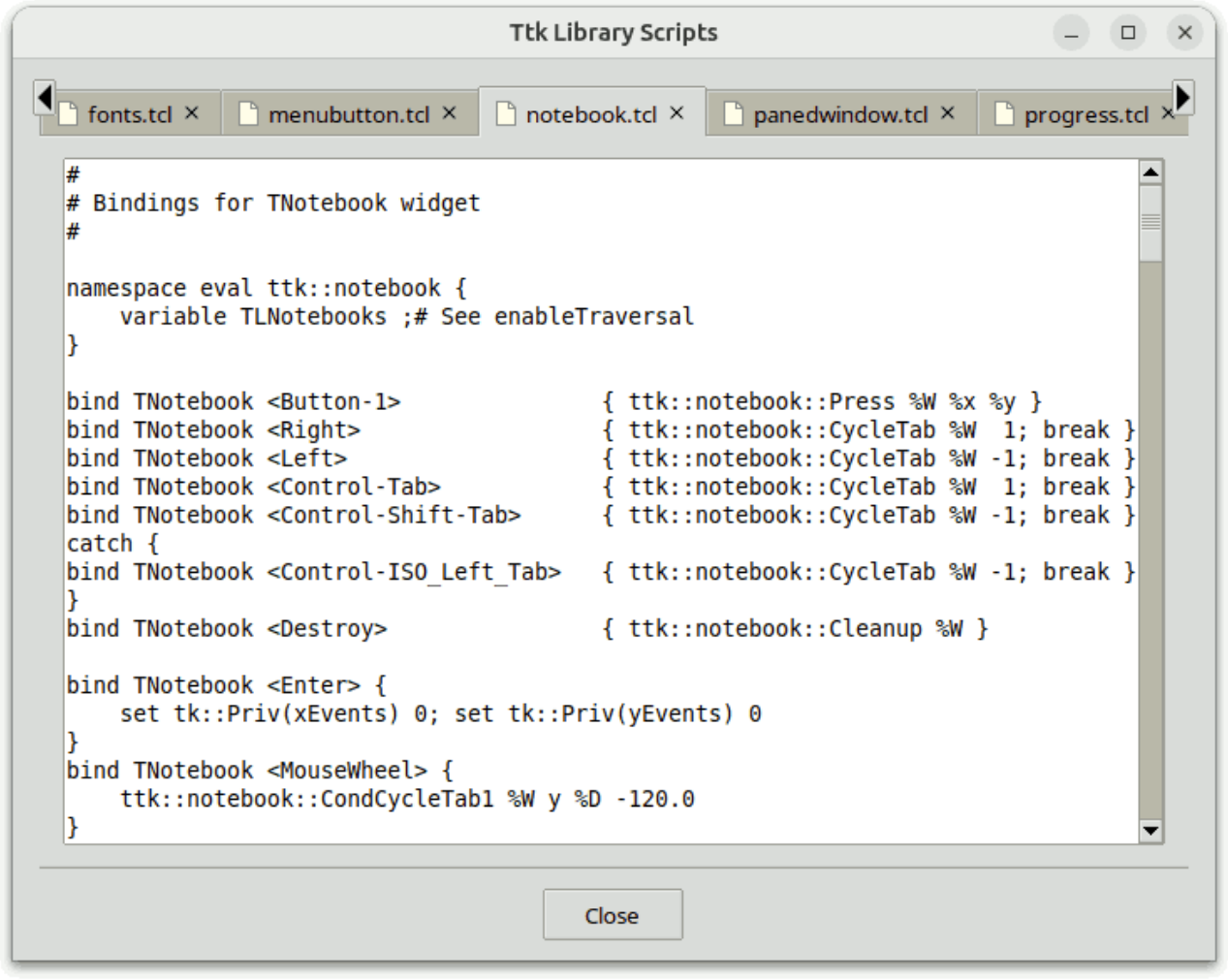
Die Datei `ScrolledFrmContent.tcl` enthält außerdem die Implementierung der Prozedur `configTablelist`, die der Schaltfläche "Configure Tablelist Widget" als Wert ihrer Option `-command` zugeordnet ist. Diese Prozedur öffnet ein Toplevel-Fenster, das ein iwidgets::scrolledframe-Widget mit einer manuell implementierten Entsprechung der `-fitcontentwidth yes` enthält scrollutil::scrollableframe und `-constrainedwidth yes` BWidget-ScrollableFrame-Einstellungen und ruft den Befehl `scrollutil::enableScrollingByWheel` für diesen ScrollableFrame auf, wodurch dieser für das Scrollen über die bereits erstellten Mausrad- und `<TouchpadScroll>`-Ereignis-Bindungen für das Bindungs-Tag `"all"` registriert wird. Danach füllt es den Inhaltsrahmen des ScrollableFrame mit den Widgets ttk::label, ttk::combobox, ttk::spinbox, ttk::entry und ttk::checkbutton oder Kippschalter, die zur Anzeige und Bearbeitung der Konfigurationsoptionen des Tablelist-Widgets dienen. Die ttk::spinbox-Widgets werden mit der Einstellung `-style Wide.TSpinbox` erstellt.

Immer wenn ein ttk::combobox, ttk::spinbox, oder ttk::entry erstellt wird, wird dafür der Befehl `scrollutil::adaptWheelEventHandling` aufgerufen, da diese Widgets über eigene Bindungen für das Mausrad und `<TouchpadScroll>`-Ereignisse verfügen.

Auch hier ist das Ganze fast identisch mit dem, was wir in den beiden vorherigen Beispielen gemacht haben.

Eine Demo von scrollutil::scrollednotebook

Das Skript `ScrolledNotebookDemo.tcl` im Verzeichnis `demos` erstellt ein scrollutil::scrollednotebook-Widget und füllt es mit Teilflächen, die scrollareas, die Text-Widgets umschließen, welche den Inhalt der Ttk-Bibliothek anzeigen. Anschließend stellt es Popup-Menüeinträge zum Verschieben nach links/rechts und zum Schließen der Teilflächen bereit und legt die Höhe und Breite des "scrollednotebook" auf Werte fest, die aus der angeforderten Höhe und Breite der "scrollarea"-Widgets sowie dem auf die Teilflächen angewendeten Innenabstand berechnet werden.



ScrolledNotebookDemo

Hier ist der entsprechende Code:

```
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Ttk Library Scripts"

scrollutil::addclosetab My.TNotebook

#
# Create an image corresponding to the display's DPI scaling level
#
if {$tk_version >= 8.7 || [catch {package require tksvg}] == 0} {
    set fmt $scrollutil::svgfmt
    image create photo fileImg -file [file join $dir file.svg] -format
    $fmt
} else {
    set pct $scrollutil::scalingpct
```

```
    image create photo fileImg -file [file join $dir file$pct.gif]
    -format gif
}

#
# Create a scrollednotebook widget having closable (and, per default,
# movable) tabs and populate it with panes that contain scrolled
# text widgets displaying the contents of the Ttk library files
#
set f  [ttk::frame .f]
set nb [scrollutil::scrollednotebook $f.nb -style My.TNotebook \
        -forgetcommand condCopySel -leavecommand saveSel]
set currentTheme [styleutil::getCurrentTheme]
set panePadding [expr {$currentTheme eq "aqua" ? 0 : "9p"}]
cd [expr {[info exists ttk::library] ? $ttk::library : $tile::library}]
foreach fileName [lsort [glob *.tcl]] {
    set baseName [string range $fileName 0 end-4]
    set sa [scrollutil::scrollarea $nb.sa_$baseName -lockinterval 10]
    $sa configure -yscrollbarmode static        ;# temporarily, for
    adjustsize
    if {$currentTheme eq "vista"} {
        $sa configure -relief solid
    }
    set txt [text $sa.txt -font TkFixedFont -takefocus 1 -wrap none]
    catch {$txt configure -tabstyle wordprocessor}      ;# for Tk 8.5
    and later
    scrollutil::addMouseWheelSupport $txt       ;# old-school wheel
    support
    $sa setwidget $txt

    set chan [open $fileName]
    $txt insert end [read -nonewline $chan]
    close $chan
    $txt configure -state disabled
    bind $txt <Button-1> { focus %W }   ;# for Tk versions <
    8.6.11/8.7a4

    $nb add $sa -text $fileName -image fileImg -compound left \
                -padding $panePadding
}

proc condCopySel {nb widget} {
    set txt $widget.txt
    if {[llength [$txt tag nextrange sel 1.0 end]] == 0} {
        return 1
    }
```

```
    set btn [tk_messageBox -title "Copy Selection?" -icon question \
             -message "Do you want to copy the selection to the
    clipboard?" \
             -type yesnocancel]
    switch $btn {
        yes     { tk_textCopy $txt; return 1 }
        no      { return 1 }
        cancel  { return 0 }
    }
}

proc saveSel {nb widget} {
    set selRange [$widget.txt tag nextrange sel 1.0 end]
    if {[llength $selRange] == 0} {
        $nb unsettabattrib $widget "selRange"
    } else {
        $nb tabattrib $widget "selRange" $selRange
    }

    return 1
}

#
# Create bindings for moving and closing the tabs interactively,
# as well as for the virtual event <<NotebookTabChanged>>
#
bind $nb <<MenuItemsRequested>> { populateMenu %W %d }
bind $nb <<NotebookTabChanged>> { restoreSel %W }

proc populateMenu {nb data} {
    foreach {menu tabIdx} $data {}
    set tabCount [$nb index end]
    set prevIdx [expr {($tabIdx - 1) % $tabCount}]
    set nextIdx [expr {($tabIdx + 1) % $tabCount}]
    set widget [lindex [$nb tabs] $tabIdx]

    $menu add command -label "Move Tab Left"  -command \
        [list $nb insert $prevIdx $widget]
    $menu add command -label "Move Tab Right" -command \
        [list $nb insert $nextIdx $widget]
    $menu add separator
    $menu add command -label "Close Tab" -command \
        [list $nb forget $tabIdx]
}
```

```
proc restoreSel nb {
    set widget [$nb select]
    if {$widget ne "" && [$nb hastabattrib $widget "selRange"]} {
        set txt $widget.txt
        $txt tag remove sel 1.0 end
        $txt tag add sel {*}[$nb tabattrib $widget "selRange"]
    }
}

. . .

#
# Set the scrollednotebook's -height and -width options to the
# maximum requested height and width of all panes, respectively
#
after 50 [list resizeNb $nb]

proc resizeNb nb {
    $nb adjustsize
    foreach sa [$nb tabs] {
        $sa configure -yscrollbarmode dynamic                   ;# the
    default
    }
}
```

Wir fügen das Element `closetab` mithilfe des Befehls `scrollutil::addclosetab` zu den Registerkarten des Stils `My.TNotebook` hinzu und erstellen ein "scrollednotebook"-Widget dieses Stils. HINWEIS: In diesem Beispiel hätten wir zwar stattdessen den Standardstil ttk::notebook `TNotebook` verwenden können, ist die Verwendung verschiedener ttk::notebook-Stile in Anwendungen erforderlich, die sowohl Notebooks mit dem Element `closetab` in ihren Registerkarten als auch Notebooks ohne dieses Element enthalten.

Wenn die Tk-Version mindestens 8.7 beträgt oder das Paket "tksvg" in den Interpreter geladen werden kann, wird das Bild `fileImg` aus der Datei `file.svg` erstellt, unter Verwendung der öffentlichen Variablen `scrollutil::svgfmt`. Andernfalls wird es aus Dateien erstellt, deren Namen den DPI-Skalierungsprozentsatz des Bildschirms enthalten, der durch die öffentliche Variable `scrollutil::scalingpct` angegeben wird. Die Datei `file100.gif` enthält ein Bild der Größe 16 × 16, die Datei `file125.gif` enthält ein Bild der Größe 20 × 20, die Datei `file150.gif` enthält ein Bild mit der Größe 24 × 24 und so weiter. Im ersten Fall wird eine einzige `file.svg`-Datei benötigt, und das Bild wird automatisch skaliert, entsprechend dem tatsächlichen Skalierungsprozentsatz des Displays, der größer sein kann als der Maximalwert `200` der Variablen `scrollutil::scalingpct`. Dieser Vorteil ist besonders nützlich, wenn AndroWish auf einem Tablet oder Smartphone ausgeführt wird. Im zweiten Fall werden insgesamt `5` `file*.gif`- (oder `file*.png`-)Dateien benötigt, um sicherzustellen, dass die Größe des Bildes dem Wert der Variablen `scrollutil::scalingpct` entspricht.

Wir setzen die Option `-yscrollbarmode` der Scrollarea-Widgets, die wir mit `add` dem Scrollednotebook zugewiesen haben, auf `static`, um sicherzustellen, dass die vertikalen

Bildlaufleisten sichtbar sind, wenn der Unterbefehl `adjustsize` des Widgets aufgerufen wird, um dessen Abmessungen festzulegen. Nach dem Aufruf dieses Unterbefehls (gegen Ende des Skripts) setzen wir den Standardwert `dynamic` wieder her.

Beachten Sie, dass der Status der Text-Widgets auf `disabled` gesetzt ist, der Benutzer jedoch einen Textbereich auswählen und die Auswahl dann über `Control-c` in die Zwischenablage kopieren kann (`Command-c` unter Mac OS X/11+). Beim Versuch, eine Registerkarte zu schließen, wird der Pfadname des entsprechenden Scrollarea-Widgets automatisch an die Prozedur `condCopySel` übergeben, die als Wert der Option `-forgetcommand` festgelegt wurde. Diese Prozedur überprüft das Text-Widget, das dem Scrollarea-Widget untergeordnet ist, auf das Vorhandensein einer Auswahl und ermöglicht es dem Benutzer, sicherzustellen, dass die Auswahl vor dem Schließen der Registerkarte in die Zwischenablage kopiert wird, oder den beabsichtigten Vorgang abzubrechen. In einer realen Anwendung würde die als Wert der oben genannten Option angegebene Prozedur den Benutzer typischerweise fragen, ob er die Änderungen vor dem Schließen der Registerkarte speichern oder den beabsichtigten Vorgang abbrechen möchte.

Beim Versuch, das aktuell ausgewählte Fenster durch Auswahl eines anderen zu verlassen, wird der Pfadname des Scrollarea-Widgets automatisch an die Prozedur `saveSel` übergeben, die als Wert der Option `-leavecommand` angegeben wurde. Diese Prozedur speichert die aus zwei Elementen bestehende Liste der Textindizes, die die Auswahl im Text-Widget-Kind des "scrollarea" begrenzen, mithilfe des Unterbefehls `tabattrib` als Wert des Tab-Attributs `"selRange"` oder setzt dieses Tab-Attribut über `unsettabattrib` zurück, wenn das Text-Widget keine Auswahl enthält. Wird diese Registerkarte später erneut ausgewählt, überprüft die Prozedur `restoreSel`, die an das virtuelle Ereignis `<<NotebookTabChanged>>` gebunden ist, überprüft mithilfe des Unterbefehls `hastabattrib` das Vorhandensein des Tab-Attributs `"selRange"` und stellt die durch den Wert dieses Tab-Attributs festgelegte Auswahl im Text-Widget wieder her, der mithilfe des Unterbefehls `tabattrib` abgerufen wurde. Wir haben diesen Ansatz zum Speichern und Wiederherstellen der Textauswahl gewählt, anstatt die Text-Widgets mit `-exportselection 0` zu erstellen, um die Verwendung der Unterbefehle im Zusammenhang mit Registerkartenattributen zu veranschaulichen.

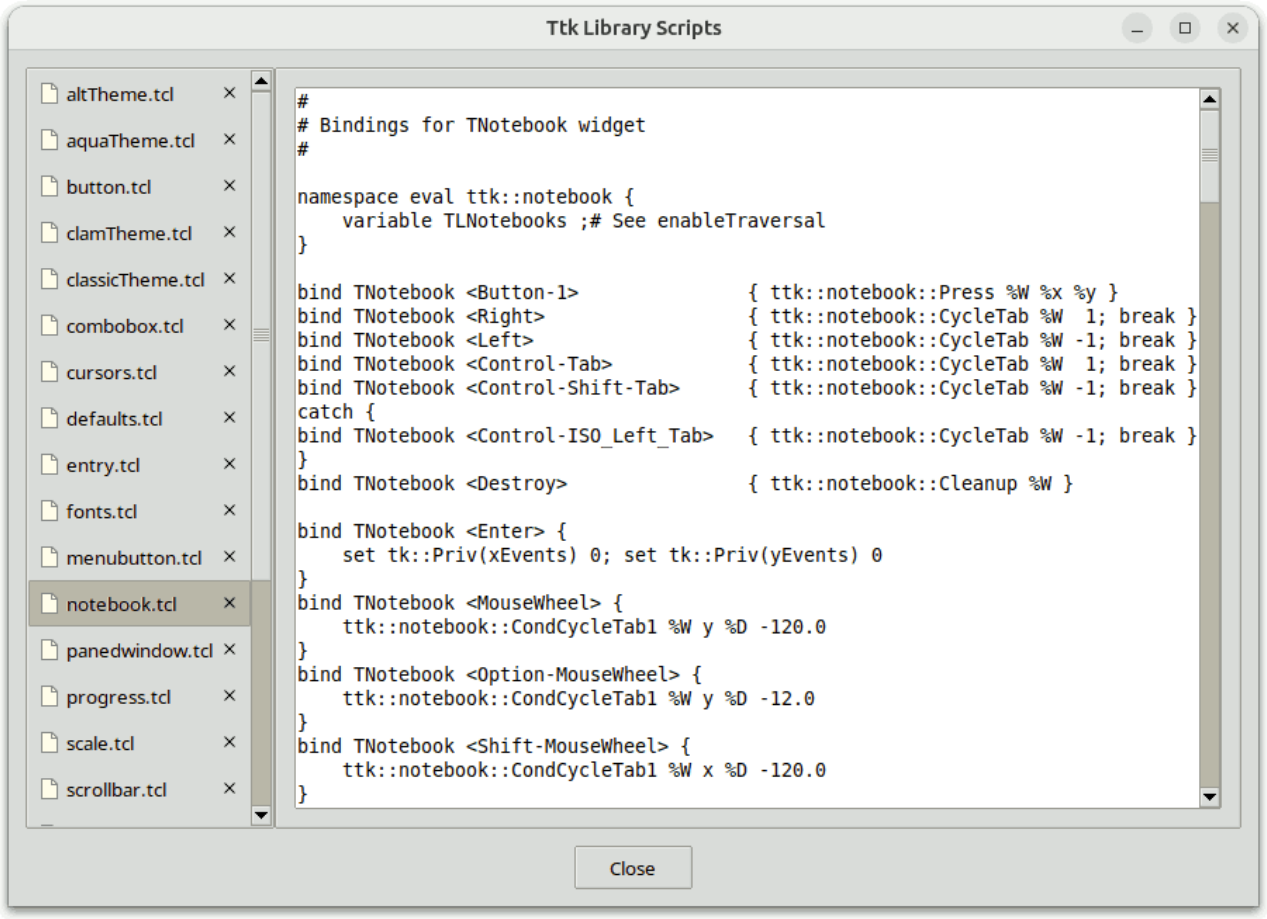
Außerdem nutzen wir die VIRTUELLEN EREIGNISSE `<<MenuItemsRequested>>`, um das Popup-Menü zu füllen, das angezeigt wird, wenn der Benutzer mit der dritten Maustaste auf eine Registerkarte klickt.

Beim Ausführen des Skripts wird das "scrollednotebook"-Widget mit der konfigurierten Breite angezeigt, wobei die Registerkarten die Dateinamen ohne Vorangestellte Zeichen anzeigen. Zum Vergleich: Das sehr ähnliche Skript `TtkNotebookDemo.tcl` im Verzeichnis `demos` erstellt ein "ttk::notebook"-Widget und füllt es auf dieselbe Weise wie das oben beschriebene Skript `ScrolledNotebookDemo.tcl`. Obwohl das Skript auch die Option `-width` des Notebooks auf einen Wert setzt, der der gewünschten Breite der "scrollarea"-Widgets und dem auf die Teilflächen angewendeten Innenabstand entspricht, erscheint das ttk::notebook mit einer exorbitant großen Breite und (auf den meisten Bildschirmen) mit zusammengedrückten Registerkarten. Wenn Sie die Breite auf ein vernünftiges Maß anpasst, werden die Registerkarten so klein, dass ihre Texte nicht mehr lesbar sind.

Eine Demo von scrollutil::plainnotebook

Das Skript `PlainNotebookDemo.tcl` im Verzeichnis `demos` erstellt ein

scrollutil::plainnotebook-Widget und füllt es mit Teilflächen, die scrollareas, die Text-Widgets umschließen, welche den Inhalt der Ttk-Bibliothek anzeigen. Anschließend stellt es Popup-Menüeinträge zum Verschieben der Registerkarten nach oben/unten und zum Schließen derselben bereit.



PlainNotebookDemo

Der entsprechende Code ist nahezu identisch mit dem im vorherigen Beispiels, mit dem Unterschied, dass wir hier die Registerkarten mithilfe der Option "plainnotebook" von `-closabletabs` schließbar machen und die Breite des Widgets automatisch so angepasst wird, dass sie der Breite seiner Teilflächen entspricht:

```
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Ttk Library Scripts"

#
# Create an image corresponding to the display's DPI scaling level
#
. . .

#
```

```
# Create a plainnotebook widget having closable (and, per default,
# movable) tabs and populate it with panes that contain scrolled
# text widgets displaying the contents of the Ttk library files
#
set f  [ttk::frame .f]
set nb [scrollutil::plainnotebook $f.nb -closabletabs 1 \
        -forgetcommand condCopySel -leavecommand saveSel]
. . .

proc condCopySel {nb widget} {
    . . .
}

proc saveSel {nb widget} {
    . . .
}

#
# Create bindings for moving and closing the tabs interactively,
# as well as for the virtual event <<NotebookTabChanged>>
#
bind $nb <<MenuItemsRequested>> { populateMenu %W %d }
bind $nb <<NotebookTabChanged>> { restoreSel %W }

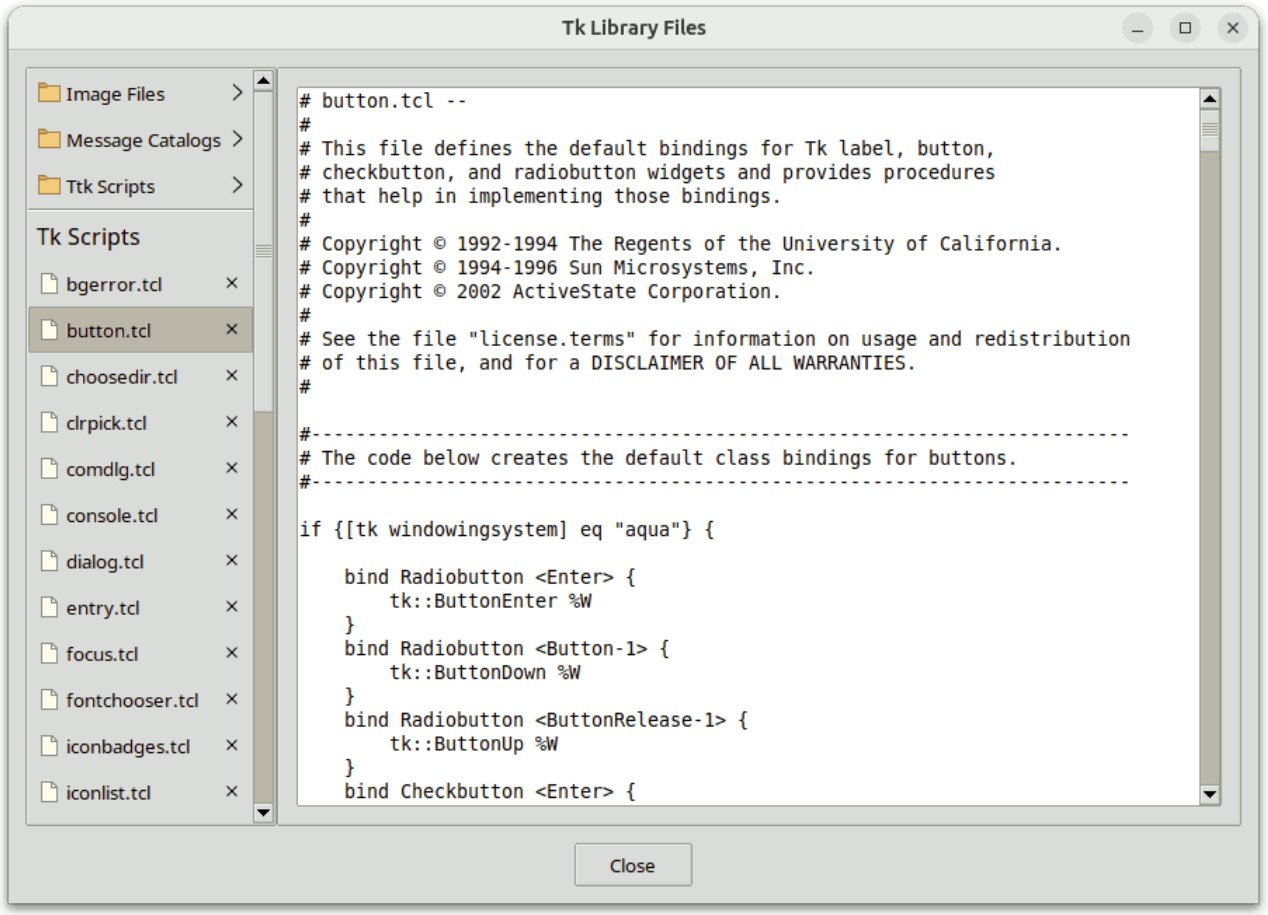
proc populateMenu {nb data} {
    . . .
}

proc restoreSel nb {
    . . .
}

. . .
```

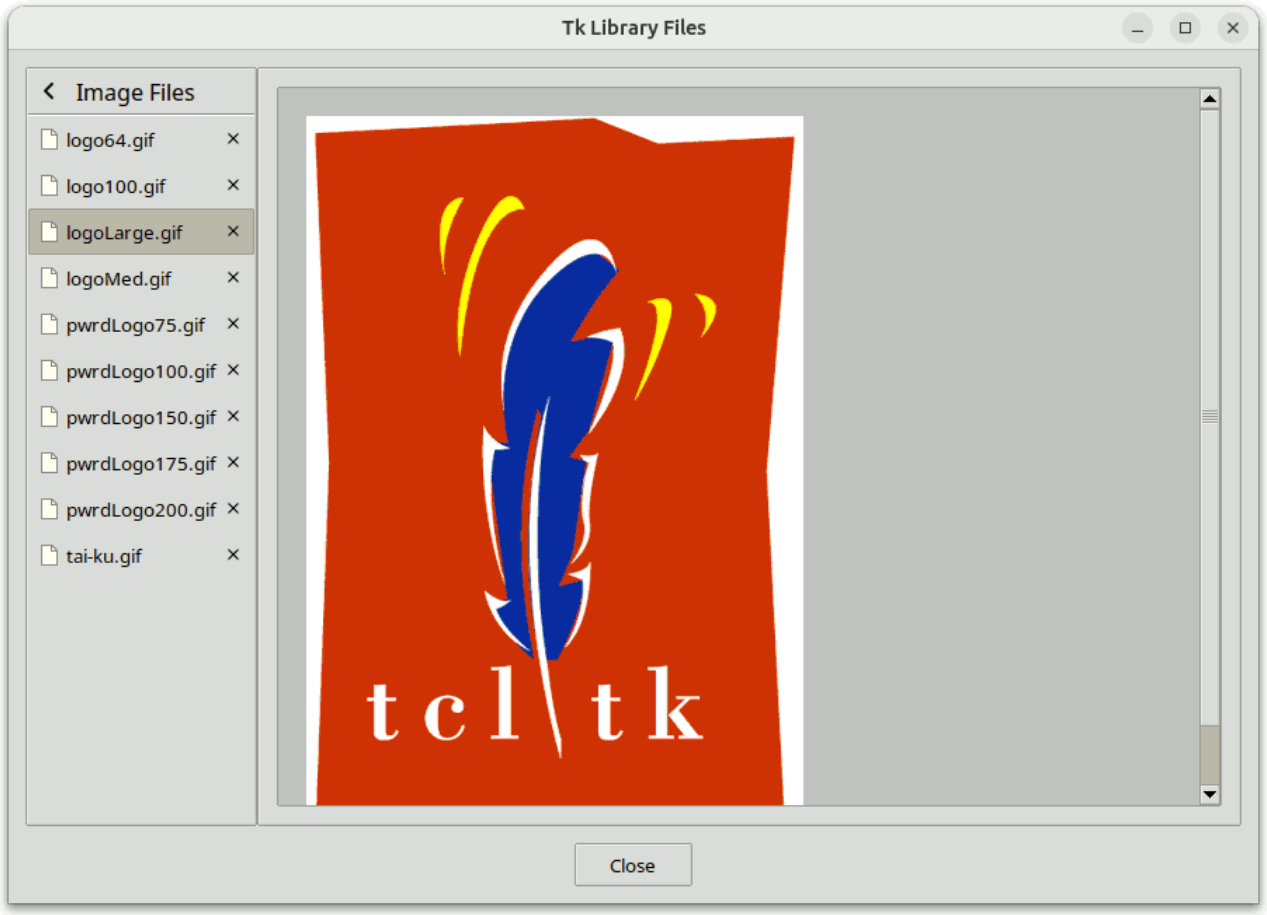
Eine Demo von scrollutil::pagesman

Das Skript `PagesManDemo.tcl` im Verzeichnis `demos` veranschaulicht die Verwendung des Widgets scrollutil::pagesman, das Widgets vom Typ scrollutil::plainnotebook als Seiten enthält. Es erstellt ein "pagesman"-Widget und fügt diesem vier "plainnotebook"-Unterelemente als Seiten hinzu. Die Teilflächen der ersten Seite zeigen den Inhalt der Tk-Bibliotheken an, die sich im Verzeichnis `$tk_library` befinden. Die Teilflächen der drei anderen Seiten zeigen den Inhalt der GIF-Bilddateien, der Nachrichtenkataloge und der Ttk-Skripte an, die sich in den Unterverzeichnissen `images`, `msgs` und (für Tk 8.5a5 und höher) `ttk` befinden.



PagesManDemo

Der Benutzer kann mithilfe der drei Schaltflächen, die ein Ordnersymbol und das Stilelement `descend` anzeigen, von der ersten Seite zu den drei anderen Seiten wechseln. Um zurückzukehren, muss er oder sie die Schaltfläche `ascend` verwenden, die in der oberen linken Ecke des jeweiligen "plainnotebook"-Widgets angezeigt wird.



PagesManDemoImages

Hier ist der entsprechende Code:

```
package require scrollutil_tile
set dir [file dirname [info script]]
source [file join $dir styleUtil.tcl]

wm title . "Tk Library Files"
wm withdraw .

#
# Create two images corresponding to the display's DPI scaling level
#
if {$tk_version >= 8.7 || [catch {package require tksvg}] == 0} {
    set fmt $scrollutil::svgfmt
    image create photo fileImg   -file [file join $dir file.svg]
    -format $fmt
    image create photo folderImg -file [file join $dir folder.svg]
    -format $fmt
} else {
    set pct $scrollutil::scalingpct
    image create photo fileImg   -file [file join $dir file$pct.gif]
    \
```

```
        -format gif
    image create photo folderImg -file [file join $dir folder$pct.gif]
    \
        -format gif
}

#
# Populates a given plainnotebook widget with panes that display the
    contents
# of the files of the specified suffix within the current working
    directory
#
proc populateNotebook {nb sfx} {
    set currentTheme [styleutil::getCurrentTheme]
    set panePadding [expr {$currentTheme eq "aqua" ? 0 : "9p"}]
    foreach fileName [lsort -dictionary [glob *.$sfx]] {
        set baseName [string range $fileName 0 end-4]
        set sa [scrollutil::scrollarea $nb.sa_$baseName]
        $sa configure -xscrollbarmode none -yscrollbarmode static ;#
    temporarily
        if {$sfx eq "gif"} {
            set canv [canvas $sa.canv -background #c0c0c0]
            set img [image create photo -file $fileName -format gif]
            $canv create image 15p 15p -anchor nw -image $img
            bind $canv <Configure> [list setScrollRegion %W %w %h $img]
            scrollutil::addMouseWheelSupport $canv
            $sa setwidget $canv
        } else {
            $sa configure -lockinterval 10
            . . .
        }
        $nb add $sa -text $fileName -image fileImg -compound left \
                    -padding $panePadding
    }
}

#
# Create a pagesman widget
#
set f [ttk::frame .f]
set pm [scrollutil::pagesman $f.pm -leavecommand pmLeaveCmd]

#
# Add option database entries for the -closabletabs,
# -forgetcommand, and -leavecommand plainnotebook options
#
```

```
option add *Plainnotebook.closableTabs  1
option add *Plainnotebook.forgetCommand condCopySel
option add *Plainnotebook.leaveCommand  saveSel

#
# Create a plainnotebook child displaying the contents of the Tk
    library files
#
set nbTk [scrollutil::plainnotebook $pm.nbTk]
$pm add $nbTk
$nbTk addbutton 1 "Image Files"      folderImg
$nbTk addbutton 2 "Message Catalogs" folderImg
$nbTk addbutton 3 "Ttk Scripts"      folderImg
$nbTk addseparator
$nbTk addlabel "Tk Scripts"
cd $tk_library
populateNotebook $nbTk "tcl"

#
# Create a plainnotebook child displaying the images for the Tcl
    (Powered) Logo
#
set nbImgs [scrollutil::plainnotebook $pm.nbImgs -caller 0 -title
    "Image Files"]
$pm add $nbImgs
cd $tk_library/images
populateNotebook $nbImgs "gif"

#
# Create a plainnotebook child displaying the contents of the message
    catalogs
#
set nbMsgs [scrollutil::plainnotebook $pm.nbMsgs -caller 0 -title \
            "Message\nCatalogs"]
$pm add $nbMsgs
cd $tk_library/msgs
populateNotebook $nbMsgs "msg"

#
# Create a plainnotebook child displaying the contents of the Ttk
    library files
#
set nbTtk [scrollutil::plainnotebook $pm.nbTtk -caller 0 -title "Ttk
    Scripts"]
$pm add $nbTtk
### cd $tk_library/ttk          ;# works for Tk versions 8.5a5 and
```

```
    later only
cd [expr {[info exists ttk::library] ? $ttk::library : $tile::library}]
populateNotebook $nbTtk "tcl"

proc setScrollRegion {canv canvWidth canvHeight img} {
    #
    # Use a margin of 15p around the image
    #
    set pixels [expr {30 * [tk scaling]}]
    set rightX [expr {[image width  $img] + $pixels}]
    set lowerY [expr {[image height $img] + $pixels}]
    if {$rightX < $canvWidth}  { set rightX $canvWidth }
    if {$lowerY < $canvHeight} { set lowerY $canvHeight }
    $canv configure -scrollregion [list 0 0 $rightX $lowerY]
}

proc pmLeaveCmd {pm nb} {
    set widget [$nb select]
    if {$widget eq ""} {
        return 1
    } else {
        return [saveSel $nb $widget]
    }
}

proc condCopySel {nb widget} {
    global nbImgs
    if {$nb eq $nbImgs || [winfo class $widget] ne "Scrollarea"} {
        return 1
    }

    . . .
}

proc saveSel {nb widget} {
    global nbImgs
    if {$nb eq $nbImgs || [winfo class $widget] ne "Scrollarea"} {
        return 1
    }

    . . .
}


#
# For each plainnotebook create bindings for moving and closing its
```

```
    tabs
# interactively, as well as for the virtual event
    <<NotebookTabChanged>>
#
foreach nb [$pm pages] {
    bind $nb <<MenuItemsRequested>> { populateMenu %W %d }
    bind $nb <<NotebookTabChanged>> { restoreSel %W }
}

proc populateMenu {nb data} {
    . . .
}

proc restoreSel nb {
    . . .
}

. . .

wm deiconify .
after 400 [list configScrollareas $pm]

proc configScrollareas pm {
    wm geometry . [wm geometry .]
    foreach nb [$pm pages] {
        foreach w [$nb tabs] {
            if {[winfo class $w] eq "Scrollarea"} {
                $w configure -xscrollbarmode dynamic -yscrollbarmode
    dynamic
            }
        }
    }
}
```

Die vier Seiten des "pagesman"-Widgets, die mit dessen Unterbefehl `add` erstellt wurden, erhalten die numerischen Indizes `0`, ..., `3`. Wir übergeben die Seitenindizes `1`, `2` und `3` an den Unterbefehl `addbutton` des ersten "plainnotebook"-Widgets weiter, mit dem wir die drei Werkzeugschaltflächen erstellen, über die Sie von dieser Seite zu den anderen wechseln kann. Wenn wir die drei anderen "plainnotebook"-Widgets erzeugen, setzen wir deren Option `-caller` auf `0`. Dadurch wird deren Schaltfläche "Nach oben" sichtbar gemacht und sichergestellt, dass durch das Anklicken dieser Schaltfläche wieder zum ersten "plainnotebook" zurückgekehrt wird.

Die Teilflächen aller vier "plainnotebook"-Widgets sind scrollareas, die jeweils Text- und Canvas-Widgets umschließen. Indem wir die Scrollarea-Widgets vorübergehend mit `-xscrollbarmode none -yscrollbarmode static` konfigurieren, schaffen wir Platz für bis zu 80 Zeichen in bis zu 30 Textzeilen sowie für die vertikale Bildlaufleiste. 400 Millisekunden

nach dem Mappen des Hauptfensters rufen wir die Prozedur `configScrollareas` auf, die die Geometrie des Anwendungsfensters einfriert und die oben genannten Scrollarea-Optionen auf ihren Standardwert `dynamic` zurücksetzt. Aufgrund der eingefrorenen Geometrie vergrößern die horizontalen Bildlaufleisten, wenn sie angezeigt werden, das Fenster nicht.

Stichwörter: scrollutil

wheelEvent

Befehle im Zusammenhang mit dem Mausrad und der Ereignisbehandlung von `<TouchpadScroll>`

Für Scrollutil Version 2.11

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

- Der Befehl scrollutil::addMouseWheelSupport
- Der Befehl scrollutil::createWheelEventBindings
- Der Befehl scrollutil::enableScrollingByWheel
- Der Befehl scrollutil::disableScrollingByWheel
- Der Befehl scrollutil::adaptWheelEventHandling
- Der Befehl scrollutil::prepareScrollingByWheel
- Der Befehl scrollutil::setFocusCheckWindow
- Der Befehl scrollutil::focusCheckWindow

Der Befehl `scrollutil::addMouseWheelSupport`

NAME

`scrollutil::addMouseWheelSupport` – Unterstützung für Mausrad-Ereignisse bei (ttk::)menubutton, (ttk::)scale oder scrollbaren Widgets hinzufügen

ÜBERSICHT

```
scrollutil::addMouseWheelSupport tag ?axes?
```

ERFORDERLICHE TK-VERSION

8.4 oder höher.

BESCHREIBUNG

Fügt den Widgets mit dem angegebenen Bindungs-Tag Unterstützung für Mausrad- und `<TouchpadScroll>`-Ereignisse hinzu, indem Bindungen für die Mausrad- und `<TouchpadScroll>`-Ereignisse entlang der Achsen erstellt werden, die durch das optionale Argument `axes` angegeben werden, das entweder `xy` (Standardwert, d. h. sowohl die x- als auch die y-Achse), `x` (d. h. nur die x-Achse) oder `y` (d. h. nur die y-Achse) lauten muss.

- Wenn `tag` der Pfadname einer Menüschaltfläche oder eines Widgets mit der Bezeichnung

"ttk::menubutton" ist oder der Name der Klasse `Menubutton` oder `TMenubutton`, dann wird `axes` ignoriert, und die durch diesen Befehl erstellten Bindungsskripte aktivieren den zyklisch nächsten oder vorherigen Eintrag des Menüs, das dem durch das Ereignisfeld `%W` angegebenen Widget zugeordnet ist, und rufen die Aktion dieses Menüeintrags auf, vorausgesetzt, dessen Typ ist nicht `cascade` oder `tearoff`.

- Wenn `tag` der Pfadname eines Skalen- oder "ttk::scale"-Widgets oder der Name der Klasse `Scale` oder `TScale` ist, dann wird `axes` ignoriert, und die durch diesen Befehl erstellten Bindungsskripte erhöhen oder verringern den Wert des Widgets, das durch das Ereignisfeld `%W` angegeben wird, und verschieben dessen Schieberegler entsprechend.
- Andernfalls scrollen die Bindungsskripte das im Ereignisfeld `%W` angegebene Fenster mithilfe der Unterbefehle `xview scroll number units` und/oder `yview scroll number units` des zugehörigen Tcl-Befehls, abhängig von `axes`.

ANMERKUNG 1: Wenn `tag` der Pfadname eines Fensters ist, werden die durch diesen Befehl erstellten Bindungsskripte durch einen Aufruf des Befehls `break` beendet, um zu verhindern, dass Mausrad- oder `<TouchpadScroll>`-Ereignisse von weiteren Bindungsskripten verarbeitet werden. Beispielsweise wenn `tag` der Pfadname eines Text-Widgets ist, stellt der abschließende Befehl `break` sicher, dass Mausrad- oder `<TouchpadScroll>`-Ereignisse nicht *zusätzlich* von den Klassenbindungen (die mit dem Bindungs-Tag `Text` verknüpft sind), die in Tk 8.5 und höher ein Scrollen in *Pixeln* auslösen – im Gegensatz zu den durch diesen Befehl erstellten Bindungen, die das Widget in *Einheiten* (d. h. Zeilen und Zeichen) scrollen.

ANMERKUNG 2: Das Canvas-Widget verfügt über keine integrierten Bindungen, aber Sie können diesen Befehl verwenden, um die Unterstützung für Mausrad- und `<TouchpadScroll>`-Ereignisse zur Widget-Klasse `Canvas` oder zu einzelnen Canvas-Widgets hinzuzufügen.

ANMERKUNG 3: *In Tk-Versionen vor 8.7a4* hängt der Umfang der Ereignisse des Mausrads vom Fenstersystem ab:

- Die Ereignisse des Mausrads entlang der vertikalen Achse sind unter Windows `<MouseWheel>`, unter Mac OS X/11+ `<MouseWheel>` und `<Option-MouseWheel>` sowie unter X11 `<MouseWheel>`, `<Button-4>` und `<Button-5>` unter X11 (wobei `<MouseWheel>` nicht vom X-Server ausgelöst wird, sondern mithilfe von `event generate` erzeugt werden kann).
- Die Ereignisse des Mausrads entlang der horizontalen Achse sind `<Shift-MouseWheel>` unter Windows, `<Shift-MouseWheel>` und `<Shift-Option-MouseWheel>` unter Mac OS X/11+ sowie `<Shift-MouseWheel>`, `<Shift-Button-4>` und `<Shift-Button-5>` unter X11 (wobei `<Shift-MouseWheel>` nicht vom X-Server ausgelöst wird, sondern mithilfe von `event generate` erzeugt werden kann). Unter X11 treten bei Verwendung von Tk 8.7a3 zwei weitere Mausrad-Ereignisse entlang der horizontalen Achse auf: `<Button-6>` und `<Button-7>`, die jeweils genauso behandelt werden wie `<Shift-Button-4>` bzw. `<Shift-Button-5>`. Diese Ereignisse werden üblicherweise durch das Neigen des Scrollrads nach links oder rechts bei einer Maus mit einer oder zwei zusätzlichen (Daumen-)Tasten ausgelöst. (In Tk-Versionen 8.6.x, wobei x >= 10 ist, löst das Neigen des Scrollrads einer solchen Maus nach links oder rechts unter Windows und Mac OS X/11+ die Ereignisse `<Shift-MouseWheel>` aus und unter X11 die Ereignisse `<Shift-Button-4>` und `<Shift-Button-5>`.)

ANMERKUNG 4: *In den Tk-Versionen 8.7a4 und höher* ist der Satz der Mausrad-Ereignisse auf allen Fenstersystemen identisch:

- Die Ereignisse des Mausrads entlang der vertikalen Achse sind `<MouseWheel>` und `<Option-MouseWheel>`, wobei der Modifikator `Option` unter Mac OS X/11+ an die Taste `Option` und unter Windows sowie X11 an die Taste `Alt` gebunden ist.
- Die Ereignisse des Mausrads entlang der horizontalen Achse sind `<Shift-MouseWheel>` und `<Shift-Option-MouseWheel>`, wobei der Modifikator `Option` unter Mac OS X/11+ an die Taste `Option` und unter Windows sowie X11 an die Taste `Alt` gebunden ist. In diesen Tk-Versionen löst das Neigen des Scrollrads einer Maus mit einer oder zwei zusätzlichen (Daumen-)Tasten nach links oder rechts auf allen Fenstersystemen die Ereignisse `<Shift-MouseWheel>` (und `<Shift-Option-MouseWheel>`) aus.

SCHLÜSSELWÖRTER

mouse wheel event, binding, menubutton, scale widget, scrolling

---

Der Befehl `scrollutil::createWheelEventBindings`

NAME

`scrollutil::createWheelEventBindings` – Erstellen Sie Bindungen für Mausrad-Ereignisse für Widgets der Toplevel-Fenster oder das Tag `all`

ÜBERSICHT

```
scrollutil::createWheelEventBindings ?tag tag ...?
```

ERFORDERLICHE TK-VERSION

8.4 oder höher unter X11 und Mac OS X/11+; 8.6b2 oder höher unter Windows.

BESCHREIBUNG

Erstellt Mausrad- und `<TouchpadScroll>`-Bindungen für die angegebenen Tags, sodass, wenn das Widget unter dem Mauszeiger (ein Nachkomme) eines der scrollbaren Widget-Container ist, der das gleiche Toplevel-Fenster wie das Widget hat und über `scrollutil::enableScrollingByWheel` registriert ist, diese Ereignisse ein Scrollen dieses Widget-Containers auslösen. Falls mehrere verschachtelte, registrierte scrollbare Widget-Container diese Bedingungen erfüllen, wird der innerste davon gescrollt. Jedes `tag` -Argument muss `all` oder der Pfadname eines vorhandenen Toplevel-Fensters (einschließlich `.`) sein.

HINWEIS: Der Grund für die Beschränkung der `tag`-Argumente auf `all` und Pfadnamen bestehender Toplevel-Fenster, anstatt auch Tags wie `"Scrollableframe"` (für scrollutil::scrollableframe), `"BwScrollableFrame"` (für BWidget ScrollableFrame) oder `"Scrolledframe"` (für iwidgets::scrolledframe) besteht darin, dass das Mausrad und `<TouchpadScroll>`-Ereignisse ein Scrollen des Widget-Containers unter dem Mauszeiger nicht nur dann auslösen sollten, wenn das Widget unter dem Mauszeiger der Widget-Container selbst ist, sondern auch, wenn es ein Nachkomme des letzteren ist (Beachten Sie, dass für jedes Fenster der Pfadname seines nächstgelegenen Vorfahren auf Toplevel und das Tag `all` in der Standardliste der Bindungs-Tags des Fensters enthalten sind).

SCHLÜSSELWÖRTER

mouse wheel event, binding, scrolling, scrollable widget container

Der Befehl `scrollutil::enableScrollingByWheel`

NAME

`scrollutil::enableScrollingByWheel` – Registrieren von scrollbaren Widget-Containern für das Scrollen mit dem Mausrad

ÜBERSICHT

```
scrollutil::enableScrollingByWheel ?scrollableWidgetContainer
    scrollableWidgetContainer ...?
```

ERFORDERLICHE TK-VERSION

8.4 oder höher unter X11 und Mac OS X/11+; 8.6b2 oder höher unter Windows.

BESCHREIBUNG

Fügt die angegebenen scrollbaren Widget-Container zur internen Liste der Widget-Container hinzu, die für das Scrollen mit dem Mausrad und für die durch den Befehl `scrollutil::createWheelEventBindings` erstellten `<TouchpadScroll>` -Ereignisbindungen registriert sind.

ANMERKUNG 1: Wenn ein scrollbarer Widget-Container, dessen Pfadname an diesen Befehl übergeben wurde, zerstört wird, wird er automatisch aus der oben genannten internen Liste der registrierten Widget-Container entfernt.

ANMERKUNG 2: Der Befehl `scrollutil::scrollableframe` ruft diesen Befehl automatisch für das von ihm erstellte "scrollableframe"-Widget auf.

SCHLÜSSELWÖRTER

mouse wheel event, binding, scrolling, scrollable widget container

---

Der Befehl `scrollutil::disableScrollingByWheel`

NAME

`scrollutil::disableScrollingByWheel` – Scrollbare Widget-Container für das Scrollen mit dem Mausrad abmelden

ÜBERSICHT

```
scrollutil::disableScrollingByWheel ?scrollableWidgetContainer
    scrollableWidgetContainer ...?
```

ERFORDERLICHE TK-VERSION

8.4 oder höher unter X11 und Mac OS X/11+; 8.6b2 oder höher unter Windows.

BESCHREIBUNG

Entfernt die angegebenen scrollbaren Widget-Container aus der internen Liste der Widget-Container, die für das Scrollen mit dem Mausrad und für die durch den Befehl `scrollutil::createWheelEventBindings` erstellten `<TouchpadScroll>` -Ereignis-Bindungen registriert sind.

SCHLÜSSELWÖRTER

mouse wheel event, binding, scrolling, scrollable widget container

Der Befehl `scrollutil::adaptWheelEventHandling`

NAME

`scrollutil::adaptWheelEventHandling` – Ereignisbehandlung für das Mausrad anpassen

ÜBERSICHT

```
scrollutil::adaptWheelEventHandling ?-ignorefocus? ?widget widget ...?
```

ERFORDERLICHE TK-VERSION

8.4 oder höher unter X11 und Mac OS X/11+; 8.6b2 oder höher unter Windows.

BESCHREIBUNG

Wenn der optionale Schalter `-ignorefocus` *nicht* vorhanden ist, führt der Befehl für jedes `widget`-Argument die folgenden Aktionen aus:

- Wenn `widget` der Pfadname eines tablelist-Widgets ist, werden die beidenOptionen `-xmousewheelwindow` und `-ymousewheelwindow` auf den Pfadnamen des Toplevel-Fensters, vorausgesetzt, die Tablelist-Version ist 6.4 oder höher (bei früheren Tablelist-Versionen ignoriert der Befehl stillschweigend jedes Tablelist-Widget, das ihm als Argument übergeben wird). Infolgedessen löst ein Mausrad- oder `<TouchpadScroll>`-Ereignis über dem Hauptbereich oder dem Bearbeitungsfenster der Tablelist (genauer gesagt: Ein Mausrad- oder `<TouchpadScroll>`-Ereignis, das an eine beliebige Komponente der Tablelist gesendet wird, die das Bindungs-Tag `TablelistBody` oder `TablelistEdit` trägt) wie folgt behandelt:
- Andernfalls wird das (erste) Bindungs-Tag gesucht, das Bindungen für das Mausrad oder `<TouchpadScroll>` aufweist und sich sowohl vom Pfadnamen des Toplevel-Fensters als auch von `all` unterscheidet. Wenn die Suche nach diesem Tag erfolgreich ist, ändert der Befehl die Liste der Bindungs-Tags des Widgets, indem er diesem Bindungs-Tag das Tag `WheeleventRedir` voranstellt und das Tag `WheeleventBreak` anhängt. Infolgedessen wird ein an dieses Widget gesendetes Mausrad- oder `<TouchpadScroll>`-Ereignis wie folgt verarbeitet:
  - Wenn der Fokus innerhalb des "Tablelist"-Widgets liegt, wird durch das Ereignis die "Tablelist" oder deren Bearbeitungsfenster gescrollt, und es erfolgt keine weitere Verarbeitung des Ereignisses.
  - Wenn der Fokus außerhalb des Tablelist-Widgets liegt, erfolgt kein Scrollen des Hauptbereichs oder des Bearbeitungsfensters der Tablelist. Stattdessen wird das Ereignis über `event generate` an das Toplevel-Fenster weitergeleitet. Dies löst wiederum ein Scrollen des (innersten) Widget-Containers aus, der ein Vorfahr von `widget` ist und das gleiche Toplevel-Fenster hat (wenn ein solcher scrollbarer Widget-Container vorhanden ist), vorausgesetzt, der Pfadname des übergeordneten Toplevel-Fensters oder das Bindungs-Tag `all` wurde an den Befehl `scrollutil::createWheelEventBindings` übergeben und dieser Widget-Container wurde über `scrollutil::enableScrollingByWheel` für das Scrollen registriert.
  - Wenn der Fokus auf oder innerhalb des Fensters `[focusCheckWindow widget]` ist, wird das Ereignis durch das diesem Tag zugeordnete Skript der Bindung verarbeitet, und es erfolgt keine weitere Verarbeitung des Ereignisses.
  - Wenn der Fokus außerhalb des Fensters `[focusCheckWindow widget]` ist, wird das

Ereignis über `event generate` an das Toplevel-Fenster weitergeleitet, anstatt vom Skript der Bindung verarbeitet zu werden, das dem oben genannten Tag zugeordnet ist. Dies löst wiederum ein Scrollen des (innersten) Widget-Containers aus, der ein Vorfahr von `widget` ist und das gleiche Toplevel-Fenster hat (wenn ein solcher scrollbarer Widget-Container vorhanden ist), vorausgesetzt, der Pfadname des übergeordneten Toplevel-Fensters oder des Bindungs-Tags `all` wurde an den Befehl `scrollutil::createWheelEventBindings` übergeben und dieser Widget-Container wurde über `scrollutil::enableScrollingByWheel` für das Scrollen registriert.

Wenn die Option `-ignorefocus` festgelegt ist, führt der Befehl für jedes Argument `widget` die folgenden Aktionen aus:

- Wenn `widget` der Pfadname eines tablelist-Widgets ist, setzt er dessen Optionen `-xmousewheelwindow` und `-ymousewheelwindow` auf `""` zurück, vorausgesetzt, die Tablelist-Version ist 6.4 oder höher (bei früheren Tablelist-Versionen ignoriert der Befehl stillschweigend jedes Tablelist-Widget, das ihm als Argument übergeben wird). Folglich führt wenn die Tablelist-Version 6.16 oder höher ist ein Mausrad- oder `<TouchpadScroll>`-Ereignis über dem Hauptbereich oder dem Bearbeitungsfenster der Tablelist zu einem Bildlauf der Tablelist oder ihres Bearbeitungsfensters, ohne dass eine weitere Verarbeitung des Ereignisses stattfindet.
- Andernfalls wird das (erste) Bindungs-Tag gesucht, das Bindungen für das Mausrad oder `<TouchpadScroll>` aufweist und sich sowohl vom Pfadnamen des Toplevel-Fensters als auch von `all` unterscheidet. Wenn die Suche nach diesem Tag erfolgreich ist, ändert der Befehl die Liste der Bindungs-Tags des Widgets, indem er das Tag `WheeleventBreak` an dieses Bindungs-Tag anhängt. Infolgedessen wird ein an dieses Widget gesendetes Mausrad- oder `<TouchpadScroll>`-Ereignis von dem mit diesem Tag verknüpften Skript der Bindung verarbeitet, und es findet keine weitere Verarbeitung des Ereignisses statt.

ANMERKUNG 1: Dieser Befehl ist dafür vorgesehen, für Widgets aufgerufen zu werden, die Bindungen für das Mausrad oder `<TouchpadScroll>` aufweisen und Nachkommen eines scrollbaren Containers für Widgets sind. Die Tk- und Tile-Widgets mit Klassenbindungen für Mausrad- und `<TouchpadScroll>`-Ereignisse sind: listbox, Text, Tk-Core-Scrollbar, ttk::scrollbar, ttk::combobox, ttk::spinbox, ttk::entry, ttk::notebook und ttk::treeview. Beispiele für Widgets mit anderen Bindungs-Tags als ihren Klassennamen, die über Bindungen für Mausrad- und `<TouchpadScroll>`-Ereignisse verfügen, sind ctext- und tablelist-Widgets sowie die Eingabekomponenten von mentry-Widgets vom Typ `"Date"`, `"Time"`, `"DateTime"`, `"IPAddr"` und `"IPv6Addr"` (für Mentry-Versionen 3.2 und höher). (Im Falle eines ctext-Widgets vom Typ `w` beziehen sich die in der obigen BESCHREIBUNG genannten Bindungs-Tags auf das untergeordnete Haupttext-Widget `w.t` des Widgets und nicht auf das Widget selbst.)

ANMERKUNG 2: Wie oben erwähnt, verfügen die Tk-Core-Scrollbar- und ttk::scrollbar-Widgets über Klassenbindungen für Mausrad- und `<TouchpadScroll>`-Ereignisse; daher sollte dieser Befehl für sie aufgerufen werden, falls sie Nachkommen eines scrollbaren Containers sind. Da diese Aufgabe mühsam werden kann, sorgt Scrollutil dafür, dass, wenn Sie diesem Befehl ein Widget übergeben und dieses Widget in ein scrollarea über dessen `setwidget` -Unterbefehl eingebettet ist, dann wird dieser Befehl automatisch auch für die Bildlaufleisten dieses scrollarea aufgerufen.

ANMERKUNG 3: Bei der Verarbeitung eines Mausrad- oder `<TouchpadScroll>` -Ereignisses, das an eine Tk-Core- oder Tile-Bildlaufleiste gesendet wurde, deren Pfadname diesem Befehl ohne die Option `-ignorefocus` übergeben wurde, Wenn der Fokus auf oder innerhalb des zugehörigen Widgets liegt, wird das Ereignis von der Bildlaufleiste verarbeitet und nicht an das übergeordnete Toplevel-Fenster weitergeleitet – genau so, als befände sich der Fokus auf der Bildlaufleiste selbst.

ANMERKUNG 4: Die Bindungen der Mausrad-Ereignisklasse für die Tk-Core-Bildlaufleiste unter Windows und X11 wurden in Tk 8.6 hinzugefügt. Vor dieser Tk-Version gab es solche Bindungen nur für das Fenstersystem `aqua` auf dem Macintosh. Scrollutil beseitigt diese Diskrepanz, indem es automatisch die `Scrollbar`-Klassenbindungen für Mausrad-Ereignisse unter Windows und X11 erstellt. Beachten Sie außerdem, dass das Widget ttk::scrollbar in Tk-Versionen vor 8.7a4 keine *integrierten* Klassenbindungen für Mausrad-Ereignisse hatte, Scrollutil erstellt die fehlenden Bindungen jedoch automatisch, indem es die Ereignisse des Mausrads der Widget-Klasse `Scrollbar` in das Bindungs-Tag `TScrollbar` kopiert.

ANMERKUNG 5: Die Bindungen der Mausrad-Ereignisklassen für das Widget ttk::entry wurden in Tk 9.1a1 hinzugefügt, werden jedoch von Scrollutil automatisch für alle Tk-Versionen ab 8.4 erstellt.

ANMERKUNG 6: Die Ereignisklassen-Bindungen für das Mausrad des Widgets "ttk::notebook" wurden in Tk 8.7a4 hinzugefügt, werden jedoch von Scrollutil automatisch für alle Tk-Versionen ab 8.4 erstellt. Diese Bindungen ermöglichen die Navigation zwischen den Registerkarten eines ttk::notebook- oder scrollutil::scrollednotebook-Widgets über das Mausrad.

ANMERKUNG 7: Es wird empfohlen, diesen Befehl ohne die Option `-ignorefocus` auszuführen, da dies die Wahrscheinlichkeit erheblich verringert, dass der Benutzer versehentlich in einem Fenster innerhalb eines Widget-Containers scrollt, der viele scrollbare Widgets enthält.

SCHLÜSSELWÖRTER

mouse wheel event, binding, event handling, scrolling, scrollable widget container, focus

---

Der Befehl `scrollutil::prepareScrollingByWheel`

NAME

`scrollutil::prepareScrollingByWheel` – Scrollbare Widget-Container für das Scrollen mit dem Mausrad vorbereiten

ÜBERSICHT

```
scrollutil::prepareScrollingByWheel ?-ignorefocus?
    ?scrollableWidgetContainer scrollableWidgetContainer ...?
```

ERFORDERLICHE TK-VERSION

8.4 oder höher unter X11 und Mac OS X/11+; 8.6b2 oder höher unter Windows.

BESCHREIBUNG

Für jeden als Argument angegebenen Container mit scrollbaren Widgets bereitet dieser Befehl das Scrollen mit dem Mausrad und dem Touchpad vor, indem er dessen untergeordnete Elemente, die sich im gleichen Toplevel-Fenster befinden und über Bindungen für Mausrad- oder `<TouchpadScroll>`-Ereignisse verfügen, an `scrollutil::adaptWheelEventHandling` übergibt.

HINWEIS: Wenn ein scrollbarer Widget-Container viele untergeordnete Elemente enthält, ist es wesentlich sicherer, diesen Befehl aufzurufen, als manuell eine große Anzahl von Widgets an `scrollutil::adaptWheelEventHandling` zu übergeben, da die Entscheidung, welche untergeordneten Elemente verarbeitet werden sollen, mühsam und fehleranfällig sein kann.

SCHLÜSSELWÖRTER

mouse wheel event, binding, scrollable widget container

Der Befehl `scrollutil::setFocusCheckWindow`

NAME

`scrollutil::setFocusCheckWindow` – "Fokus-Prüffenster" festlegen

ÜBERSICHT

```
scrollutil::setFocusCheckWindow widget ?widget ...? otherWidget
```

ERFORDERLICHE TK-VERSION

8.4 oder höher.

BESCHREIBUNG

Für jedes `widget`-Argument setzt der Befehl das zugehörige "Fokusprüfungsfenster" auf `otherWidget`. Dies ist das Fenster, das in den Skripten für die Tags `WheeleventRedir` und `Btn2EventRedir` anstelle des Widgets verwendet werden soll, wenn geprüft wird, ob sich der Fokus auf/innerhalb oder außerhalb dieses Fensters befindet. Für jedes `widget`-Argument muss `otherWidget` ein übergeordnetes Element von `widget` sein oder mit diesem identisch sein.

ANMERKUNG 1: Wenn ein Widget, dessen Pfadname diesem Befehl als eines seiner `widget`-Argumente übergeben wurde, zerstört wird, wird die Verknüpfung zwischen dem Widget und seinem "Fokusprüfungsfenster" automatisch aufgehoben.

ANMERKUNG 2: Dieser Befehl ist nützlich, wenn Sie für bestimmte Widgets die Fokusprüfung in den Bindungen für die Tags `WheeleventRedir` und `Btn2EventRedir` weniger restriktiv gestalten möchten. Wenn das Widget unter dem Mauszeiger beispielsweise eine Eingabekomponente eines mentry `me` vom Typ `"Date"`, `"Time"`, `"DateTime"`, `"IPAddr"`, oder `"IPv6Addr"` ist und der Fokus auf einem seiner Geschwister liegt, sollten die an dieses Mentry-Widget gesendeten Mausrad- und `<TouchpadScroll>`-Ereignisse vom Mentry-Widget selbst verarbeitet werden, anstatt den Container zu scrollen, der ein übergeordneter Vorfahr des Mentry ist. Dies können Sie erreichen, indem Sie

```
set entryList [$me entries]
eval scrollutil::adaptWheelEventHandling $entryList
eval scrollutil::setFocusCheckWindow     $entryList [list $me]
```

Ab Tcl/Tk 8.5 können Sie die kompaktere Form verwenden

```
set entryList [$me entries]
scrollutil::adaptWheelEventHandling {*}$entryList
scrollutil::setFocusCheckWindow     {*}$entryList $me
```

ANMERKUNG 3: Nehmen wir als ähnliches Beispiel an, dass `ss` ein scrollsync-Widget ist, das über seinen Unterbefehl `setwidgets` mit untergeordneten Widgets gefüllt wurde. Wenn das Widget unter dem Mauszeiger eines dieser untergeordneten Widgets ist und der Fokus auf einem der anderen an diesen Unterbefehl übergebenen untergeordneten Widgets liegt, sollten die an das unter dem Mauszeiger befindliche untergeordnete Widget gesendeten Mausrad- und `<TouchpadScroll>`-Ereignisse von diesem untergeordneten Widget selbst verarbeitet werden, anstatt den Widget-Container zu scrollen, der ein übergeordnetes Element des scrollsync ist. Dies lässt sich mit dem folgenden Code erreichen:

```
set widgetList [$ss widgets]
eval scrollutil::adaptWheelEventHandling $widgetList
eval scrollutil::setFocusCheckWindow     $widgetList [list $ss]
```

Auch hier gilt: Ab Tcl/Tk 8.5 können Sie die kompaktere Form verwenden

```
set widgetList [$ss widgets]
scrollutil::adaptWheelEventHandling {*}$widgetList
scrollutil::setFocusCheckWindow     {*}$widgetList $ss
```

SCHLÜSSELWÖRTER

binding, focus, "focus check window"

Der Befehl `scrollutil::focusCheckWindow`

NAME

`scrollutil::focusCheckWindow` – Abfrage des "Fokusprüfungsfensters"

ÜBERSICHT

```
scrollutil::focusCheckWindow widget
```

ERFORDERLICHE TK-VERSION

8.4 oder höher.

BESCHREIBUNG

Gibt den Pfadnamen des "Fokusprüfungsfensters" zurück, das dem Argument `widget` zugeordnet ist. Dies ist das Fenster, das in den Skripten für die Tags `WheeleventRedir` und `Btn2EventRedir` anstelle des Widgets verwendet wird, um zu prüfen, ob sich der Fokus auf/innerhalb oder außerhalb dieses Fensters befindet. Wenn der Befehl `scrollutil::setFocusCheckWindow` nicht für `widget` aufgerufen wurde, ist der Wert `widget` selbst.

SCHLÜSSELWÖRTER
binding, focus, "focus check window"

Stichwörter: wheelEvent

Lizenz

Deutsche Übersetzung © 2026 Gregor Ebbing.

Die Übersetzung steht unter denselben Bedingungen wie das jeweilige englische Original. Diese Bedingungen folgen. Die Copyright-Angaben der Originalseiten stehen bei jeder Seite.

Scrolling utilities package Scrollutil 2.11 Copyright (c) 2019-2026 Csaba Nemethi (E-mail: csaba.nemethi@t-online.de)

This library is free software; you can use, modify, and redistribute it for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions.

This software is distributed WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Stichwortverzeichnis

I
index 3

P
pagesman 10
plainnotebook 23

S
scrollableframe 38
scrollarea 46
scrollednotebook 59
scrollsync 64
scrollutil 117

W
wheelEvent 126